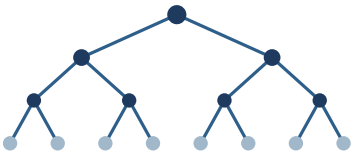
Informatik von Grund auf — Band 1

Die Ordnung der Daten

Eine vollständige Einführung in Algorithmen und Datenstrukturen



Vom Suchen und Sortieren bis zu Graphen und Bäumen
Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 1
Die Ordnung der Daten · Algorithmen und Datenstrukturen

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 1: Die Ordnung der Daten*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 216 Seiten.

Inhaltsverzeichnis

Vorwort **1**

I Grundlagen **9**

1 Aufwand messen **11**
1.1 Die Notation 12
1.2 Rekursionsgleichungen 14
1.3 Amortisierte Analyse 17

2 Felder, Listen und ihre Verwandten **25**
2.1 Stapel und Warteschlange 26

II Ordnen und Suchen **35**

3 Die elementaren Sortierverfahren **37**
3.1 Die drei klassischen Verfahren 38

4 Die schnellen Sortierverfahren **47**
4.1 Mergesort 47
4.2 Quicksort 49
4.3 Heapsort und Timsort 53

5 Suchen und Auswählen **63**
5.1 Binäre Suche und ihre Verwandten 63
5.2 Das Auswahlproblem 65

III Bäume **73**

6 Binäre Suchbäume **75**
6.1 Die Höhe – das ganze Problem 76
6.2 Durchmusterung 78

7 Balancierte Bäume **85**
7.1 AVL-Bäume 86
7.2 Rot-Schwarz-Bäume 87
7.3 B-Bäume 89

8 Heaps und Prioritätswarteschlangen **95**
8.1 Der binäre Heap 96
8.2 Fortgeschrittene Heaps 98

IV Streuspeicherung **105**

9 Hashing **107**
9.1 Hashfunktionen 109
9.2 Kollisionsbehandlung 111

V Graphen **121**

10 Graphen darstellen und durchmustern **123**
10.1 Zwei Darstellungen 124
10.2 Die beiden Durchmusterungen 126

11 Kürzeste Wege **137**
11.1 Der Algorithmus von Dijkstra 138
11.2 Bellman-Ford und Floyd-Warshall 141

12 Spannbäume und Flüsse **149**
12.1 Die Schnitteigenschaft 150
12.2 Flüsse in Netzwerken 153

VI Entwurfsmuster **161**

13 Teile und herrsche, und die gierigen Verfahren **163**
13.1 Teile und herrsche 163
13.2 Gierige Verfahren 165

14 Dynamische Programmierung **175**
14.1 Das Schema 177

VII Aktueller Stand **185**

15 Wo die Algorithmik heute steht **187**
15.1 Die Matrixmultiplikation 187
15.2 Was sich seit 2000 verschoben hat 189

VIIIAnhang **195**

A Übersicht und Formelsammlung **197**
A.1 Die große Tabelle 197
A.2 Formeln und Rekursionen 199
A.3 Die Landkarte des Bandes 200
A.4 Zeittafel 201
A.5 Glossar 204

B Kommentierte Literatur **207**

Vorwort: Warum eine Woche und nicht eine Minute

Es gibt eine Erfahrung, die fast jeder macht, der lange genug programmiert: *Man startet eine Rechnung, geht nach Hause, kommt am nächsten Morgen wieder – und sie läuft noch.*

Der Unterschied, um den es in diesem Band geht

> *Zwei Programme lösen dieselbe Aufgabe. Beide sind richtig. Beide sind etwa gleich lang.* ***Das eine braucht eine Sekunde, das andere eine Woche.***

Der Unterschied liegt nicht in der Sprache, nicht im Rechner und nicht in der Sorgfalt des Programmierers. *Er liegt darin,* ***wie*** *das Problem zerlegt wurde.*

n	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	**Faktor**
100	664	10^4	15
10^4	$1{,}3 \cdot 10^5$	10^8	750
10^6	$2{,}0 \cdot 10^7$	10^{12}	50 000
10^8	$2{,}7 \cdot 10^9$	10^{16}	3,8 Mio.

> ***Bei einer Million Datensätzen trennt ein Faktor 50 000 die beiden Verfahren.*** Das ist der Unterschied zwischen einer Sekunde und vierzehn Stunden.

Und der Faktor wächst weiter. *Kein schnellerer Rechner holt ihn ein: Wer die Rechenleistung verdoppelt, gewinnt beim quadratischen Verfahren nur den Faktor 2 – der Abstand zum besseren Verfahren bleibt.*

Das ist der Grund, warum dieser Band existiert. **Nicht Ästhetik, nicht akademische Gründlichkeit – sondern der Unterschied zwischen „läuft“ und „läuft nicht“.**

Eine Woche Rechenzeit, und was daraus folgte

In den 1990er Jahren gab es in der Hirnforschung ein Projekt namens CoCoMac – *eine Datenbank aller in der Literatur beschriebenen Verbindungen zwischen Arealen der Makakenrinde.*

> *Die Aufgabe: Aus mehreren hundert Einzelstudien, die jeweils andere Parzellierungen des Gehirns benutzten, ein einheitliches Konnektivitätsdiagramm errechnen* – **und dann Fragen daran stellen: Welche Areale liegen zentral? Gibt es Module? Wie lang sind die Wege?**

Das sind Graphenprobleme, *und die Programme dafür wurden in Basic geschrieben, weil die Datenbank in Basic ansprechbar war.*

Ein vollständiger Lauf über die Kartierungen brauchte gelegentlich ***länger als eine Woche****.*

> *Man kann das auf den Interpreter schieben, und ein Teil davon war es auch.* ***Der größere Teil war fast immer die Struktur:*** *eine Adjazenzmatrix, wo eine Liste gereicht hätte; eine dreifach geschachtelte Schleife, wo eine Breitensuche genügt hätte; ein wiederholtes Durchsuchen, wo eine Streuspeicherung in konstanter Zeit geantwortet hätte.*

Der Floyd-Warshall-Algorithmus für alle kürzesten Wege (Kapitel 11) **braucht** $\mathcal{O}(n^3)$ **Schritte.** *Bei* $n = 100$ *Arealen sind das* 10^6 *– selbst in interpretiertem Basic eine Sache von Minuten.* **Wer stattdessen für jedes Paar einzeln sucht und dabei jedes Mal von vorn beginnt, landet bei** $\mathcal{O}(n^5)$ **oder schlimmer** – *und dann sind es Tage.*

> *Die Erfahrung, eine Woche auf ein Ergebnis zu warten, ist die beste Motivation, die es für dieses Fach gibt.* **Sie macht aus der $\mathcal{O}$-Notation etwas Anschauliches.**

Was dieses Buch will

Erstens: Die Analyse ernst nehmen. *Kapitel 1 führt die $\mathcal{O}$-Notation nicht als Schreibweise ein, sondern als Werkzeug* – **mit Rekursionsgleichungen, dem Master-Theorem und der amortisierten Analyse.**

Wer eine Rekursionsgleichung aufstellen und lösen kann, muss nicht mehr raten, ob ein Verfahren skaliert.

Zweitens: Datenstruktur und Algorithmus zusammen denken.

> NIKLAUS WIRTH *nannte sein Lehrbuch 1975 „Algorithmen + Datenstrukturen = Programme“* – **und der Titel ist die These.** *Fast jede Beschleunigung in diesem Band kommt daher, dass die Daten anders* ***abgelegt*** *werden, nicht dass anders gerechnet wird.*

Aufgabe	naiv	mit Struktur
Suchen	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$ oder $\mathcal{O}(1)$
Minimum entnehmen	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$ (Heap)
Mengenvereinigung	$\mathcal{O}(n)$	$\mathcal{O}(\alpha(n))$ (Union-Find)
Kürzeste Wege	$\mathcal{O}(n^3)$	$\mathcal{O}(m + n \log n)$

Drittens: Die Geschichten erzählen.

> *Fast jeder Algorithmus dieses Bandes hat eine Entstehungsgeschichte, und die meisten sind erstaunlich.* **Quicksort entstand aus einem Übersetzungsprojekt Russisch-Englisch, Dijkstras Algorithmus in zwanzig Minuten in einem Amsterdamer Café ohne Bleistift und Papier, und Timsort in der Python-Gemeinde aus der Beobachtung, dass wirkliche Daten fast nie zufällig sind.**

Drei Warnungen

Erstens: Die $\mathcal{O}$-Notation ist ein Werkzeug, kein Urteil.
Ein Verfahren mit $\mathcal{O}(n^2)$ kann in der Praxis das bessere sein – **Einfügesortieren schlägt Quicksort für** $n < 20$, *und deshalb schaltet jede ernsthafte Sortierbibliothek um.* **Kapitel 1 sagt ausdrücklich, wo die Notation lügt.**
Zweitens: Der Code ist Pseudocode.
Ich schreibe in einer bewussten Zwischensprache – lesbar wie Pascal, ohne die Umständlichkeit einer echten Sprache. **Wer es in Python, C oder Go übertragen will, findet die Übersetzung an keiner Stelle schwierig.**
Drittens: Die Speicherhierarchie ist die große Auslassung.

> *Die klassische Analyse zählt Operationen und nimmt an, jeder Speicherzugriff koste gleich viel.* **Das ist seit etwa 1990 falsch** – *ein Cachetreffer und ein Hauptspeicherzugriff unterscheiden sich um den Faktor 100* (Band 0, Kapitel 12).

Ich weise an den Stellen darauf hin, wo es das Ergebnis umkehrt – *etwa bei der Frage, warum verkettete Listen in der Praxis so viel schlechter sind, als die Theorie erwarten lässt.* **Die vollständige Behandlung steht in Band 4.**

Zum Aufbau

- **Teil I** (Kap. 1–2): Aufwandsanalyse mit Rekursionsgleichungen und amortisierter Rechnung; die elementaren Strukturen Feld, Liste, Stapel, Warteschlange.
- **Teil II** (Kap. 3–5): Sortieren – die elementaren Verfahren, die schnellen Verfahren, das Suchen und das Auswahlproblem.
- **Teil III** (Kap. 6–8): Bäume – Suchbäume, Balancierung, Heaps und Prioritätswarteschlangen.
- **Teil IV** (Kap. 9): Streuspeicherung, Kollisionsbehandlung und Bloom-Filter.
- **Teil V** (Kap. 10–12): Graphen – Darstellung und Durchmusterung, kürzeste Wege, Spannbäume und Flüsse.
- **Teil VI** (Kap. 13–14): Die großen Entwurfsmuster – gierige Verfahren und dynamische Programmierung.
- **Teil VII** (Kap. 15): Aktueller Stand, August 2026.

Achtzig Jahre Algorithmik

≈300 v. Chr.	Euklid	ggT-Algorithmus
≈825	al-Chwarizmi	*Algorismi*; Name des Fachs
1844	Lamé	erste Laufzeitanalyse überhaupt
1945	von Neumann	**Mergesort** – erstes Programm für EDVAC
1946	Mauchly	binäre Suche beschrieben
1953	Luhn (IBM)	**Hashing** in einem internen Memo
1954	Kruskal	minimaler Spannbaum
1956	Prim	minimaler Spannbaum (nach Jarník 1930)
1956	Ford, Fulkerson	Max-Flow-Min-Cut
1959	**Dijkstra**	**kürzeste Wege**
1959	Shell	Shellsort
1960/62	**Hoare**	**Quicksort**
1962	Adelson-Velskij, Landis	**AVL-Baum** – erste Balancierung
1962	Floyd, Warshall	alle kürzesten Wege
1964	Williams, Floyd	**Heapsort**, Heap-Struktur
1965	Levenshtein	Editierabstand
1968	Hart, Nilsson, Raphael	A*
1970	Bloom	Bloom-Filter
1970	Bayer, McCreight	**B-Baum** – die Struktur jeder Datenbank
1972	Bayer	Rot-Schwarz-Bäume (als 2-3-4-Bäume)
1972	**Tarjan**	**Tiefensuche als Werkzeug**; Zusammenhangskomponenten
1973	Blum u. a.	Median in linearer Zeit
1975	Tarjan	Union-Find mit Pfadverkürzung
1975	Wirth	*Algorithmen + Datenstrukturen = Programme*
1984	Fredman, Tarjan	Fibonacci-Heap
1985	Sleator, Tarjan	Splay-Bäume; **amortisierte Analyse**
2002	**Tim Peters**	**Timsort** für Python

Die Zeile 1945 verdient Beachtung. JOHN VON NEUMANN *schrieb im Mai 1945 – noch bevor der erste Rechner mit gespeichertem Programm lief – ein Sortierprogramm für den geplanten EDVAC.*

> *Es war Mergesort, in 23 Seiten handschriftlichem Code.* **Der erste nichttriviale Algorithmus der Computergeschichte war ein Sortierverfahren** – *und er ist bis heute in Gebrauch.*

Und die Zeile 1985 ist die theoretisch folgenreichste. ROBERT TARJAN *und* DANIEL SLEATOR *führten die* ***amortisierte Analyse*** *systematisch ein: die Einsicht, dass man nicht jede Einzeloperation abschätzen muss, sondern eine* ***Folge*** *von Operationen im Mittel betrachten darf.*

Damit wurden Strukturen analysierbar, die im Einzelfall langsam, in der Summe aber schnell sind – **und das ist die Mehrzahl der praktisch benutzten.**

Vorzeitige Optimierung ist die Wurzel allen Übels.

— DONALD KNUTH, 1974.
Der vollständige Satz lautet: „Wir sollten kleine Verbesserungen vergessen, sagen wir in 97 Prozent der Fälle: Vorzeitige Optimierung ist die Wurzel allen Übels. Doch sollten wir unsere Chancen in jenen kritischen drei Prozent nicht verspielen.“ Der zweite Teil wird selten mitzitiert.

Part I

Grundlagen

1 Aufwand messen

Was man überhaupt zählt

> *Man könnte Sekunden messen. Das wäre naheliegend und ist fast nutzlos:* **Die Zahl hängt vom Rechner ab, vom Übersetzer, von der Tageszeit, davon, was sonst noch läuft.**

Stattdessen zählt man Elementaroperationen.

Bei Sortierverfahren	Vergleiche und Vertauschungen
Bei Suchverfahren	Vergleiche
Bei Graphenverfahren	besuchte Ecken und Kanten
Bei Arithmetik	Additionen und Multiplikationen

Die Wahl ist eine Modellentscheidung, und man sollte sie aussprechen. **Wer Vergleiche zählt, unterstellt, dass ein Vergleich immer gleich viel kostet** – *bei Zahlen stimmt das, bei langen Zeichenketten nicht.*

Definition 1.1. Für eine Eingabegröße n betrachtet man

$T_{\max}(n)$	**schlechtester Fall** – Maximum über alle Eingaben
$T_{\text{mit}}(n)$	**mittlerer Fall** – Erwartungswert bei angenommener Verteilung
$T_{\min}(n)$	bester Fall – meist uninteressant

Der mittlere Fall braucht eine Annahme – und die ist meist falsch

„Im Mittel“ setzt eine Wahrscheinlichkeitsverteilung über die Eingaben voraus.
Üblicherweise nimmt man an, alle Anordnungen seien gleich wahrscheinlich. **Reale Daten sind das fast nie:**

Namenslisten	meist schon fast alphabetisch
Messreihen	meist schon fast nach Zeit sortiert
Nachträge an eine Datei	sortierter Block am Ende

Genau diese Beobachtung führte 2002 zu ***Timsort*** (Kapitel 4) – **einem Verfahren, das vorsortierte Abschnitte erkennt und ausnutzt.** *Es ist heute der Standard in Python, Java und Android.*

1.1 Die Notation

Definition 1.2. Für Funktionen $f, g : \mathbb{N} \to \mathbb{R}^+$:

Zeichen	Bedeutung	gesprochen
$f = \mathcal{O}(g)$	$\exists c, n_0 : f(n) \leq c\,g(n)$	höchstens
$f = \Omega(g)$	$\exists c, n_0 : f(n) \geq c\,g(n)$	mindestens
$f = \Theta(g)$	beides zugleich	genau
$f = o(g)$	$f(n)/g(n) \to 0$	echt kleiner

(jeweils für alle $n \geq n_0$).

Wie man damit rechnet

Die Regeln, die man braucht:

$\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(\max(f,g))$	Summenregel: der größere gewinnt
$\mathcal{O}(f) \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g)$	Produktregel: geschachtelte Schleifen
$\mathcal{O}(c \cdot f) = \mathcal{O}(f)$	Konstanten fallen weg
$\log_a n = \Theta(\log_b n)$	**die Basis ist gleichgültig**

Zur letzten Zeile, weil sie oft irritiert.

$$\log_a n = \frac{\log_b n}{\log_b a},$$

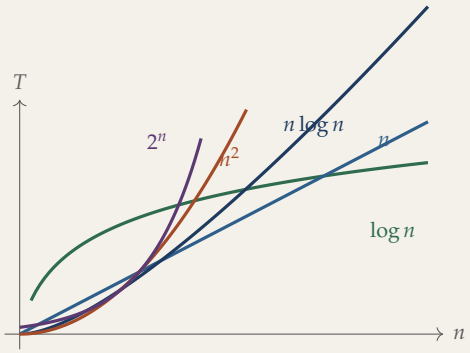
und $\frac{1}{\log_b a}$ *ist eine Konstante* – **die $\mathcal{O}$-Notation schluckt sie.** ✓

> *Deshalb schreibt man in der Algorithmik einfach* $\mathcal{O}(\log n)$ *ohne Basis.* **Bei der *Anzahl* der Vergleiche muss man die Basis natürlich angeben** – $\log_2 10^6 = 20$, *aber* $\log_{10} 10^6 = 6$.

Die Standardhierarchie:

$$1 \prec \log\log n \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

wobei $\prec$ *für „wächst echt langsamer“ steht.*



Das Gleichheitszeichen ist ein Missbrauch

$f = \mathcal{O}(g)$ **ist keine Gleichung.** *$\mathcal{O}(g)$ ist eine **Menge** von Funktionen, und korrekt wäre $f \in \mathcal{O}(g)$.*

Die Schreibweise mit $=$ ist historisch (sie stammt von Edmund Landau, *1909) und bleibt, weil sie sich eingebürgert hat.*

Man merkt den Missbrauch daran, dass sie nicht symmetrisch ist:

$$n = \mathcal{O}(n^2) \quad \text{ist wahr,} \qquad n^2 = \mathcal{O}(n) \quad \text{ist falsch.}$$

Eine Gleichung, die man nicht umdrehen darf, ist keine. **Knuth schlug 1976 vor, $=$ hier als „ist" zu lesen – wie in „Aristoteles ist ein Mensch".**

1.2 Rekursionsgleichungen

Wenn ein Verfahren sich selbst aufruft

Bei rekursiven Verfahren beschreibt man den Aufwand durch eine Gleichung, die sich auf sich selbst bezieht.

Beispiel Mergesort (Kapitel 4): *Man teilt in zwei Hälften, sortiert beide und mischt sie in linearer Zeit.*

$$T(n) = 2\,T\left(\frac{n}{2}\right) + c\,n, \qquad T(1) = c.$$

Verfahren 1 – Der Rekursionsbaum.

Man zeichnet auf, wie viel Arbeit auf jeder Rekursionsebene anfällt.

Ebene	**Teilprobleme**	**Größe je**	**Arbeit**
0	1	n	cn
1	2	$n/2$	$2 \cdot c\frac{n}{2} = cn$
2	4	$n/4$	cn
k	2^k	$n/2^k$	cn
unterste	n	1	cn

***Auf jeder Ebene fällt gleich viel Arbeit an – nämlich** cn**.** Die Teilprobleme werden doppelt so viele und halb so groß; beides hebt sich auf.

Wie viele Ebenen gibt es? Die Größe halbiert sich, bis sie 1 erreicht:

$$\frac{n}{2^k} = 1 \Longrightarrow k = \log_2 n.$$

$$\boxed{T(n) = cn \cdot (\log_2 n + 1) = \Theta(n \log n)}$$

Verfahren 2 – Einsetzen und raten.
Man rät die Lösung und prüft sie durch Einsetzen. **Behauptung:** $T(n) \leq c\, n \log_2 n + cn$.
Induktionsschritt:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn \leq 2\left[c\frac{n}{2}\log_2 \frac{n}{2} + c\frac{n}{2}\right] + cn$$

$$= cn\left(\log_2 n - 1\right) + cn + cn = cn \log_2 n + cn. \qquad \checkmark$$

Die Methode ist zuverlässig, verlangt aber, dass man die Lösung schon kennt – **sie prüft, sie findet nicht.**

Satz 1.3 (Master-Theorem). *Für*

$$T(n) = a\, T\left(\frac{n}{b}\right) + f(n), \qquad a \geq 1,\ b > 1$$

sei $\kappa := \log_b a$. *Dann gilt:*

(1)	$f(n) = \mathcal{O}(n^{\kappa-\varepsilon})$	$T(n) = \Theta(n^{\kappa})$
(2)	$f(n) = \Theta(n^{\kappa})$	$T(n) = \Theta(n^{\kappa} \log n)$
(3)	$f(n) = \Omega(n^{\kappa+\varepsilon})$ *und Regularität*	$T(n) = \Theta(f(n))$

Was das Theorem sagt – ohne Formeln

Es vergleicht zwei Kräfte: **die Arbeit an den Blättern des Rekursionsbaums (n^{κ}) gegen die Arbeit an der Wurzel ($f(n)$).**

Fall	Wer dominiert	Ergebnis
(1)	**die Blätter**	$\Theta(n^{\kappa})$
(2)	**Gleichstand**	$\Theta(n^{\kappa} \log n)$ – ein log kommt hinzu
(3)	**die Wurzel**	$\Theta(f(n))$

Woher $\kappa = \log_b a$ kommt.
Der Baum hat $\log_b n$ Ebenen, und auf Ebene k gibt es a^k Teilprobleme. Die unterste Ebene hat also

$$a^{\log_b n} = n^{\log_b a} = n^{\kappa}$$

Blätter. **Das ist die Arbeit, die allein durch die Aufteilung entsteht.** ✓
(Die Umformung benutzt $a^{\log_b n} = (b^{\log_b a})^{\log_b n} = b^{\log_b a \cdot \log_b n} = n^{\log_b a}$.)
Die Standardbeispiele:

Verfahren	a, b	κ	**Ergebnis**
Binäre Suche	1, 2	0	$\Theta(\log n)$ (Fall 2)
Mergesort	2, 2	1	$\Theta(n \log n)$ (Fall 2)
Karatsuba	3, 2	1,585	$\Theta(n^{1{,}585})$ (Fall 1)
Strassen	7, 2	2,807	$\Theta(n^{2{,}807})$ (Fall 1)
Naive Matrixmult.	8, 2	3	$\Theta(n^3)$ (Fall 2)

Die Zeilen 3 und 4 sind lehrreich: ***Karatsuba und Strassen gewinnen allein dadurch, dass sie a um eins verkleinern*** – eine Multiplikation weniger, dafür mehr Additionen. **Der Gewinn steckt vollständig im Exponenten $\log_b a$.**

Das Theorem hat Lücken

Es deckt nicht alle Fälle ab. *Zwischen Fall 1 und 2 – und zwischen 2 und 3 – klafft eine Lücke, wenn f nur um einen logarithmischen Faktor abweicht.*
Beispiel:

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}.$$

Hier ist $\kappa = 1$, und $\frac{n}{\log n}$ ist weder $\mathcal{O}(n^{1-\varepsilon})$ noch $\Theta(n)$. **Das Master-Theorem schweigt.**

Die Antwort lautet $\Theta(n \log\log n)$ und braucht den Rekursionsbaum. **Eine Verallgemeinerung (Akra-Bazzi, 1998) schließt die Lücken, ist aber unhandlich.**

1.3 Amortisierte Analyse

Wenn selten etwas teuer ist

Manche Strukturen sind fast immer schnell und gelegentlich langsam. **Wer nur den schlechtesten Einzelfall betrachtet, bekommt ein völlig falsches Bild.**

Das Musterbeispiel: das wachsende Feld.
Ein Feld hat feste Größe. Ist es voll und soll ein Element hinzukommen, legt man ein neues an und kopiert um.

normalerweise	Eintragen: $\mathcal{O}(1)$
wenn voll	Neues Feld, alles kopieren: $\mathcal{O}(n)$

Der schlechteste Einzelfall ist $\mathcal{O}(n)$. **Und trotzdem ist das Anhängen an ein dynamisches Feld in jeder Sprache als „konstant" dokumentiert.** *Wieso?*

Definition 1.4. Die **amortisierte Kosten** einer Operation sind die Gesamtkosten einer Folge von m Operationen, geteilt durch m.

Die Rechnung – Verdopplungsstrategie.
Man verdopple die Größe bei jedem Überlauf. Beim Einfügen von n Elementen findet Umkopieren statt bei den Größen

$$1, 2, 4, 8, \ldots, 2^{\lfloor \log_2 n \rfloor}.$$

Die Gesamtkosten des Kopierens betragen

$$1 + 2 + 4 + \cdots + 2^{\lfloor \log_2 n \rfloor} < 2n.$$

Das ist die geometrische Reihe (Mathematik-Band 0, Kapitel 11):
$\sum_{k=0}^{K} 2^k = 2^{K+1} - 1 < 2 \cdot 2^K \leq 2n$. ✓

$$\boxed{\text{Gesamtkosten} < n + 2n = 3n \Longrightarrow \text{amortisiert } \mathcal{O}(1)}$$

Die teuren Operationen sind so selten, dass sie im Mittel nichts kosten. Jede Verdopplung ist doppelt so teuer wie die vorige – aber es liegen auch doppelt so viele billige Operationen dazwischen.

Die drei Methoden der amortisierten Analyse:

Aggregat	Gesamtkosten ausrechnen, durch m teilen (oben benutzt)
Konto	Jede billige Operation zahlt etwas ein; die teure hebt ab
Potential	Eine Funktion Φ misst die „gespeicherte Energie" der Struktur

Die Kontomethode am selben Beispiel.
Jedes Einfügen zahlt 3 Einheiten:

1	für das eigene Eintragen
1	auf das Konto, um sich selbst später umzukopieren
1	auf das Konto, um ein Element der ersten Hälfte mitzukopieren

Beim Verdoppeln von n auf $2n$ müssen n Elemente kopiert werden. Die $\frac{n}{2}$ zuletzt eingefügten haben je 2 Einheiten angespart – **genau n Einheiten.** *Das Konto geht auf und wird nie negativ.* ✓

Warum man verdoppelt und nicht um eine feste Zahl erhöht

Erhöht man die Größe jedes Mal um eine Konstante k, so kopiert man bei den Größen $k, 2k, 3k, \ldots$

$$k + 2k + \cdots + n = \frac{n^2}{2k} + \mathcal{O}(n) = \Theta(n^2).$$

Aus einer konstanten wird eine lineare Operation – **das Einfügen von n Elementen kostet quadratisch statt linear.**

Strategie	Gesamt	amortisiert
$+k$	$\Theta(n^2)$	$\Theta(n)$
$\times 2$	$\Theta(n)$	$\Theta(1)$
$\times 1{,}5$	$\Theta(n)$	$\Theta(1)$

Der Faktor 1,5 statt 2 ist ein Kompromiss beim Speicherverbrauch: Er verschwendet weniger und erlaubt es dem Speicherverwalter eher, freigewordene Blöcke wiederzuverwenden. **Die C++-Standardbibliothek benutzt je nach Implementierung 1,5 oder 2, Python etwa 1,125, Go bis Größe 256 den Faktor 2 und danach etwa 1,25.**

Aufgabe 1.1.

(a) Bestimmen Sie den Aufwand geschachtelter Schleifen und beweisen Sie eine $\mathcal{O}$-Aussage direkt aus der Definition.

(b) Lösen Sie mehrere Rekursionsgleichungen mit verschiedenen Verfahren.

(c) Analysieren Sie einen binären Zähler amortisiert.

Lösung Schritt für Schritt.

(a) Schleifen zählen.

Beispiel 1 – Die einfache Schachtelung.

```
for i := 1 to n do  for j := 1 to n do  x := x+1
```

Die innere Schleife läuft n-mal, die äußere ebenso: $T(n) = n^2 = \Theta(n^2)$. ✓

Beispiel 2 – Die dreieckige Schachtelung.

```
for i := 1 to n do  for j := i to n do  x := x+1
```

Für festes i läuft die innere Schleife $n - i + 1$*-mal:*

$$T(n) = \sum_{i=1}^{n} (n - i + 1) = \sum_{k=1}^{n} k = \frac{n(n+1)}{2}.$$

$$\Longrightarrow T(n) = \Theta(n^2).$$

> ***Die Hälfte der Arbeit ist immer noch quadratisch.*** Das ist der häufigste Irrtum bei der Aufwandsabschätzung: Der Faktor $\frac{1}{2}$ fällt weg, die Größenordnung bleibt.

Beispiel 3 – Die verdoppelnde Schleife.

```
i := 1;  while i < n do { x := x+1;  i := i*2 }
```

i nimmt die Werte $1, 2, 4, \ldots$ *an; die Schleife endet, wenn* $2^k \geq n$*:*

$$T(n) = \lceil \log_2 n \rceil = \Theta(\log n). \qquad ✓$$

Beispiel 4 – Die tückische Kombination.

```
for i := 1 to n do { j := 1;  while j < n do j := j*2 }
```

$$T(n) = n \cdot \Theta(\log n) = \Theta(n \log n). \qquad ✓$$

Beispiel 5 – Wo die Intuition versagt.

```
for i := 1 to n do { j := 1;  while j < i do j := j*2 }
```

$$T(n) = \sum_{i=1}^{n} \log_2 i = \log_2 \Big(\prod_{i=1}^{n} i \Big) = \log_2(n!).$$

Mit der Stirlingformel (Mathematik-Band 15, Kapitel 2):

$$\log_2(n!) \approx n \log_2 n - 1{,}44\, n = \Theta(n \log n). \qquad ✓$$

Dieselbe Größenordnung wie Beispiel 4 – **obwohl die innere Schleife im Mittel nur halb so oft läuft.**

Ein Beweis direkt aus der Definition.
Behauptung: $3n^2 + 7n + 12 = \mathcal{O}(n^2)$.
Gesucht sind c *und* n_0 *mit* $3n^2 + 7n + 12 \leq c\,n^2$ *für* $n \geq n_0$.
Für $n \geq 1$ *gilt* $n \leq n^2$ *und* $1 \leq n^2$, *also*

$$3n^2 + 7n + 12 \leq 3n^2 + 7n^2 + 12n^2 = 22\,n^2.$$

$$\boxed{c = 22,\ n_0 = 1 \text{ leisten es.} \qquad \checkmark}$$

Die Konstante ist grob – man könnte mit $n_0 = 4$ *auch* $c = 5$ *nehmen.* **Das ist gleichgültig: Die Definition verlangt nur die *Existenz* eines Paars, nicht das beste.**

(b) Rekursionsgleichungen lösen.
Gleichung 1 – $T(n) = T(n-1) + c$, $T(1) = c$.
Aufrollen:

$$T(n) = T(n-1) + c = T(n-2) + 2c = \cdots = T(1) + (n-1)c = nc.$$

$$\boxed{T(n) = \Theta(n)}$$

Das ist die lineare Suche, rekursiv geschrieben.
Gleichung 2 – $T(n) = T(n-1) + n$, $T(1) = 1$.

$$T(n) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} = \Theta(n^2).$$

Das ist das Einfügesortieren im schlechtesten Fall (Kapitel 3).
Gleichung 3 – $T(n) = T\left(\frac{n}{2}\right) + c$, $T(1) = c$.

$$T(n) = c \cdot (\log_2 n + 1) = \Theta(\log n).$$

Master-Theorem: $a = 1$, $b = 2$, *also* $\kappa = \log_2 1 = 0$; $f(n) = c = \Theta(n^0)$ – *Fall 2.* ✓

Das ist die binäre Suche. **Bemerkenswert: Man teilt in zwei Hälften, aber *bearbeitet nur eine* – daher** $a = 1$ **und nicht 2.**

Gleichung 4 – $T(n) = 2T\left(\frac{n}{2}\right) + c$, $T(1) = c$.
Jetzt werden ***beide*** *Hälften bearbeitet:* $\kappa = \log_2 2 = 1$, $f(n) = c = \mathcal{O}(n^{1-\varepsilon})$ – **Fall 1.**

$$\boxed{T(n) = \Theta(n)}$$

Das ist ein hübscher Fall: ***Der Aufwand ist linear, obwohl der Baum*** $\log n$ ***Ebenen hat.*** Die Arbeit sitzt fast vollständig in den n Blättern; alle inneren Knoten zusammen tragen nur $n - 1$ bei.

Gleichung 5 – $T(n) = 2T\left(\frac{n}{2}\right) + n^2$.
$\kappa = 1$, $f(n) = n^2 = \Omega(n^{1+\varepsilon})$ – ***Fall 3.*** *Die Regularitätsbedingung* $af(n/b) \le \delta f(n)$ *prüfen:*

$$2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2} \le \frac{1}{2}\,n^2. \qquad \checkmark$$

$$\boxed{T(n) = \Theta(n^2)}$$

Hier dominiert die Wurzel: Die oberste Ebene allein kostet schon n^2*, alles Weitere ist eine geometrische Reihe mit Quotient* $\frac{1}{2}$.

Gleichung 6 – die unangenehme.

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n.$$

Ungleiche Teilung – das Master-Theorem ist nicht anwendbar.

Rekursionsbaum: Auf jeder Ebene beträgt die Arbeit wieder n (die Teilgrößen summieren sich zu n). Die Tiefe ist unterschiedlich:

Pfad	**Schrumpffaktor**	**Tiefe**
immer links	$\frac{1}{3}$	$\log_3 n$
immer rechts	$\frac{2}{3}$	$\log_{3/2} n$

$$\log_{3/2} n = \frac{\ln n}{\ln 1{,}5} = 2{,}466 \ln n, \qquad \log_3 n = 0{,}910 \ln n.$$

Der längste Pfad ist etwa 2,7-mal so lang wie der kürzeste – **aber beide sind** $\Theta(\log n)$**.**

$$\boxed{T(n) = \Theta(n \log n)}$$

> ***Das ist das entscheidende Ergebnis für Quicksort*** (Kapitel 4): **Selbst eine ziemlich schiefe Teilung – ein Drittel zu zwei Dritteln – liefert noch** $\Theta(n \log n)$**.** *Die Aufteilung muss nicht ausgewogen sein, sie darf nur nicht* ***konstant schlecht*** *sein.*

(c) Der binäre Zähler.
Ein k-Bit-Zähler wird n-mal um 1 erhöht. Wie oft kippt insgesamt ein Bit?
Schritt 1 – Der schlechteste Einzelfall.
Von $(0111)_2$ *auf* $(1000)_2$ *kippen alle vier Bits.* **Einzelfall:** $\mathcal{O}(k)$**.**

Naive Abschätzung für n Erhöhungen: $\mathcal{O}(nk)$. **Das ist richtig, aber viel zu pessimistisch.**

Schritt 2 – Genau hinsehen.

Wert	binär	Kippungen	Summe
0	0000	—	0
1	0001	1	1
2	0010	2	3
3	0011	1	4
4	0100	3	7
5	0101	1	8
6	0110	2	10
7	0111	1	11
8	1000	**4**	15

Nach 8 Erhöhungen: 15 Kippungen – **knapp 2 pro Erhöhung, nicht 4.**

Schritt 3 – Die Aggregatmethode.

*Man zählt **nach Bitposition** statt nach Erhöhung.*

Bit	kippt bei jeder	Anzahl in n Schritten
0 (unterstes)	Erhöhung	n
1	zweiten	$\lfloor n/2 \rfloor$
2	vierten	$\lfloor n/4 \rfloor$
i	2^i-ten	$\lfloor n/2^i \rfloor$

Gesamtzahl:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \;<\; n \sum_{i=0}^{\infty} \frac{1}{2^i} = n \cdot \frac{1}{1-\frac{1}{2}} = 2n.$$

Amortisiert weniger als 2 Kippungen pro Erhöhung $= \mathcal{O}(1)$

Von $\mathcal{O}(k)$ ***auf*** $\mathcal{O}(1)$ – und der ganze Beweis ist eine geometrische Reihe.
Der Trick bestand darin, in der anderen Richtung zu summieren.

Schritt 4 – Dieselbe Rechnung mit der Kontomethode.
Regel: Wer ein Bit von 0 auf 1 setzt, zahlt **2** *Einheiten – eine für das Setzen, eine aufs Konto des Bits.*

*Beim Erhöhen wird **genau ein** Bit auf 1 gesetzt (kostet 2), und eine Reihe von Einsen wird auf 0 zurückgesetzt.* **Jedes dieser Rücksetzungen wird aus dem Konto bezahlt, das beim Setzen angelegt wurde.** ✓

$$\Longrightarrow \text{amortisierte Kosten} = 2 = \mathcal{O}(1). \qquad \checkmark$$

Das Konto ist stets nichtnegativ – es enthält genau so viele Einheiten, wie der Zähler gesetzte Bits hat. **Das ist die Potentialfunktion** $\Phi = \text{Anzahl der Einsen}$.

Amortisiert ist nicht „im Mittel“

Der Unterschied ist wesentlich und wird oft verwischt.

im Mittel	Aussage über eine **Wahrscheinlichkeitsverteilung** der Eingaben; kann im Einzelfall schiefgehen
amortisiert	Aussage über eine **beliebige Folge** von Operationen; gilt **garantiert**

*Die amortisierte Schranke enthält keinen Zufall. Es gibt **keine** Eingabefolge, bei der n Erhöhungen mehr als $2n$ Kippungen kosten* – **das ist bewiesen, nicht erwartet.**

Und trotzdem gibt es eine Einschränkung, die in der Praxis zählt: *Die amortisierte Garantie sagt nichts über die **einzelne** Operation.* **Bei Systemen mit harten Zeitschranken – Flugsteuerung, Herzschrittmacher – ist eine gelegentliche $\mathcal{O}(n)$-Operation nicht hinnehmbar**, *auch wenn sie amortisiert billig ist.*

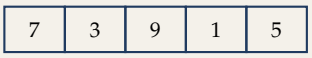
*Dafür gibt es **echtzeitfähige** Varianten, die die Arbeit gleichmäßig verteilen* – **etwa inkrementelles Umkopieren, bei dem jede Einfügung zwei alte Elemente mitnimmt.** *Sie sind langsamer im Mittel und schneller im Maximum.*

Cormen, Leiserson, Rivest und Stein[6] behandeln die amortisierte Analyse in drei Varianten mit vielen Beispielen; Sedgewick und Flajolet[10] entwickeln die exakte Analyse mit erzeugenden Funktionen; Graham, Knuth und Patashnik[13] liefern das mathematische Handwerkszeug.

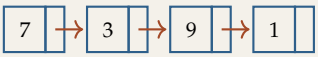
2 Felder, Listen und ihre Verwandten

Zwei Grundformen, alles andere folgt

Eine Folge von Daten lässt sich auf zwei grundsätzlich verschiedene Arten im Speicher ablegen – **zusammenhängend oder verkettet.** *Aus dieser einen Entscheidung folgt fast alles Weitere.*



Feld – zusammenhängend



Liste – verkettet

Operation	**Feld**	**verkettete Liste**
Zugriff auf i-tes	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Einfügen am Anfang	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Einfügen am Ende	$\mathcal{O}(1)$ amort.	$\mathcal{O}(1)$
Einfügen in der Mitte	$\mathcal{O}(n)$	$\mathcal{O}(1)^*$
Löschen	$\mathcal{O}(n)$	$\mathcal{O}(1)^*$
Speicherbedarf	n Werte	n Werte + n Zeiger

* *sofern man den Vorgängerknoten bereits kennt – sonst kommt die Suche hinzu.*

Diese Tabelle ist theoretisch richtig und praktisch irreführend

Sie zählt Operationen und unterstellt, dass jeder Speicherzugriff gleich viel kostet. *Seit etwa 1990 ist das falsch* (Band 0, Kapitel 12).

Zugriff	Zeit	Verhältnis
L1-Cache	1 ns	1
Hauptspeicher	80 ns	**80**

Ein Feld liegt am Stück. Wer es durchläuft, holt mit jedem Cachezugriff gleich 8 bis 16 Nachbarwerte mit – **der Prozessor erkennt das Muster und lädt sogar vorausschauend.**

Eine verkettete Liste liegt verstreut. Jeder Schritt ist ein Sprung an eine unvorhersagbare Adresse – **jedes Mal ein Cachefehler.**

Die Folge ist drastisch: *Beim Einfügen in der **Mitte** einer Folge von 100 000 Elementen ist ein Feld in der Praxis oft **schneller** als eine Liste* – **obwohl es** $\mathcal{O}(n)$ **statt** $\mathcal{O}(1)$ **kostet.**

Denn das Umkopieren von 50 000 zusammenhängenden Werten ist ein einziger schneller Blockdurchlauf, während das Auffinden der Einfügestelle in der Liste 50 000 Cachefehler bedeutet.

BJARNE STROUSTRUP *führte das um 2012 in Vorträgen vor und formulierte die Faustregel:* **„Benutze Felder, bis du gemessen hast, dass etwas anderes besser ist.“**

Verkettete Listen bleiben trotzdem wichtig – *als Bausteine innerhalb anderer Strukturen (Hashtabellen, Freispeicherlisten), wenn Zeiger von außen stabil bleiben müssen, und in funktionalen Sprachen, wo Unveränderlichkeit erzwungen ist.*

2.1 Stapel und Warteschlange

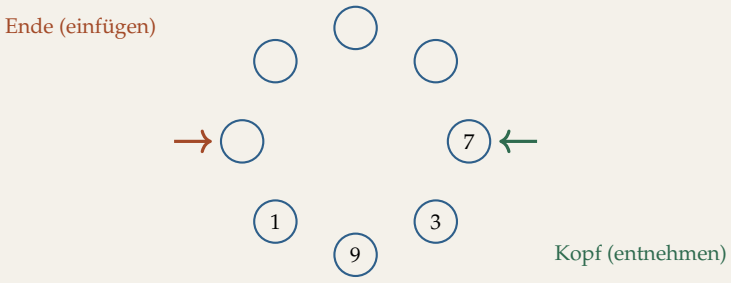
Zwei Disziplinen, ein Speicher

Beide halten eine Menge von Elementen und geben auf Verlangen eines heraus. ***Der Unterschied ist nur, welches.***

	Stapel	Warteschlange
Prinzip	**LIFO**	**FIFO**
Herausgabe	zuletzt eingefügtes	zuerst eingefügtes
Operationen	`push`, `pop`	`enqueue`, `dequeue`
Bild	Tellerstapel	Warteschlange an der Kasse
Anwendung	Rekursion, Klammern, Rückverfolgung	Pufferung, **Breitensuche**

Beide Operationen kosten $\mathcal{O}(1)$ – bei geeigneter Implementierung. Beim Stapel ist das offensichtlich; bei der Warteschlange braucht es einen Kunstgriff.

Die Warteschlange im Feld: der Ringpuffer.
Naiv würde man beim Entnehmen alle Elemente nach vorn schieben – $\mathcal{O}(n)$. **Stattdessen merkt man sich zwei Zeiger und lässt sie im Kreis laufen.**




```
kopf := (kopf + 1) mod N,    ende := (ende + 1) mod N.
```

Die Modulorechnung schließt das Feld zum Ring. **Beide Operationen:** $\mathcal{O}(1)$**.** ✓

Voll oder leer? – ein klassischer Stolperstein

Sind Kopf und Ende gleich, kann der Puffer leer *oder* voll sein.

Beide Zustände sind an den Zeigern nicht unterscheidbar.

Drei übliche Auswege:

Ein Platz frei lassen	Voll heißt `(ende+1) mod N = kopf`; kostet einen Platz
Zähler mitführen	Braucht ein zusätzliches Feld und dessen Pflege
Zeiger nicht kürzen	Kopf und Ende laufen unbegrenzt; Modulo nur beim Zugriff

Die dritte Lösung ist die eleganteste und wird in nebenläufigen Ringpuffern bevorzugt – **man muss dann nur auf den Überlauf der Zähler achten** (Band 0, Kapitel 2).

Zwei Stapel ergeben eine Warteschlange

Eine hübsche Konstruktion, die zugleich ein Musterbeispiel amortisierter Analyse ist.

`enqueue(x)`	Lege x auf Stapel **E** (Eingang)
`dequeue()`	Ist **A** leer, kippe ganz **E** nach **A** um; nimm dann von **A**

Das Umkippen dreht die Reihenfolge um – aus LIFO wird FIFO. **Und weil zweimal Umdrehen die ursprüngliche Reihenfolge wiederherstellt, stimmt es.**

Die Analyse.
Der schlechteste Einzelfall ist $\mathcal{O}(n)$ – das Umkippen. **Aber jedes Element wird höchstens *einmal* umgekippt:**

Jedes Element wird	einmal auf E gelegt, einmal von E genommen, einmal auf A gelegt, einmal von A genommen

$\Longrightarrow$ 4 Operationen pro Element $\Longrightarrow \mathcal{O}(1)$ amortisiert. ✓

> *Diese Konstruktion ist in funktionalen Sprachen der Standardweg zu einer Warteschlange* – **denn dort gibt es keine veränderbaren Felder, aber unveränderliche Listen als Stapel.** CHRIS OKASAKI *hat solche Strukturen systematisch untersucht*[21].

Aufgabe 2.1.

(a) Kehren Sie eine verkettete Liste in $\mathcal{O}(n)$ Zeit und $\mathcal{O}(1)$ zusätzlichem Speicher um.

(b) Erkennen Sie einen Zyklus in einer Liste – und finden Sie seinen Anfang.

(c) Prüfen Sie einen Ausdruck auf korrekte Klammerung und werten Sie ihn aus.

Lösung Schritt für Schritt.

(a) Liste umkehren.

Schritt 1 – Die Idee.

> *Man läuft einmal durch und dreht dabei jeden Zeiger um.* **Dazu braucht man drei Merker: den Vorgänger, den aktuellen Knoten und den Nachfolger** *– denn sobald man den Zeiger umdreht, ist der Nachfolger verloren.*

Schritt 2 – Das Verfahren.

```
vor := nil;  akt := kopf;
while akt <> nil do {
  nach := akt.next;      Nachfolger retten
  akt.next := vor;       Zeiger umdrehen
  vor := akt;
  akt := nach; }
kopf := vor;
```

Schritt 3 – Ein Durchlauf, Schritt für Schritt.

Liste $7 \to 3 \to 9 \to$*nil:*

Runde	`vor`	`akt`	**Liste danach**
Start	nil	7	$7 \to 3 \to 9$
1	7	3	$7 \to$nil, $3 \to 9$
2	3	9	$3 \to 7 \to$nil, 9
3	9	nil	$9 \to 3 \to 7 \to$nil

Neuer Kopf: 9 ✓

Schritt 4 – Die Invariante.

(I) `vor` zeigt auf die bereits umgedrehte Teilliste, `akt` auf den Rest.

Anfangs ist die umgedrehte Teilliste leer (nil) und der Rest die ganze Liste. ✓
Der Schritt hängt genau ein Element vorn an die umgedrehte Liste an. ✓
Am Ende ist der Rest leer, also ist alles umgedreht. ✓
Schritt 5 – Aufwand.

$\mathcal{O}(n)$ Zeit, $\mathcal{O}(1)$ zusätzlicher Speicher. ✓

Drei Zeiger, unabhängig von n – **deshalb $\mathcal{O}(1)$.** *Die rekursive Fassung dagegen braucht $\mathcal{O}(n)$ Stapelspeicher.*

(b) Zyklus erkennen – der Hase und die Schildkröte.

Eine fehlerhaft verkettete Liste kann einen Zyklus enthalten. Ein naiver Durchlauf läuft dann ewig. **Wie erkennt man das mit $\mathcal{O}(1)$ Speicher?**

Schritt 1 – Die Idee von Floyd (1967).
Zwei Zeiger laufen los: einer mit Schrittweite 1 („Schildkröte"), einer mit Schrittweite 2 („Hase").

Kein Zyklus	Der Hase erreicht das Ende – fertig
Zyklus	Der Hase holt die Schildkröte ein – sie treffen sich

Schritt 2 – Warum sie sich treffen müssen.
Sind beide im Zyklus (der Hase kommt zuerst hinein, die Schildkröte folgt), so verringert sich ihr Abstand im Zyklus in ***jedem*** *Schritt um genau 1 –* **denn der Hase nähert sich mit Differenzgeschwindigkeit $2 - 1 = 1$.**

Ein Abstand, der bei jedem Schritt um 1 fällt und im Zyklus der Länge λ modulo λ gerechnet wird, erreicht nach höchstens λ Schritten den Wert 0. ✓

Wichtig ist, dass die Differenz genau 1 beträgt. Liefe der Hase mit Schrittweite 3 und die Schildkröte mit 1, wäre die Differenz 2 – und bei ungeradem λ könnte er über sie hinwegspringen. *Er würde sie trotzdem treffen, aber der Beweis wird umständlicher.*

Schritt 3 – Den Zyklusanfang finden.

Das ist der eigentlich schöne Teil.

Sei μ die Länge des Vorlaufs (bis zum Zykluseingang) und λ die Zykluslänge. Beim Treffen gilt:

Schildkröte hat	s Schritte gemacht
Hase hat	$2s$ Schritte gemacht
Differenz	s Schritte – und das muss ein Vielfaches von λ sein

$$s \equiv 0 \pmod{\lambda}.$$

Die Schildkröte steht also s Schritte vom Anfang entfernt, und s ist ein Vielfaches der Zykluslänge.

> ***Jetzt setzt man einen Zeiger auf den Listenanfang zurück und lässt beide mit Schrittweite 1 laufen.***

Nach μ Schritten steht der zurückgesetzte Zeiger am Zykluseingang. Der andere steht dann bei $s + \mu$ – und da s ein Vielfaches von λ ist, ist auch das der Zykluseingang.

Sie treffen sich genau am Zyklusanfang. ✓

Schritt 4 – Aufwand.

$$\mathcal{O}(\mu + \lambda) = \mathcal{O}(n) \text{ Zeit}, \qquad \mathcal{O}(1) \text{ Speicher.} \quad ✓$$

> ***Die naive Lösung – alle besuchten Knoten in einer Menge merken – braucht $\mathcal{O}(n)$ Speicher.*** Floyds Verfahren braucht zwei Zeiger. **Das ist der ganze Unterschied.**

Das Verfahren hat unerwartete Anwendungen: *Es findet die Periode jedes deterministischen Prozesses, der einen Zustand in den nächsten überführt* – **Zufallszahlengeneratoren, Pollards ρ-Verfahren zur Faktorisierung** (Band 8), *Zyklen in Funktionsiterationen.*

(c) Klammern prüfen und auswerten.
Teil 1 – Die Prüfung.

```
for jedes Zeichen c do
  if c ist öffnend then push(c)
  else if c ist schließend then {
    if Stapel leer then return falsch;
    if pop() passt nicht zu c then
return falsch }
return (Stapel ist leer)
```

Beispiel `{[()]}`:

Zeichen	Aktion	Stapel danach
`{`	push	`{`
`[`	push	`{[`
`(`	push	`{[(`
`)`	pop → `(` ✓	`{[`
`]`	pop → `[` ✓	`{`
`}`	pop → `{` ✓	leer

Stapel leer $\Longrightarrow$ korrekt. ✓

Gegenbeispiel `([)]`: *Bei* `)` *liegt* `[` *obenauf – passt nicht.* **Falsch.** ✓

> ***Warum ein Stapel die richtige Struktur ist:*** *Klammerung ist* ***geschachtelt****, und geschachtelt heißt: Was zuletzt geöffnet wurde, muss zuerst geschlossen werden.* **Das ist genau LIFO.**

Teil 2 – Auswertung in umgekehrter polnischer Notation.
In der ***Postfixnotation*** *steht der Operator hinter seinen Operanden:*

$$3\ 4\ +\ 5\ *\qquad \text{bedeutet} \qquad (3+4)\cdot 5 = 35.$$

Auswertung mit einem Stapel:

Symbol	Aktion	Stapel
3	push	3
4	push	3, 4
+	pop 4, pop 3, push 7	7
5	push	7, 5
*	pop 5, pop 7, push 35	**35**

> ***Keine Klammern, keine Vorrangregeln, kein Rückverfolgen.*** Ein einziger Durchlauf, $\mathcal{O}(n)$, mit einem Stapel als einzigem Hilfsmittel.

Die Geschichte dazu. JAN ŁUKASIEWICZ *erfand 1924 die klammerfreie Präfixschreibweise – daher „polnische Notation“.* **CHARLES HAMBLIN drehte sie 1957 um und schlug die Postfixform ausdrücklich für die Auswertung mit einem *Kellerspeicher* vor.**

Der Taschenrechner HP-35 (1972) und alle folgenden HP-Modelle rechneten so. **Ihre Benutzer bildeten eine erstaunlich treue Gemeinde** – *unter Ingenieuren galt lange als Ausweis der Zunft, wer ohne Klammertasten rechnen konnte.*

Und die Umwandlung von Infix in Postfix leistet der Shunting-Yard-Algorithmus *von* EDSGER DIJKSTRA *(1961)* – **ebenfalls mit einem Stapel, ebenfalls in $\mathcal{O}(n)$.** *Der Name kommt vom Rangierbahnhof: Operatoren werden auf ein Nebengleis geschoben und in der richtigen Reihenfolge wieder eingefädelt.*

Er steckt bis heute in jedem Taschenrechner und in der Ausdrucksverarbeitung jedes Übersetzers (Band 6). Bentley[2] und Skiena[25] zeigen viele solcher Aufgaben mit dem Blick des Praktikers; Wirth[28] entwickelt die Grundstrukturen mit besonderer Strenge.

Part II

Ordnen und Suchen

3 Die elementaren Sortierverfahren

Warum Sortieren das Musterproblem ist

> *Kein anderes Problem der Informatik ist so gründlich untersucht.* **Knuth widmet ihm einen halben Band von 400 Seiten**[17], *und es gibt Dutzende von Verfahren, die alle dasselbe leisten.*

Die Gründe dafür sind praktisch.

Häufigkeit	Nach alten Schätzungen entfielen bis zu einem Viertel aller Rechenzeit auf Sortieren
Voraussetzung	Binäre Suche, Verschmelzen, Duplikatsuche, Median – alles setzt Sortierung voraus
Musterfall	Fast jede Entwurfstechnik lässt sich am Sortieren zeigen

Definition 3.1. Ein Sortierverfahren heißt

stabil	wenn gleiche Schlüssel ihre ursprüngliche Reihenfolge behalten
in situ	wenn es nur $\mathcal{O}(1)$ zusätzlichen Speicher braucht
adaptiv	wenn es bei vorsortierten Daten schneller ist

Warum Stabilität zählt

Eine Tabelle sei bereits nach Vornamen sortiert. Nun sortiert man sie nach Nachnamen.

vorher	stabil	instabil
Anna Meier	Anna Meier	Clara Meier
Clara Meier	Clara Meier	Anna Meier
Bert Schulz	Bert Schulz	Bert Schulz

Bei einem stabilen Verfahren ist die Tabelle danach nach Nachname* und*, innerhalb gleicher Nachnamen, nach Vorname sortiert. Man kann also mehrstufig sortieren, indem man einfach nacheinander nach den Kriterien sortiert – von hinten nach vorn.

Genau darauf beruht das Radixsortieren (Kapitel 4). *Ohne Stabilität funktioniert es nicht.*

3.1 Die drei klassischen Verfahren

Auswahlsortieren

Das Verfahren, das jeder von selbst erfindet: Suche das Kleinste, tausche es nach vorn, wiederhole.

```
for i := 1 to n-1 do {
  min := i;
  for j := i+1 to n do if a[j] < a[min]
then min := j;
  vertausche a[i] und a[min] }
```

Die Analyse.

$$\text{Vergleiche} = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = \Theta(n^2).$$

$$\text{Vertauschungen} = n - 1 = \Theta(n).$$

Die Vergleichszahl ist* immer *dieselbe – bester, mittlerer und schlechtester Fall sind identisch. **Das Verfahren ist nicht adaptiv: Eine bereits sortierte Folge kostet genauso viel.**

Sein einziger Vorzug: *die minimale Zahl von Vertauschungen.* **Wenn ein Datensatz sehr groß und ein Vergleich sehr billig ist – etwa Sortieren nach einem kurzen Schlüssel in einer Struktur von 4 Kilobyte –, kann das den Ausschlag geben.**

Auswahlsortieren ist nicht stabil

Beispiel: $[\,2_a,\ 2_b,\ 1\,]$.
Der erste Schritt vertauscht die 1 mit der vordersten 2:

$$[\,1,\ 2_b,\ 2_a\,].$$

Die beiden Zweien haben ihre Reihenfolge getauscht. **Instabil.** ✓

Man kann es stabil machen, indem man statt zu vertauschen alles um eine Stelle verschiebt – **dann kostet es aber $\Theta(n^2)$ Bewegungen und verliert seinen einzigen Vorzug.**

Einfügesortieren

Das Verfahren, mit dem jeder Kartenspieler sein Blatt ordnet: Nimm die nächste Karte und schiebe sie an die richtige Stelle der schon geordneten.

```
for i := 2 to n do {
  x := a[i];  j := i-1;
  while j >= 1 and a[j] > x do { a[j+1]
:= a[j];  j := j-1 }
  a[j+1] := x }
```

Die Invariante:

Vor jedem Durchlauf ist $a[1 \ldots i-1]$ sortiert

und enthält dieselben Werte wie zu Beginn.

Anfang: $a[1 \ldots 1]$ *ist trivial sortiert.* ✓
Schritt: Die `while`*-Schleife schiebt alle Elemente, die größer als* x *sind, um eine Stelle nach rechts, und setzt* x *in die entstandene Lücke.* ✓
Schluss: Bei $i = n + 1$ *ist* $a[1 \ldots n]$ *sortiert.* ✓
Die Analyse – und hier wird es interessant.

Fall	Vergleiche	Eingabe
bester	$n-1=\Theta(n)$	**bereits sortiert**
mittlerer	$\approx \frac{n^2}{4}=\Theta(n^2)$	zufällig
schlechtester	$\frac{n(n-1)}{2}=\Theta(n^2)$	**umgekehrt sortiert**

Der beste Fall ist linear. Bei einer sortierten Folge bricht die innere Schleife sofort ab. **Das Verfahren ist *adaptiv*** – und das ist sein entscheidender Vorzug.

Satz 3.2. *Einfügesortieren braucht $\Theta(n+I)$ Zeit, wobei I die Zahl der **Inversionen** ist – also der Paare (i,j) mit $i<j$ und $a[i]>a[j]$.*

Denn jede Verschiebung in der inneren Schleife beseitigt genau eine Inversion. ✓

Eingabe	Inversionen	Zeit
sortiert	0	$\Theta(n)$
ein Element falsch	$\mathcal{O}(n)$	$\Theta(n)$
k-fast-sortiert	$\mathcal{O}(kn)$	$\Theta(kn)$
zufällig	$\approx \frac{n^2}{4}$	$\Theta(n^2)$
umgekehrt	$\frac{n(n-1)}{2}$	$\Theta(n^2)$

Das ist der Grund, warum Einfügesortieren nicht ausstirbt. Es ist stabil, in situ, adaptiv, hat winzige Konstanten und ist auf kleinen Feldern schneller als jedes $\mathcal{O}(n \log n)$-Verfahren.

Jede ernsthafte Sortierbibliothek benutzt es – als Nachbehandlung für kleine Teilfelder und als Baustein von Timsort (Kapitel 4).

Blasensortieren – und warum man es nicht benutzen sollte

Man vergleicht benachbarte Elemente und vertauscht sie, wenn sie in falscher Reihenfolge stehen. Das wiederholt man, bis nichts mehr zu tun ist.

```
repeat { getauscht := falsch;
  for j := 1 to n-1 do
    if a[j] > a[j+1] then { vertausche;
getauscht := wahr } }
until not getauscht
```

	Blasensortieren	Einfügesortieren
Vergleiche (mittel)	$\frac{n^2}{2}$	$\frac{n^2}{4}$
Vertauschungen	$\frac{n^2}{4}$	$\frac{n^2}{4}$ (Verschiebungen)
bester Fall	$\Theta(n)$	$\Theta(n)$
stabil	ja	ja

Es ist in jeder Hinsicht schlechter als Einfügesortieren – doppelt so viele Vergleiche bei gleicher Bewegungszahl, und es hat keinen einzigen Vorzug, den das andere nicht auch hätte.

Knuths Urteil

DONALD KNUTH schrieb 1973:

> *„Es scheint, dass das Blasensortieren nichts empfiehlt außer seinem eingängigen Namen und der Tatsache, dass es zu einigen interessanten theoretischen Problemen führt."*

Der Name stammt daher, dass große Elemente wie Blasen nach oben steigen. **Er ist der Grund, warum das Verfahren in Lehrbüchern überlebt hat – nicht seine Eigenschaften.**

BARACK OBAMA wurde 2007 im Wahlkampf von ERIC SCHMIDT scherzhaft nach dem effizientesten Weg gefragt, eine Million Zahlen zu sortieren. *Seine Antwort: „Ich denke, Blasensortieren wäre der falsche Weg."* **Das Publikum lachte; die Antwort war korrekt.**

Shellsort – das übersehene Verfahren

> DONALD SHELL *beobachtete 1959: Einfügesortieren ist langsam, weil Elemente nur um* ***eine*** *Stelle wandern können. Ein Element, das ganz hinten steht und nach vorn gehört, braucht n Verschiebungen.*

Seine Idee: *Man sortiert zuerst mit* ***großer*** *Schrittweite h – also die Teilfolgen* $a[1], a[1+h], a[1+2h], \ldots$ *jeweils für sich – und verringert h dann schrittweise bis auf 1.*

```
for jedes h in der Schrittweitenfolge
(absteigend, endet mit 1) do
  for i := h+1 to n do {
    x := a[i];  j := i-h;
    while j >= 1 and a[j] > x do {
a[j+h] := a[j];  j := j-h }
    a[j+h] := x }
```

> *Der letzte Durchlauf mit* $h = 1$ *ist gewöhnliches Einfügesortieren –* **aber auf einer Folge, die bereits fast sortiert ist.** *Und dort ist Einfügesortieren fast linear.*

Der Aufwand hängt von der Schrittweitenfolge ab – und das ist ein bis heute offenes Problem.

Folge	schlechtester Fall	Urheber
$n/2, n/4, \ldots, 1$	$\Theta(n^2)$	Shell 1959
$2^k - 1$	$\Theta(n^{3/2})$	Hibbard 1963
$2^p 3^q$	$\Theta(n \log^2 n)$	Pratt 1971
$1, 5, 19, 41, 109, \ldots$	$\mathcal{O}(n^{4/3})$	Sedgewick 1986
optimale Folge	**unbekannt**	—

> ***Nach über sechzig Jahren weiß niemand, welche Schrittweitenfolge optimal ist*** – und auch nicht, ob $\mathcal{O}(n \log n)$ mit Shellsort überhaupt erreichbar ist. **Es ist eines der ältesten offenen Probleme der Algorithmik.**

In der Praxis ist Shellsort selten geworden, hat aber eine Nische: **eingebettete Systeme**, *wo es keinen Platz für Rekursion oder Zusatzspeicher gibt, das Verfahren aber deutlich schneller als* $\mathcal{O}(n^2)$ *sein soll.* **Es ist in situ, braucht keine Rekursion und passt in wenige Zeilen.**

Aufgabe 3.1.

(a) Sortieren Sie eine konkrete Folge mit allen drei Verfahren und zählen Sie die Operationen.

(b) Bestimmen Sie die mittlere Zahl der Inversionen einer zufälligen Permutation.

(c) Zeigen Sie, dass jedes Verfahren, das nur benachbarte Elemente vertauscht, $\Omega(n^2)$ braucht.

Lösung Schritt für Schritt.

(a) Eine Folge, drei Verfahren.
Wir sortieren $[5,\ 2,\ 8,\ 1,\ 9,\ 3]$.
Auswahlsortieren.

i	**Feld**	**Minimum**	**Vergl.**
1	5, 2, 8, 1, 9, 3	1 (Pos. 4)	5
2	1, 2, 8, 5, 9, 3	2 (Pos. 2)	4
3	1, 2, 8, 5, 9, 3	3 (Pos. 6)	3
4	1, 2, 3, 5, 9, 8	5 (Pos. 4)	2
5	1, 2, 3, 5, 9, 8	8 (Pos. 6)	1
	1, 2, 3, 5, 8, 9		**15**

Probe: $\frac{6\cdot 5}{2} = 15$ ✓ – **genau wie die Formel sagt, unabhängig von der Eingabe.**
Einfügesortieren.

i	x	**Feld danach**	**Verschieb.**
2	2	2, 5, 8, 1, 9, 3	1
3	8	2, 5, 8, 1, 9, 3	0
4	1	1, 2, 5, 8, 9, 3	3
5	9	1, 2, 5, 8, 9, 3	0
6	3	1, 2, 3, 5, 8, 9	3
			7

Die Inversionen zählen – zur Probe.
Paare (i, j) *mit* $i < j$ *und* $a[i] > a[j]$ *in* $[5, 2, 8, 1, 9, 3]$*:*

Element	größer als spätere	Anzahl
5	2, 1, 3	3
2	1	1
8	1, 3	2
1	—	0
9	3	1
		7

Sieben Inversionen, sieben Verschiebungen. **Der Satz stimmt exakt.** ✓

Blasensortieren.

Durchl.	Feld danach	Vergl.	Tausch.
1	2, 5, 1, 8, 3, 9	5	4
2	2, 1, 5, 3, 8, 9	5	2
3	1, 2, 3, 5, 8, 9	5	2
4	1, 2, 3, 5, 8, 9	5	0
		20	**8**

Der Vergleich.

Verfahren	Vergleiche	Bewegungen	stabil
Auswahl	15	5 Tausche	nein
Einfügen	12	7 Verschieb.	ja
Blase	20	8 Tausche	ja

Schon bei sechs Elementen zeigt sich das Bild, das in der Theorie steht: **Einfügesortieren gewinnt bei den Vergleichen, Auswahlsortieren bei den Bewegungen, Blasensortieren bei nichts.**

(b) Die mittlere Zahl der Inversionen.
Schritt 1 – Der Ansatz.
Sei π eine zufällige Permutation von $1, \dots, n$, alle $n!$ gleich wahrscheinlich. Für jedes Paar $i < j$ sei

$$X_{ij} := \begin{cases} 1 & \text{falls } \pi(i) > \pi(j) \text{ (Inversion)}, \\ 0 & \text{sonst.} \end{cases}$$

Schritt 2 – Die Wahrscheinlichkeit eines einzelnen Paares.

Für ein festes Paar sind die beiden Anordnungen „$\pi(i) < \pi(j)$“ und „$\pi(i) > \pi(j)$“ aus Symmetriegründen gleich wahrscheinlich.

$$\Pr[X_{ij} = 1] = \frac{1}{2} \Longrightarrow \mathbb{E}[X_{ij}] = \frac{1}{2}.$$

Schritt 3 – Linearität des Erwartungswerts.
Die Gesamtzahl ist $I = \sum_{i<j} X_{ij}$, und es gibt $\binom{n}{2}$ Paare:

$$\mathbb{E}[I] = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}.$$

$$\boxed{\mathbb{E}[I] = \frac{n(n-1)}{4} \approx \frac{n^2}{4}}$$

Der entscheidende Punkt: *Die X_{ij} sind* ***nicht unabhängig*** – aus $\pi(1) > \pi(2)$ und $\pi(2) > \pi(3)$ folgt $\pi(1) > \pi(3)$. **Für die Linearität des Erwartungswerts spielt das keine Rolle** (Mathematik-Band 4, Kapitel 7).

Schritt 4 – Die Folgerung.

$$T_{\text{Einfügen}}(n) = \Theta(n + \mathbb{E}[I]) = \Theta(n^2).$$

Und die Konstante ist $\frac{1}{4}$, nicht $\frac{1}{2}$ – **Einfügesortieren macht im Mittel nur halb so viele Vergleiche wie im schlechtesten Fall.**

Schritt 5 – Eine Nebenbemerkung.

Die Zahl der Inversionen ist zugleich ein Maß für die „Unordnung“ einer Folge und wird in der Statistik als ***Kendalls*** *τ benutzt* – **ein Rangkorrelationskoeffizient.**

Man kann sie in $\mathcal{O}(n \log n)$ zählen, indem man Mergesort laufen lässt und beim Verschmelzen mitzählt (Kapitel 4, Aufgabe).

(c) Die Schranke für benachbarte Vertauschungen.

Satz 3.3. *Jedes Sortierverfahren, das ausschließlich* ***benachbarte*** *Elemente vertauscht, braucht im schlechtesten Fall $\Omega(n^2)$ Vertauschungen.*

Schritt 1 – Der Schlüssel: Eine Vertauschung ändert wenig.

Vertauscht man $a[j]$ und $a[j+1]$, so ändert sich für ***alle anderen*** *Paare nichts an ihrer relativen Ordnung.* **Nur das eine Paar $(j, j+1)$ ändert seinen Zustand.**

$\Longrightarrow$ Jede Vertauschung ändert die Inversionszahl um genau ± 1.

Schritt 2 – Die schlechteste Eingabe.
Die umgekehrt sortierte Folge $[n, n-1, \ldots, 1]$ *hat* ***jedes*** *Paar invertiert:*

$$I = \binom{n}{2} = \frac{n(n-1)}{2}.$$

Schritt 3 – Der Schluss.
Die sortierte Folge hat $I = 0$. *Da jede erlaubte Operation* I *um höchstens 1 verringert, braucht man mindestens*

$$\frac{n(n-1)}{2} = \Omega(n^2)$$

Vertauschungen.

> ***Damit sind Blasensortieren, Einfügesortieren und alle verwandten Verfahren grundsätzlich erledigt*** – nicht weil sie schlecht programmiert wären, sondern weil ihre **Bewegungsart** zu schwach ist.

Schritt 4 – Was daraus für die schnellen Verfahren folgt.

> *Wer* $\mathcal{O}(n \log n)$ *erreichen will,* ***muss*** *Elemente über weite Strecken bewegen.*
> **Und genau das tun alle schnellen Verfahren:**

Shellsort	vertauscht über die Schrittweite h hinweg
Quicksort	bewegt Elemente über den ganzen Bereich
Heapsort	tauscht zwischen Wurzel und Blatt
Mergesort	schreibt in ein zweites Feld um

Das ist kein Zufall, sondern eine Notwendigkeit – **und der Satz sagt genau, warum.**

Eine hübsche Ausnahme, die die Regel bestätigt. *Beim* ***Odd-Even-Transposition-Sort*** *vertauscht man ausschließlich Nachbarn – aber* ***alle gleichzeitig.*** **Auf** n **Prozessoren sortiert das in** $\mathcal{O}(n)$ ***Runden.***

> *Die Gesamtzahl der Vertauschungen bleibt* $\Omega(n^2)$ *– der Satz gilt weiter.* **Aber sie verteilen sich auf** n **Prozessoren, und die *Zeit* sinkt.** *Solche* ***Sortiernetze*** *sind die Grundlage des Sortierens in Hardware und auf Grafikprozessoren.*

Knuth[17] behandelt sämtliche elementaren Verfahren mit vollständiger Analyse; Sedgewick und Wayne[24] zeigen sie mit ausgezeichneten Bildern; Ottmann und Widmayer[22] sind die gründlichste deutschsprachige Darstellung.

4 Die schnellen Sortierverfahren

Der Sprung von n^2 auf $n \log n$

Alle drei schnellen Verfahren beruhen auf demselben Gedanken: ***Teile und herrsche.*** **Man zerlegt das Problem in kleinere gleichartige, löst diese und setzt die Lösungen zusammen.**

Verfahren	Aufwand steckt im	Kennzeichen
Mergesort	Zusammensetzen	Teilen ist trivial
Quicksort	Teilen	Zusammensetzen ist trivial
Heapsort	Struktur aufbauen	keine Rekursion

Die ersten beiden sind zueinander spiegelbildlich – **ein Muster, das in der Algorithmik häufig auftritt.**

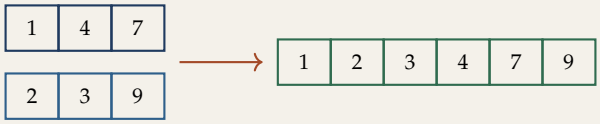
4.1 Mergesort

Das älteste Verfahren – von Neumann 1945

```
sortiere(a, links, rechts):
  if links >= rechts then return;
  m := (links + rechts) div 2;
  sortiere(a, links, m);
  sortiere(a, m+1, rechts);
  verschmelze(a, links, m, rechts)
```

Das Verschmelzen – der eigentliche Kern.

Zwei sortierte Folgen werden zu einer verschmolzen, indem man an beiden Anfängen steht und jeweils das Kleinere übernimmt.



Jeder Vergleich befördert genau ein Element ins Ergebnis. **Also: $\mathcal{O}(n)$ für das Verschmelzen von insgesamt n Elementen. ✓**

Die Rekursionsgleichung (Kapitel 1):

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \Longrightarrow T(n) = \Theta(n \log n).$$

Eigenschaft	Mergesort	Bemerkung
schlechtester Fall	$\Theta(n \log n)$	garantiert
mittlerer Fall	$\Theta(n \log n)$	
bester Fall	$\Theta(n \log n)$	$\Theta(n)$ bei Vorprüfung
stabil	**ja**	bei $\leq$ im Vergleich
in situ	**nein**	$\Theta(n)$ Zusatzspeicher

Die Stabilität hängt an einem einzigen Zeichen

Beim Verschmelzen muss man bei Gleichstand das Element aus der *linken* Folge nehmen.

`if links[i] <= rechts[j]`	stabil
`if links[i] < rechts[j]`	instabil

Denn die linke Folge enthält die ursprünglich früheren Elemente. **Ein einziges Zeichen entscheidet über eine Eigenschaft, die im Einsatz erheblich sein kann. ✓**

Warum Mergesort trotz des Zusatzspeichers überlebt hat.

Garantie	Kein schlechter Fall – wichtig, wo Zeitschranken gelten
Stabilität	Für mehrstufiges Sortieren unentbehrlich
Externes Sortieren	Braucht nur sequentiellen Zugriff – perfekt für Bänder und Platten
Parallelisierbar	Die Teilprobleme sind unabhängig
Listen	Auf verketteten Listen sogar *in situ*

Der dritte Punkt war jahrzehntelang der entscheidende: ***Wer eine Datei sortieren muss, die nicht in den Hauptspeicher passt****, zerlegt sie in Blöcke, sortiert diese einzeln und verschmilzt sie in mehreren Läufen.* **Das ist bis heute das Verfahren jeder Datenbank für große Sortierungen.**

4.2 Quicksort

Das Verfahren, das aus einem Übersetzungsprojekt entstand

```
sortiere(a, links, rechts):
  if links >= rechts then return;
  p := zerlege(a, links, rechts);
  sortiere(a, links, p-1);
  sortiere(a, p+1, rechts)
```

Das Zerlegen – hier steckt die ganze Arbeit.

Man wählt ein ***Pivotelement*** *und ordnet das Feld so um, dass links davon alle kleineren und rechts alle größeren stehen.* **Danach steht das Pivot an seiner endgültigen Stelle** – *und muss nie wieder angefasst werden.*



endgültige Position

Die Zerlegung nach Lomuto (einfach zu verstehen):

```
zerlege(a, l, r):
  p := a[r];      letztes Element als Pivot
  i := l-1;
  for j := l to r-1 do
    if a[j] < p then { i := i+1;
vertausche a[i], a[j] }
  vertausche a[i+1], a[r];
  return i+1
```

Die Invariante:

$$a[l \ldots i] < p, \qquad a[i{+}1 \ldots j{-}1] \geq p, \qquad a[j \ldots r{-}1] \text{ noch ungeprüft}$$

Die Analyse.

Fall	Aufwand	wann
bester	$\Theta(n \log n)$	Pivot trifft die Mitte
mittlerer	$\Theta(n \log n)$	zufällige Eingabe
schlechtester	$\Theta(n^2)$	**Pivot immer extrem**

Im schlechtesten Fall gilt

$$T(n) = T(n-1) + \Theta(n) \Longrightarrow T(n) = \Theta(n^2).$$

Und der schlechteste Fall tritt genau dann ein, wenn das Feld bereits sortiert ist – wenn man das letzte oder erste Element als Pivot nimmt. **Das ist bitter: Der häufigste Praxisfall ist der schlimmste.**

Warum es trotzdem „quick“ heißt

Satz 4.1. *Bei zufällig gewähltem Pivot beträgt die erwartete Zahl der Vergleiche*

$$\mathbb{E}[C_n] = 2(n+1)H_n - 4n \approx 1{,}386\, n \log_2 n,$$

wobei $H_n = \sum_{k=1}^{n} \frac{1}{k}$ die harmonische Zahl ist.

Der Beweis – ein Musterstück der Wahrscheinlichkeitsrechnung.

Seien $z_1 < z_2 < \cdots < z_n$ die sortierten Werte. Für $i < j$ sei

$$X_{ij} := \begin{cases} 1 & \text{falls } z_i \text{ und } z_j \text{ jemals verglichen werden,} \\ 0 & \text{sonst.} \end{cases}$$

Schlüsselbeobachtung.

> *Zwei Elemente werden **genau dann** verglichen, wenn das **erste** Pivot aus dem Bereich $\{z_i, \ldots, z_j\}$ eines von beiden ist.*

Denn: Ist das erste Pivot aus diesem Bereich ein z_k mit $i < k < j$, so werden z_i und z_j auf verschiedene Seiten getrennt und nie mehr verglichen. **Ist es z_i oder z_j selbst, so wird es mit allen anderen des Bereichs verglichen – also auch mit dem Partner.** ✓
Der Bereich hat $j - i + 1$ Elemente, jedes ist gleich wahrscheinlich das erste Pivot:

$$\Pr[X_{ij} = 1] = \frac{2}{j - i + 1}.$$

Linearität des Erwartungswerts:

$$\mathbb{E}[C_n] = \sum_{i<j} \frac{2}{j - i + 1} = \sum_{i=1}^{n-1} \sum_{d=2}^{n-i+1} \frac{2}{d} < 2n \sum_{d=2}^{n} \frac{1}{d} = 2n(H_n - 1).$$

Mit $H_n \approx \ln n + 0{,}5772$ (Mathematik-Band 3):

$$\mathbb{E}[C_n] \approx 2n \ln n = 2n \cdot \frac{\log_2 n}{\log_2 \mathrm{e}} = 1{,}386\, n \log_2 n. \qquad \checkmark$$

Verfahren	**Vergleiche**	**Faktor zum Optimum**
Untere Schranke	$n \log_2 n$	1,00
Mergesort	$\approx n \log_2 n$	$\approx$ 1,0
Quicksort (mittel)	$1{,}386\, n \log_2 n$	1,39
Heapsort	$\approx 2\, n \log_2 n$	$\approx$ 2,0

> ***Quicksort macht mehr Vergleiche als Mergesort und ist trotzdem in der Praxis schneller.*** Der Grund liegt nicht in der Zählung, sondern im Speicherverhalten: **Quicksort arbeitet in situ und durchläuft das Feld sequentiell** – *ideal für den Cache.*

Die Konstanten sind hier wichtiger als die Größenordnung – *einer der wenigen Fälle, wo die $\mathcal{O}$-Notation die Praxis nicht entscheidet.*

Wie Quicksort entstand

Tony Hoare war 1959 ein 25-jähriger britischer Austauschstudent in Moskau. *Er arbeitete an einem Projekt zur maschinellen Übersetzung vom Russischen ins Englische.*

> *Die Aufgabe: Ein russischer Satz sollte Wort für Wort in einem Wörterbuch nachgeschlagen werden. Das Wörterbuch lag auf Magnetband und war sortiert.* **Also musste man die Wörter des Satzes ebenfalls sortieren, um das Band nur einmal durchlaufen zu müssen.**

Hoare kannte Einfügesortieren und merkte, dass es zu langsam war. *Ihm fiel auf, dass man das Problem halbieren kann:*

> *„Nimm ein Wort. Bringe alle Wörter, die im Alphabet davor kommen, nach links, alle anderen nach rechts. Dann hat man zwei kleinere Probleme derselben Art."*

Das Problem war die Programmierung. *Hoare hatte damals nur Assembler zur Verfügung, und der Rekursionsmechanismus – ein Stapel für die Rücksprungadressen – war ihm nicht geläufig.* **Er konnte das Verfahren beschreiben, aber nicht schreiben.**

Erst als er 1960 bei Elliott Brothers in England mit ***ALGOL 60*** *zu tun bekam – der ersten weit verbreiteten Sprache mit Rekursion – ließ es sich in wenigen Zeilen ausdrücken.*

> *Hoare veröffentlichte es 1961 als Algorithmus 64 in den* Communications of the ACM *– auf einer halben Seite – und 1962 mit vollständiger Analyse im* Computer Journal.

Er erhielt 1980 den Turing-Award, *unter anderem dafür.* **In seiner Preisrede sprach er allerdings über etwas anderes:** *über die Null-Referenz, die er 1965 in ALGOL W eingeführt hatte und die er später seinen „Milliarden-Dollar-Fehler" nannte.*

„Ich konnte der Versuchung nicht widerstehen, eine Null-Referenz einzubauen, einfach weil es so leicht zu implementieren war."

Wie man den schlechtesten Fall vermeidet

Zufälliges Pivot	Der schlechteste Fall wird unwahrscheinlich statt unmöglich
Median von drei	Pivot = Median aus erstem, mittlerem, letztem Element
Median von neun	Bei großen Feldern; „Ninther“
Introsort	Zählt die Rekursionstiefe; ab $2 \log_2 n$ Umschalten auf Heapsort

Introsort *(*DAVID MUSSER*, 1997)* ***ist die praktisch überzeugendste Lösung:*** Quicksort für die Geschwindigkeit, Heapsort als Notbremse für die Garantie, Einfügesortieren für kleine Teilfelder.

Es ist der Standard in der C++-Bibliothek (`std::sort`) und in Go.

Der Angriff auf Quicksort

Ein *deterministisches* Pivot ist eine Sicherheitslücke.

> *Wer die Pivotregel kennt, kann eine Eingabe konstruieren, die den quadratischen Fall erzwingt.* **DOUGLAS MCILROY zeigte 1999, wie man das für *jede* deterministische Regel tut** *– mit einem „gegnerischen“ Vergleicher, der seine Antworten erst während des Laufs festlegt.*

In einem Webdienst, der Benutzereingaben sortiert, ist das ein Angriff: *Statt Millisekunden braucht die Anfrage Minuten, und wenige solcher Anfragen legen den Dienst lahm* (**algorithmische Komplexitätsangriffe**).

Dieselbe Angriffsform gibt es beim Hashing (Kapitel 9) **– und dort hat sie 2011 zu einer koordinierten Sicherheitswarnung für praktisch alle Webframeworks geführt.**

Die Gegenmaßnahme ist in beiden Fällen dieselbe: *echter Zufall, den der Angreifer nicht kennt.*

4.3 Heapsort und Timsort

Heapsort – die Garantie ohne Zusatzspeicher

J. W. J. WILLIAMS *veröffentlichte 1964 eine Datenstruktur namens* ***Heap*** *und ein darauf beruhendes Sortierverfahren.* **ROBERT FLOYD verbesserte es im selben Jahr.**

Der Ablauf in zwei Phasen:

Aufbau	Das Feld wird in einen Max-Heap verwandelt: $\Theta(n)$
Abbau	n-mal: Wurzel (das Maximum) mit dem letzten Element tauschen, Heap verkleinern, absinken lassen: $\Theta(n \log n)$

Kapitel 8 entwickelt die Struktur vollständig. **Hier nur die Bilanz:**

	Merge	**Quick**	**Heap**
schlechtester Fall	$n \log n$	n^2	$n \log n$
in situ	nein	ja	ja
stabil	ja	nein	nein
Cacheverhalten	gut	sehr gut	schlecht
Praxis	Standard	schnellster	Notbremse

Heapsort ist theoretisch das schönste und praktisch das langsamste der drei. Der Grund steht in der vierten Zeile: Es springt beim Absinken zwischen weit entfernten Feldstellen hin und her – **fast jeder Zugriff ein Cachefehler.**

Seine Rolle ist die des Sicherheitsnetzes: garantiert $\mathcal{O}(n \log n)$, garantiert $\mathcal{O}(1)$ Speicher, keine schlechten Eingaben.

Timsort – als jemand die Daten anschaute

TIM PETERS *schrieb 2002 für Python ein neues Sortierverfahren. Sein Ausgangspunkt war keine Theorie, sondern eine Beobachtung:*

Reale Daten sind fast nie zufällig. **Sie enthalten längere Abschnitte, die bereits sortiert oder umgekehrt sortiert sind.**

Das Verfahren.

1. Das Feld wird in bereits monotone Abschnitte (**runs**) zerlegt; absteigende werden umgedreht.
2. Zu kurze Abschnitte werden mit Einfügesortieren auf eine Mindestlänge (32–64) gebracht.
3. Die Abschnitte werden nach einer Regel verschmolzen, die die Stapelgrößen ausgewogen hält.
4. Beim Verschmelzen wird **galoppiert**: Findet ein Abschnitt mehrfach hintereinander den Zuschlag, wird binär gesucht, wie weit er am Stück übernommen werden kann.

Eingabe	Timsort	Bemerkung
bereits sortiert	$\Theta(n)$	ein einziger Abschnitt
umgekehrt sortiert	$\Theta(n)$	umdrehen genügt
k Abschnitte	$\Theta(n \log k)$	**das ist der Gewinn**
zufällig	$\Theta(n \log n)$	wie Mergesort

Die dritte Zeile ist die entscheidende. Eine Datei, an die man einen sortierten Block angehängt hat, hat $k = 2$ – **und Timsort sortiert sie in linearer Zeit.** *Mergesort und Quicksort merken davon nichts.*

Timsort ist heute der Standard in Python, Java (für Objekte), Android, Swift und Rust.

Ein Fehler, den erst ein Beweisassistent fand

2015 untersuchte eine Gruppe an der TU Eindhoven um Stijn de Gouw den Timsort-Code der Java-Bibliothek mit einem formalen Verifikationswerkzeug.

> *Sie wollten die Korrektheit* ***beweisen*** *– und scheiterten. Der Beweis ging nicht durch.* **Beim Nachsehen fand sich der Grund: Die Regel für die Stapelgrößen war nicht stark genug.**

Der Stapel der noch zu verschmelzenden Abschnitte konnte in seltenen Fällen tiefer werden als vorgesehen – **und dann lief das vorab dimensionierte Feld über.**

Die Gruppe konstruierte eine Eingabe, die den Absturz auslöste. **Sie brauchte 67 108 864 Elemente** *– was erklärt, warum der Fehler dreizehn Jahre unentdeckt blieb.*

> *Der Fehler steckte in Python seit 2002, in Java seit 2009 und in Android auf hunderten Millionen Geräten.* **Er wurde nicht durch Testen gefunden, nicht durch Codelesen und nicht durch Abstürze in der Praxis** *– sondern durch den gescheiterten Versuch, einen Beweis zu führen.*

Das ist das stärkste Argument für formale Verifikation, das ich kenne – *und zugleich eine Mahnung: Der Code war von einem sehr guten Programmierer geschrieben, jahrelang benutzt und millionenfach ausgeführt.*

Aufgabe 4.1.

(a) Führen Sie Mergesort und Quicksort an derselben Folge vollständig vor.
(b) Zählen Sie Inversionen in $\mathcal{O}(n \log n)$ durch Abwandlung von Mergesort.
(c) Sortieren Sie in linearer Zeit – und erklären Sie, warum das der unteren Schranke nicht widerspricht.

Lösung Schritt für Schritt.

(a) Zwei Verfahren, eine Folge.
Wir sortieren $[\,38,\ 27,\ 43,\ 3,\ 9,\ 82,\ 10\,]$.
Mergesort – der Abstieg.

Ebene 0	$[38, 27, 43, 3, 9, 82, 10]$
Ebene 1	$[38, 27, 43]$ $[3, 9, 82, 10]$
Ebene 2	$[38]$ $[27, 43]$ $[3, 9]$ $[82, 10]$
Ebene 3	$[27]$ $[43]$ $[3]$ $[9]$ $[82]$ $[10]$

Der Aufstieg – Verschmelzen.

Verschmelzen		**Ergebnis**	**Vergleiche**
$[27]$ & $[43]$		$[27, 43]$	1
$[38]$ & $[27, 43]$		$[27, 38, 43]$	2
$[3]$ & $[9]$		$[3, 9]$	1
$[82]$ & $[10]$		$[10, 82]$	1
$[3, 9]$ & $[10, 82]$		$[3, 9, 10, 82]$	2
$[27, 38, 43]$ $[3, 9, 10, 82]$	&	$[3, 9, 10, 27, 38, 43, 82]$	6
			13

Zum Vergleich: $n \log_2 n = 7 \cdot 2{,}807 = 19{,}7$. **Die Schranke wird eingehalten.** ✓

Quicksort – mit dem letzten Element als Pivot.

Ruf	**Bereich**	**Pivot**	**danach**
1	$[38, 27, 43, 3, 9, 82, 10]$	10	$[3, 9, \mathbf{10}, 43, 38, 82, 27]$
2	$[3, 9]$	9	$[3, \mathbf{9}]$
3	$[3]$	—	fertig
4	$[43, 38, 82, 27]$	27	$[\mathbf{27}, 38, 82, 43]$
5	$[38, 82, 43]$	43	$[38, \mathbf{43}, 82]$
6	$[38]$, $[82]$	—	fertig

Ergebnis: $[3, 9, 10, 27, 38, 43, 82]$. ✓

Rekursionstiefe: 4. **Bei ausgewogener Teilung wären es** $\log_2 7 = 2{,}8$**, im schlechtesten Fall 7.**

(b) Inversionen zählen.

Die naive Zählung prüft alle Paare: $\Theta(n^2)$. **Mergesort kann es in** $\Theta(n \log n)$ *– fast ohne Zusatzaufwand.*

Schritt 1 – Die Zerlegung der Inversionen.
Teilt man die Folge in linke und rechte Hälfte, so zerfallen die Inversionen in drei Gruppen:

beide links	rekursiv zählen
beide rechts	rekursiv zählen
gemischt	**beim Verschmelzen zählen**

Schritt 2 – Die Zählung beim Verschmelzen.

*Beide Hälften sind bereits sortiert. Nimmt man beim Verschmelzen ein Element aus der **rechten** Hälfte, so ist es kleiner als **alle noch verbliebenen** Elemente der linken.*

$\Longrightarrow$ Man addiert die Zahl der Restelemente der linken Hälfte.

Schritt 3 – Ein Durchlauf.
Verschmelzen von $[3,\ 27,\ 38]$ *und* $[9,\ 10,\ 43]$*:*

Nimm	**aus**	**Rest links**	**Inversionen**
3	links	—	0
9	**rechts**	$\{27, 38\}$	$+2$
10	**rechts**	$\{27, 38\}$	$+2$
27	links	—	0
38	links	—	0
43	rechts	$\{\}$	$+0$
			4

Probe: Die gemischten Inversionen sind $(27, 9), (27, 10), (38, 9), (38, 10)$ – **genau vier.** ✓
Schritt 4 – Aufwand.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n). \qquad \checkmark$$

Das ist ein Muster, das sich lohnt zu merken: *Ein Sortierverfahren lässt sich oft so erweitern, dass es nebenbei eine Zusatzgröße berechnet* – **ohne dass die Größenordnung steigt.**

Anwendungen: Rangkorrelation in der Statistik, Ähnlichkeit von Genomsequenzen, Bewertung von Suchmaschinenrankings.

(c) Sortieren in linearer Zeit.
Verfahren 1 – Zählsortieren.
Voraussetzung: Die Schlüssel sind ganze Zahlen aus $\{0, \ldots, k\}$.

```
for i := 0 to k do z[i] := 0;        Zähler leeren
for j := 1 to n do z[a[j]] := z[a[j]] + 1;     zählen
for i := 1 to k do z[i] := z[i] + z[i-1];     aufsummieren
for j := n downto 1 do {
  b[z[a[j]]] := a[j];  z[a[j]] := z[a[j]] - 1 }
```

Beispiel $a = [2,5,3,0,2,3,0,3]$, $k = 5$:

Wert	0	1	2	3	4	5
Häufigkeit	2	0	2	3	0	1
kumuliert	2	2	4	7	7	8

Die kumulierte Zeile sagt: Alle Nullen enden bei Position 2, alle Zweien bei 4, alle Dreien bei 7. **Damit kann man jedes Element direkt an seinen Platz schreiben.** ✓

Ergebnis: $[0,0,2,2,3,3,3,5]$. ✓

$$\boxed{T(n) = \Theta(n+k)}$$

Die ***rückwärts*** *laufende letzte Schleife ist kein Zufall: Sie sorgt für* ***Stabilität*** *–* **gleiche Werte behalten ihre Reihenfolge.** *Und Stabilität ist die Voraussetzung für das nächste Verfahren.*

Verfahren 2 – Radixsortieren.
Man sortiert nach der letzten Ziffer, dann nach der vorletzten, und so fort – jeweils mit einem ***stabilen*** *Verfahren.*

Anfang	nach Einer	nach Zehner	nach Hunderter
329	72<u>0</u>	7<u>2</u>0	<u>3</u>29
457	35<u>5</u>	3<u>2</u>9	<u>3</u>55
657	43<u>6</u>	4<u>3</u>6	<u>4</u>36
839	45<u>7</u>	8<u>3</u>9	<u>4</u>57
436	65<u>7</u>	3<u>5</u>5	<u>6</u>57
720	32<u>9</u>	4<u>5</u>7	<u>7</u>20
355	83<u>9</u>	6<u>5</u>7	<u>8</u>39

$$\boxed{T(n) = \Theta(d \cdot (n + b))}$$

bei d Stellen zur Basis b.
Warum die Stabilität wesentlich ist.

Nach dem Sortieren der Zehnerstelle sind alle Zahlen mit gleicher Zehnerziffer noch nach der Einerziffer geordnet – **aber nur, wenn das Verfahren stabil war.** *Sonst geht die Vorarbeit verloren.* ✓

Warum das der unteren Schranke nicht widerspricht.

Die Schranke sagt	Jedes Verfahren, das nur **vergleicht**, braucht $\Omega(n \log n)$
Diese Verfahren	**vergleichen nicht** – sie schauen sich die Werte an

Ein Vergleichsverfahren erfährt pro Vergleich ***ein Bit*** (Band 0, Kapitel 1), *und es muss unter* $n!$ *Anordnungen unterscheiden, also* $\log_2(n!)$ *Bit sammeln.* **Zählsortieren erfährt beim Anschauen eines Werts gleich** $\log_2 k$ **Bit auf einmal.**

Der Haken sitzt in k und d

$\Theta(n+k)$ **ist nur dann linear, wenn** $k = \mathcal{O}(n)$**.**
Beim Sortieren von 1000 32-Bit-Zahlen wäre $k = 4{,}3 \cdot 10^9$ **– das Zählfeld allein braucht 17 Gigabyte.**

Radixsortieren umgeht das, indem es die Zahl in Stellen zerlegt. **Bei 32-Bit-Zahlen und Basis** 2^8 **sind es** $d = 4$ **Durchläufe mit** $k = 256$ *– also* $\Theta(4(n+256))$.

> ***Und dann steckt*** log ***doch wieder drin:*** Um n verschiedene Werte darstellen zu können, braucht man $d = \Omega(\log_b n)$ Stellen. **Damit ist** $\Theta(d \cdot n) = \Theta(n \log n)$ *– die Schranke holt einen ein.*

In der Praxis gewinnt Radixsortieren trotzdem oft, *weil die Schlüssellänge fest ist (32 oder 64 Bit) und nicht mit n wächst* **– und weil es sehr cachefreundlich arbeitet.** *Bei Milliarden von Ganzzahlen ist es das schnellste bekannte Verfahren.*

Cormen u. a.[6] beweisen sämtliche Schranken vollständig; Sedgewick und Wayne[24] vergleichen die Verfahren experimentell; Kleinberg und Tardos[15] entwickeln Teile-und-herrsche als Entwurfsmuster; Hoares Originalarbeit[14] ist zwei Seiten lang.

5 Suchen und Auswählen

Die drei Grundfragen

Frage	unsortiert	sortiert
Kommt x vor?	$\Theta(n)$	$\Theta(\log n)$
Was ist das k-kleinste?	$\Theta(n)$	$\Theta(1)$
Wie viele sind $< x$?	$\Theta(n)$	$\Theta(\log n)$

Die rechte Spalte setzt voraus, dass jemand vorher sortiert hat – und das kostet $\Theta(n \log n)$. **Es lohnt sich also nur, wenn oft gesucht wird.**

Die Faustregel:

$$n \log n + q \cdot \log n \quad \text{gegen} \quad q \cdot n$$

bei q Suchanfragen. **Sortieren lohnt ab etwa** $q > \log n$ *– also fast immer.*

5.1 Binäre Suche und ihre Verwandten

Die Randfälle – wo alle Fehler sitzen

Die binäre Suche wurde 1946 beschrieben. ***Der erste korrekte Code für den Fall doppelter Schlüssel erschien 1962*** – sechzehn Jahre später. **KNUTH bemerkt dazu, dass zwar die Idee einfach ist, die Einzelheiten aber „überraschend heikel".**

Eine Studie von 1986 unter Berufsprogrammierern ergab, dass nur etwa zehn Prozent eine fehlerfreie binäre Suche schrieben – *obwohl alle das Verfahren kannten.* **Die typischen Fehler:**

Überlauf	`(lo+hi) div 2` statt `lo + (hi-lo) div 2` (Band 0, Kap. 2)
Endlosschleife	`hi := m` statt `hi := m-1`
Grenzen	< statt <= in der Schleifenbedingung
Duplikate	Welches der gleichen Elemente wird gefunden?

Die saubere Fassung für *Duplikate*: die untere Schranke.

```
untere_schranke(a, x):      erste Position mit a[i] >= x
  lo := 1;  hi := n+1;
  while lo < hi do {
    m := lo + (hi-lo) div 2;
    if a[m] < x then lo := m+1 else hi := m }
  return lo
```

Die Invariante:

$$\boxed{a[i] < x \text{ für } i < lo, \qquad a[i] \geq x \text{ für } i \geq hi}$$

Anfangs sind beide Bereiche leer. ✓ Der Schritt verkleinert das Intervall echt (weil $m < hi$ stets gilt). ✓ Am Ende ist $lo = hi$, und lo ist die gesuchte Grenze. ✓

> ***Diese Fassung hat keine Sonderfälle.*** Sie liefert die erste Position mit $a[i] \geq x$; ist x nicht vorhanden, liefert sie die Einfügestelle. **Mit einer Zeile mehr erhält man auch die obere Schranke und damit die Zahl der Vorkommen.**

Das ist genau, was `bisect_left` in Python, `lower_bound` in C++ und `sort.SearchInts` in Go leisten.

Interpolationssuche – Suchen wie im Telefonbuch

> *Wer im Telefonbuch „Zimmermann“ sucht, schlägt nicht in der Mitte auf, sondern weit hinten.* **Die Interpolationssuche tut dasselbe: Sie schätzt die Position aus dem Wert.**

$$m = lo + \left\lfloor \frac{(x - a[lo]) \cdot (hi - lo)}{a[hi] - a[lo]} \right\rfloor .$$

Verteilung	Aufwand	Beispiel
gleichmäßig	$\mathcal{O}(\log \log n)$	Messwerte, Zufallszahlen
beliebig	$\mathcal{O}(n)$	exponentiell wachsend

$\mathcal{O}(\log \log n)$ *ist bemerkenswert: Bei einer Milliarde Elementen sind das* ***fünf*** *Vergleiche statt dreißig.*

Der Preis ist die Empfindlichkeit. *Bei den Werten* $1, 2, 4, 8, \ldots, 2^n$ *schätzt die Interpolation immer viel zu weit links und tastet sich einzeln vor* – **lineare Zeit.**

In der Praxis benutzt man sie deshalb selten pur, sondern als ***Interpolation-Sequential-Search****: interpolieren, dann linear in der Nachbarschaft suchen* – **oder man schaltet nach ein paar erfolglosen Schritten auf binäre Suche um.**

5.2 Das Auswahlproblem

Das k-kleinste Element finden

Gesucht ist das k-kleinste Element einer unsortierten Folge – **für** $k = 1$ **das Minimum, für** $k = \lceil n/2 \rceil$ **der Median.**

Verfahren	Aufwand	Bemerkung
Sortieren, dann ablesen	$\mathcal{O}(n \log n)$	Verschwendung
k-mal Minimum suchen	$\mathcal{O}(kn)$	gut für kleines k
Heap der Größe k	$\mathcal{O}(n \log k)$	**gut für Bestenlisten**
Quickselect	$\mathcal{O}(n)$ *erwartet*	Hoare 1961
Median der Mediane	$\mathcal{O}(n)$ *garantiert*	Blum u. a. 1973

Quickselect – Quicksort, das nur eine Seite verfolgt.

```
select(a, l, r, k):
  p := zerlege(a, l, r);
  if p = k then return a[p]
  else if k < p then return select(a,
l, p-1, k)
  else return select(a, p+1, r, k)
```

> ***Der einzige Unterschied zu Quicksort:*** *Es wird nur* ***ein*** *rekursiver Aufruf gemacht statt zweier.* **Und das ändert die Größenordnung.**

Die Analyse bei zufälligem Pivot.
Im Mittel halbiert sich der Bereich:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(n).$$

Master-Theorem mit $a = 1$, $b = 2$, $\kappa = 0$, $f(n) = n = \Omega(n^{0+\varepsilon})$ – ***Fall 3****:*

$$\boxed{T(n) = \Theta(n)}$$

> *Anschaulich: Die Kosten bilden eine geometrische Reihe*
>
> $$n + \frac{n}{2} + \frac{n}{4} + \cdots = 2n.$$
>
> **Die erste Zerlegung kostet fast alles; alles Weitere zusammen noch einmal so viel.**

Die genaue Analyse liefert $\mathbb{E}[C_n] \approx 3{,}39\,n$ *für den Median.* **Im schlechtesten Fall bleibt es bei** $\Theta(n^2)$ **–** *wie bei Quicksort.*

Median der Mediane – die Garantie

> *1973 zeigten* Blum, Floyd, Pratt, Rivest *und* Tarjan, *dass es auch* ***garantiert*** *linear geht*[5] **– ein Ergebnis, das viele für unmöglich gehalten hatten.**

Die Autorenliste ist bemerkenswert: Vier der fünf erhielten später den Turing-Award.

Die Idee: ein Pivot, das garantiert nicht extrem ist.

1. Teile die n Elemente in $\lceil n/5 \rceil$ Fünfergruppen.
2. Bestimme den Median jeder Gruppe (durch Sortieren – 5 Elemente, konstanter Aufwand).
3. Bestimme **rekursiv** den Median dieser $\lceil n/5 \rceil$ Medianen.
4. Benutze ihn als Pivot und fahre wie Quickselect fort.

Warum das Pivot gut ist.



Sei p der Median der Mediane. Von den $\lceil n/5 \rceil$ Gruppenmedianen sind mindestens die Hälfte $\leq p$. In jeder dieser Gruppen sind mindestens 3 der 5 Elemente $\leq$ ihrem Median, also $\leq p$:

$$\text{mindestens } 3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10} \text{ Elemente sind } \leq p.$$

Ebenso sind mindestens $\frac{3n}{10}$ Elemente $\geq p$. **Also enthält jede Seite höchstens $\frac{7n}{10}$ Elemente.** ✓

Die Rekursionsgleichung.

$$T(n) \leq \underbrace{T\left(\frac{n}{5}\right)}_{\text{Median der Mediane}} + \underbrace{T\left(\frac{7n}{10}\right)}_{\text{Rekursion}} + \underbrace{\Theta(n)}_{\text{Gruppieren, Zerlegen}}.$$

Warum das linear ist.
Man setzt $T(n) \leq cn$ an und prüft:

$$T(n) \leq c\frac{n}{5} + c\frac{7n}{10} + an = cn\left(\frac{1}{5} + \frac{7}{10}\right) + an = \frac{9}{10}cn + an.$$

Das ist $\leq cn$, sobald $an \leq \frac{1}{10}cn$, also $c \geq 10a$. ✓

$$\boxed{T(n) = \mathcal{O}(n) \quad \text{garantiert}}$$

Warum ausgerechnet Fünfergruppen

Der Schlüssel ist, dass $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$ sein muss.

Gruppe	Rekursion 1	Rekursion 2	Summe
3	$\frac{1}{3}$	$\frac{2}{3}$	1 – scheitert
5	$\frac{1}{5}$	$\frac{7}{10}$	0,90 ✓
7	$\frac{1}{7}$	$\frac{5}{7}$	0,857 ✓

Bei Dreiergruppen ergibt die Summe genau 1 – und dann bricht der Beweis zusammen. Die Laufzeit wird tatsächlich $\Theta(n \log n)$. **Fünf ist die kleinste Gruppengröße, die funktioniert.**

Und trotzdem wird das Verfahren praktisch kaum eingesetzt. *Die versteckte Konstante ist groß – man rechnet mit etwa $24n$ Vergleichen gegenüber $3{,}4n$ bei Quickselect.*

Man benutzt es als ***Notbremse****, analog zu Introsort* (Kapitel 4): **Quickselect mit zufälligem Pivot, und bei zu vielen erfolglosen Runden Umschalten auf die Garantie.** *Das heißt* ***Introselect*** *und steckt in* `std::nth_element`*.*

Aufgabe 5.1.

(a) Finden Sie das Minimum *und* das Maximum mit möglichst wenigen Vergleichen – und beweisen Sie die Optimalität.

(b) Bestimmen Sie den Median zweier sortierter Felder in $\mathcal{O}(\log n)$.

(c) Finden Sie das k-größte Element eines Datenstroms, dessen Länge unbekannt ist.

Lösung Schritt für Schritt.

(a) Minimum und Maximum zugleich.
Schritt 1 – Der naive Weg.
Zwei getrennte Durchläufe: $2(n-1) = 2n-2$ Vergleiche.
Ein Durchlauf mit zwei Merkern: **ebenfalls** $2n-2$, denn jedes Element wird mit beiden verglichen.
Schritt 2 – Der bessere Weg: paarweise vergleichen.

Man nimmt die Elemente zu ***zweit****. Erst vergleicht man die beiden miteinander – dann kann nur der Kleinere neues Minimum und nur der Größere neues Maximum sein.*

```
for je zwei Elemente x, y do {
  if x > y then vertausche x, y;        1 Vergleich
  if x < min then min := x;     1 Vergleich
  if y > max then max := y }    1 Vergleich
```

Schritt 3 – Zählen.
Pro Paar: 3 Vergleiche für 2 Elemente. Bei geradem n nimmt man das erste Paar zur Initialisierung (1 Vergleich):

$$1 + 3 \cdot \frac{n-2}{2} = \frac{3n}{2} - 2.$$

n	**naiv**	**paarweise**	**Ersparnis**
100	198	148	25 %
10^6	$2 \cdot 10^6$	$1{,}5 \cdot 10^6$	25 %

Schritt 4 – Die Optimalität beweisen.

> *Wir zeigen:* ***Weniger als*** $\lceil 3n/2 \rceil - 2$ ***Vergleiche genügen nie.*** Das Argument ist ein **Gegnerargument** – man denkt sich einen Widersacher, der die Antworten so wählt, dass man möglichst viel arbeiten muss.

Man verfolge für jedes Element vier mögliche Zustände:

N	noch nie verglichen – kann noch Min *und* Max sein
K	hat schon verloren – kann nicht mehr Max sein
G	hat schon gewonnen – kann nicht mehr Min sein
B	hat gewonnen *und* verloren – kann weder noch

Am Anfang sind alle n Elemente im Zustand N; am Ende müssen $n-2$ im Zustand B sein (alle außer Min und Max).
Wie viel kann ein Vergleich leisten?

Vergleich	Fortschritt	Zustandswechsel
N gegen N	**2 Einheiten**	einer wird G, einer K
N gegen andere	1 Einheit	N wird G oder K
K gegen K	1 Einheit	einer wird B
G gegen G	1 Einheit	einer wird B
G gegen K	0 Einheiten	**nutzlos**

Gesamtbedarf: $2n-2$ *Einheiten (jedes der* $n-2$ *Elemente muss zweimal wechseln, Min und Max je einmal).*
Nur $\lfloor n/2 \rfloor$ *Vergleiche können 2 Einheiten liefern – danach ist kein N-Paar mehr übrig.*
Die restlichen liefern höchstens 1:

$$\text{Vergleiche} \;\geq\; \frac{n}{2} + \left(2n - 2 - 2\cdot\frac{n}{2}\right) = \frac{n}{2} + n - 2 = \frac{3n}{2} - 2. \qquad \checkmark$$

$$\boxed{\lceil 3n/2 \rceil - 2 \text{ ist optimal.}}$$

> ***Das Gegnerargument ist die wichtigste Technik für untere Schranken.*** Man beweist nicht über alle Algorithmen – man zeigt, dass ein Widersacher jeden Algorithmus zwingen kann, so lange zu arbeiten.

(b) Median zweier sortierter Felder.
Gegeben: zwei sortierte Felder A *und* B *der Längen* m *und* n*. Gesucht: der Median der Vereinigung.*
Schritt 1 – Was der naive Weg kostet.
Verschmelzen und in die Mitte greifen: $\mathcal{O}(m+n)$. **Geht besser.**
Schritt 2 – Die Umformulierung.

> *Man sucht eine* ***Schnittstelle****: i Elemente aus A und j Elemente aus B bilden zusammen die linke Hälfte.*

$$i + j = \left\lceil \frac{m+n}{2} \right\rceil.$$

Ist i *gewählt, so ist* j *festgelegt* – **es bleibt eine eindimensionale Suche.**
Schritt 3 – Die Bedingung für die richtige Schnittstelle.

$$\boxed{A[i] \geq B[j-1] \quad \text{und} \quad B[j] \geq A[i-1]}$$

In Worten: Alles links vom Schnitt ist $\leq$ *allem rechts davon.*

Schritt 4 – Binär nach i **suchen.**

$A[i-1] > B[j]$	i ist zu groß – nach links suchen
$B[j-1] > A[i]$	i ist zu klein – nach rechts suchen
sonst	**gefunden**

Schritt 5 – Ein Beispiel.
$A = [1,3,8,9,15]$, $B = [7,11,18,19,21,25]$; $m+n = 11$, *linke Hälfte hat 6 Elemente.*

i	j	$A[i-1] \leq B[j]$?	$B[j-1] \leq A[i]$?	
2	4	$3 \leq 21$ ✓	$19 \leq 8$? nein	i zu klein
4	2	$9 \leq 18$ ✓	$7 \leq 15$ ✓	**gefunden**

Linke Hälfte: $\{1,3,8,9\} \cup \{7,11\}$*; das Maximum davon ist* 11.

$$\boxed{\text{Median} = 11}$$

Probe durch Verschmelzen: $[1,3,7,8,9,\mathbf{11},15,18,19,21,25]$ **– elf Elemente, das sechste ist 11.** ✓

Schritt 6 – Aufwand.
Man sucht binär im kleineren Feld:

$$\boxed{T = \mathcal{O}(\log \min(m,n))}$$

Bei zwei Feldern von je einer Million: 20 Schritte statt zwei Millionen. ✓

Die Randfälle – leere Teilbereiche – behandelt man elegant, indem man $A[-1] := -\infty$ *und* $A[m] := +\infty$ *setzt.* **Dann entfallen sämtliche Sonderfallabfragen.**

(c) Das k-größte im Datenstrom.

Die Daten kommen nacheinander an, ihre Zahl ist unbekannt, und sie passen nicht alle in den Speicher. **Man darf jedes Element nur einmal sehen.**

Die Lösung: ein Min-Heap der festen Größe k.

```
for jedes ankommende x do {
  if Heapgröße < k then einfügen(x)
  else if x > wurzel() then {
entnimm_min(); einfügen(x) } }
return wurzel()
```

Die Invariante:

> Der Heap enthält stets die k größten der bisher gesehenen Elemente.

Warum ein* Min-*Heap für die* größten *Elemente? Weil man wissen muss, welches der aufbewahrten das **schwächste** ist – das ist der Kandidat zum Hinauswerfen. **Und die Wurzel eines Min-Heaps ist genau das.**

Der Aufwand.

Größe	**Wert**	**Bemerkung**
Zeit	$\mathcal{O}(n \log k)$	pro Element $\mathcal{O}(\log k)$
Speicher	$\mathcal{O}(k)$	**unabhängig von** n
Durchläufe	1	**ein einziger**

Zahlenbeispiel: die 100 häufigsten Suchbegriffe aus einer Milliarde Anfragen.

$\mathcal{O}(10^9 \cdot \log_2 100) = 10^9 \cdot 7 = 7 \cdot 10^9$ Operationen, Speicher: 100 Einträge.

Der Speicherbedarf hängt nicht von der Datenmenge ab. Das ist die kennzeichnende Eigenschaft von **Stromalgorithmen** – und der Grund, warum sie in der Datenauswertung so wichtig geworden sind.

Die Grenze dieser Technik. *Der* ***Median*** *eines Stroms lässt sich* nicht *exakt mit wenig Speicher bestimmen* – **man kann beweisen, dass dafür $\Omega(n)$ Speicher nötig sind.**

Für Näherungen genügt aber wenig: Verfahren wie ***t-digest*** *oder* ***Greenwald-Khanna*** *liefern Quantile mit garantierter Fehlerschranke in $\mathcal{O}(\frac{1}{\varepsilon} \log n)$ Speicher* – **und stecken heute in jedem Überwachungssystem, das Antwortzeiten in Perzentilen ausweist.**

Blum und Mitautoren[5] führten die lineare Auswahl ein; Motwani und Raghavan[20] behandeln die randomisierten Varianten; Bentley[2] widmet der binären Suche ein eigenes Kapitel voller Warnungen.

Part III

Bäume

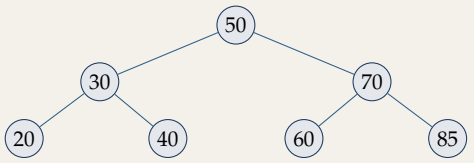
6 Binäre Suchbäume

Die Struktur, die Suchen und Einfügen verbindet

Ein sortiertes Feld erlaubt Suchen in $\mathcal{O}(\log n)$, aber Einfügen kostet $\mathcal{O}(n)$. **Eine verkettete Liste erlaubt Einfügen in $\mathcal{O}(1)$, aber Suchen kostet $\mathcal{O}(n)$.** *Der Suchbaum leistet beides in $\mathcal{O}(\log n)$.*

Definition 6.1. Ein **binärer Suchbaum** ist ein Binärbaum, in dem für jeden Knoten v gilt:

$$\text{alle Schlüssel im linken Teilbaum} \;<\; \text{Schlüssel}(v) \;<\; \text{alle im rechten Teilbaum.}$$



Inorder: 20, 30, 40, 50, 60, 70, 85

Die kennzeichnende Eigenschaft: *Die **Inorder-Durchmusterung** – links, Wurzel, rechts – liefert die Schlüssel in sortierter Reihenfolge.* **Ein Suchbaum ist ein sortiertes Feld, das sich verzweigt hat.**

Die drei Grundoperationen.

Suchen	Vergleiche mit der Wurzel; gehe links oder rechts weiter
Einfügen	Suche, bis man ins Leere läuft; dort hängt man den neuen Knoten an
Löschen	Drei Fälle – siehe unten

Alle drei kosten $\mathcal{O}(h)$, wobei h die Höhe des Baumes ist. **Und damit steht und fällt alles mit der Höhe.**

Das Löschen – drei Fälle

Blatt	einfach entfernen
ein Kind	das Kind an die Stelle rücken
zwei Kinder	**den Nachfolger holen**

Der dritte Fall im Einzelnen.

> *Man kann den Knoten nicht einfach entfernen – es blieben zwei Teilbäume ohne Anschluss.* **Stattdessen ersetzt man seinen Schlüssel durch den *Inorder-Nachfolger*** *– den kleinsten Schlüssel des rechten Teilbaums –* **und löscht diesen dann rekursiv.**

Warum das die Suchbaumeigenschaft erhält: **Der Nachfolger ist größer als alles links und kleiner als alles übrige rechts** *– er passt genau an die freie Stelle.* ✓

*Und der Nachfolger hat höchstens **ein** Kind (er ist der linkeste Knoten des rechten Teilbaums, hat also kein linkes Kind).* **Die Rekursion landet also stets in Fall 1 oder 2 – sie geht nur eine Stufe tief.** ✓

Löschen macht Bäume schief – messbar

Nimmt man immer den Nachfolger, so wird der rechte Teilbaum systematisch ausgedünnt.

> JEFFREY EPPINGER *zeigte 1983 experimentell, dass ein Baum unter abwechselndem Einfügen und Löschen mit dieser Regel **schlechter** wird – die mittlere Suchtiefe wächst über die $\mathcal{O}(\log n)$ hinaus, die man für zufällige Einfügungen erwartet.*

Die Abhilfe ist banal: *Man wählt zufällig zwischen Vorgänger und Nachfolger.* **Dann bleibt der Baum im Mittel ausgewogen.**

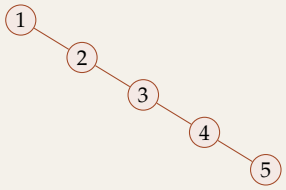
Ein hübsches Beispiel dafür, dass eine willkürlich getroffene Entwurfsentscheidung – „nimm immer den Nachfolger" – messbare Folgen haben kann, die erst zwanzig Jahre später auffallen.

6.1 Die Höhe – das ganze Problem

Zwischen bestem und schlechtestem Fall liegen Welten

Form	Höhe	entsteht bei
vollständig	$\lfloor \log_2 n \rfloor$	idealer Einfügereihenfolge
zufällig	$\approx 4{,}31 \ln n$	zufälliger Reihenfolge
entartet	$n - 1$	**sortierter Eingabe**

Der entartete Fall.
Fügt man $1, 2, 3, 4, 5$ *in dieser Reihenfolge ein, so entsteht:*



eine verkettete Liste mit Zusatzkosten

> ***Und wieder ist der häufigste Praxisfall der schlimmste*** – genau wie bei Quicksort mit festem Pivot (Kapitel 4). **Wer Daten aus einer sortierten Datei einliest, erzeugt eine Liste.**

Satz 6.2 (Mittlere Höhe). *Fügt man n Schlüssel in zufälliger Reihenfolge ein, so beträgt die erwartete* ***mittlere Suchtiefe*** *etwa* $2 \ln n \approx 1{,}39 \log_2 n$ *und die erwartete* ***Höhe*** *etwa* $4{,}31 \ln n \approx 2{,}99 \log_2 n$.

Die mittlere Suchtiefe – der Beweis.

> *Das Einfügen von n Schlüsseln in zufälliger Reihenfolge erzeugt* ***genau denselben*** *Baum, den Quicksort mit zufälligem Pivot als Rekursionsbaum aufbaut.*

Denn: Der erste eingefügte Schlüssel wird zur Wurzel und teilt die übrigen in „kleiner" und „größer" – ***genau wie ein Pivot.*** ✓

Also ist die Summe aller Suchtiefen gleich der Zahl der Quicksort-Vergleiche (Kapitel 4):

$$\mathbb{E}\left[\sum_v \text{Tiefe}(v)\right] \approx 2n \ln n \implies \text{mittlere Tiefe} \approx 2 \ln n. \qquad \checkmark$$

> ***Das ist eine der schönsten Entsprechungen der Algorithmik:*** Ein Sortierverfahren und eine Datenstruktur haben dieselbe Analyse, weil sie im Kern dasselbe tun.

Der Erwartungswert hilft nicht, wenn die Eingabe nicht zufällig ist

$2 \ln n$ **ist ein Mittelwert über** ***zufällige Einfügereihenfolgen.***

Reale Daten kommen sortiert, umgekehrt sortiert oder blockweise sortiert – **fast nie zufällig** (Kapitel 1).

Es gibt zwei Auswege:

Zufall einbauen	Treaps, randomisierte Suchbäume: die Struktur würfelt selbst
Balance erzwingen	AVL, Rot-Schwarz, B-Bäume: **Kapitel 7**

6.2 Durchmusterung

Vier Reihenfolgen, vier Anwendungen

Ordnung	Reihenfolge	wozu
Inorder	links, W, rechts	**sortierte Ausgabe**
Preorder	W, links, rechts	Baum kopieren, serialisieren
Postorder	links, rechts, W	Baum löschen, Ausdruck auswerten
Levelorder	ebenenweise	Breitensuche, Ausgabe

Zur dritten Zeile. *Beim Löschen muss man die Kinder freigeben,* ***bevor*** *man den Elternknoten freigibt – sonst verliert man die Zeiger.* **Postorder ist die einzige Reihenfolge, die das leistet.**

> *Und die Auswertung eines Ausdrucksbaums ist Postorder* – **das ist genau die umgekehrte polnische Notation** (Kapitel 2). *Der Zusammenhang ist kein Zufall: Postfixnotation* ist *die Postorder-Ausgabe des Syntaxbaums.*

Die ersten drei brauchen einen Stapel, die vierte eine Warteschlange.

Das ist der Kern eines Musters, das in Kapitel 10 bei Graphen wiederkehrt: **Stapel ergibt Tiefensuche, Warteschlange ergibt Breitensuche.** *Der Algorithmus ist derselbe – nur die Datenstruktur wechselt.*

Aufgabe 6.1.

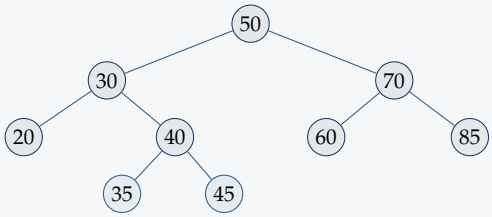
(a) Bauen Sie einen Suchbaum auf, führen Sie alle Löschfälle vor und prüfen Sie die Ordnung.

(b) Prüfen Sie, ob ein gegebener Binärbaum ein Suchbaum ist – und finden Sie den häufigsten Fehler.

(c) Zählen Sie, wie viele verschiedene Suchbäume es mit n Schlüsseln gibt.

Lösung Schritt für Schritt.

(a) Aufbau und Löschen.
Wir fügen ein: 50, 30, 70, 20, 40, 60, 85, 35, 45.



Fall 1 – Blatt löschen: die 35. *Einfach abhängen.* **Der Baum bleibt gültig.** ✓
Fall 2 – Ein Kind: die 20 nach Einfügen von 25. *Die 20 hat nur ein rechtes Kind (25). Man hängt 25 direkt unter die 30.* ✓
Fall 3 – Zwei Kinder: die 30 löschen.

1. Rechter Teilbaum von 30 ist der mit Wurzel 40.
2. Dessen kleinster Schlüssel: von 40 nach links zur 35 – **Nachfolger ist 35**.
3. Schlüssel 30 wird durch 35 ersetzt.
4. Der ursprüngliche Knoten 35 (ein Blatt) wird gelöscht.

Die Probe – Inorder vor und nach dem Löschen.

vorher	20, 30, 35, 40, 45, 50, 60, 70, 85
nachher	20, 35, 40, 45, 50, 60, 70, 85

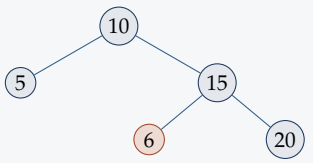
Sortiert, und genau die 30 fehlt. **Der Baum ist noch ein Suchbaum.** ✓

Die Inorder-Ausgabe ist die beste Probe für jede Suchbaumoperation. Ist sie sortiert, stimmt die Struktur – ist sie es nicht, hat man einen Fehler.

(b) Ist das ein Suchbaum?
Der falsche Ansatz – und warum ihn fast jeder zuerst schreibt.

```
ist_suchbaum(v):      FALSCH
  if v = nil then return wahr;
  if v.links <> nil and v.links.wert >
v.wert then return falsch;
  if v.rechts <> nil and v.rechts.wert
< v.wert then return falsch;
  return ist_suchbaum(v.links) and
ist_suchbaum(v.rechts)
```

Das Gegenbeispiel.



6 < 10 – gehört nach links!

Lokal ist alles in Ordnung: $6 < 15$ *steht links von der 15.* **Global ist es falsch: Die 6 steht im *rechten* Teilbaum der 10 und muss deshalb** > 10 **sein.**

Die Inorder-Ausgabe ist $5, 10, 6, 15, 20$ **– nicht sortiert. ✓**
Der richtige Ansatz – Schranken mitführen.

```
pruefe(v, min, max):
  if v = nil then return wahr;
  if v.wert <= min or v.wert >= max
then return falsch;
  return pruefe(v.links, min, v.wert)
    and pruefe(v.rechts, v.wert, max)
```

Aufruf: `pruefe(wurzel, −∞, +∞)`.

Beim Abstieg nach links wird die obere Schranke enger, beim Abstieg nach rechts die untere. Jeder Knoten kennt damit den Bereich, in dem sein Wert liegen muss.

Am Gegenbeispiel: Die 6 wird mit `pruefe(v, 10, 15)` aufgerufen; $6 \leq 10$ **– Fehler erkannt. ✓**
Der zweite richtige Ansatz – Inorder mit Merker.

```
Durchlaufe inorder und merke den
zuletzt gesehenen Wert;
ist ein Wert nicht größer als der
vorige, ist es kein Suchbaum.
```

Beide Verfahren kosten $\mathcal{O}(n)$. **Das zweite ist kürzer, das erste erklärt besser, worum es geht.**

(c) Wie viele Suchbäume gibt es?
Schritt 1 – Die Rekursion aufstellen.

Sei C_n *die Zahl der Suchbaumformen mit n Schlüsseln.* **Man wähle, welcher Schlüssel die Wurzel wird.**

Ist der k-kleinste Schlüssel die Wurzel, so enthält der linke Teilbaum $k-1$ *Schlüssel und der rechte* $n-k$*:*

$$\boxed{C_n = \sum_{k=1}^{n} C_{k-1} \cdot C_{n-k}, \qquad C_0 = 1}$$

Schritt 2 – Die Werte ausrechnen.

n	**Rechnung**	C_n
0	—	1
1	C_0C_0	1
2	$C_0C_1 + C_1C_0$	2
3	$C_0C_2 + C_1C_1 + C_2C_0 = 2 + 1 + 2$	5
4	$5 + 2 + 2 + 5$	14
5	$14 + 5 + 4 + 5 + 14$	42

Schritt 3 – Die Folge erkennen.

$$1, 1, 2, 5, 14, 42, 132, 429, \ldots$$

Das sind die ***Catalanzahlen*** (Mathematik-Band 15, Kapitel 7):

$$C_n = \frac{1}{n+1}\binom{2n}{n}.$$

Probe für $n = 4$: $\frac{1}{5}\binom{8}{4} = \frac{70}{5} = 14$. ✓

Schritt 4 – Das Wachstum.

$$C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}.$$

n	C_n	**davon ausgewogen**
10	16 796	wenige
20	$6{,}56 \cdot 10^9$	
30	$3{,}81 \cdot 10^{15}$	

Von den C_n ***möglichen Formen sind die allermeisten ziemlich unausgewogen.*** Der ideale Baum ist einer unter Milliarden – und deshalb genügt es nicht, auf Glück zu hoffen.

Schritt 5 – Der wichtige Unterschied.

	Anzahl	**Verteilung**
Baum**formen**	C_n	jede gleich oft?
Einfüge**reihenfolgen**	$n!$	**jede gleich wahrscheinlich**

> ***Das ist der Schlüssel zur mittleren Höhe.*** Da $n! \gg C_n$, führen **viele verschiedene Reihenfolgen zum selben Baum** – und zwar besonders viele zu den *ausgewogenen*.

*Beispiel $n = 3$: Von den $3! = 6$ Reihenfolgen erzeugen **zwei** den ausgewogenen Baum mit Wurzel 2 (nämlich $2, 1, 3$ und $2, 3, 1$), aber nur **je eine** die vier entarteten Formen.*

*Deshalb ist der **zufällig aufgebaute** Baum im Mittel gut, obwohl die **zufällig gewählte Form** es nicht wäre* – **ein Unterschied, der bei der Analyse leicht übersehen wird.**

Cormen u. a.[6] behandeln Suchbäume vollständig; Sedgewick und Wayne[24] zeigen die Zusammenhänge zu Quicksort; Knuth[17] enthält die exakte Analyse der mittleren Höhe.

7 Balancierte Bäume

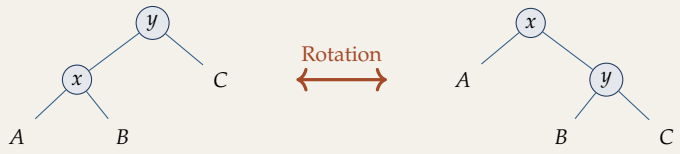
Die Aufgabe

Ein Suchbaum ist gut, solange er ausgewogen bleibt (Kapitel 6). **Man braucht also ein Verfahren, das die Balance beim Einfügen und Löschen *erzwingt*** – *und zwar in $\mathcal{O}(\log n)$, sonst gewinnt man nichts.*

Struktur	Balancekriterium	Höhe
AVL	Höhendifferenz ≤ 1	$\leq 1{,}44 \log_2 n$
Rot-Schwarz	Schwarzhöhe gleich	$\leq 2 \log_2 n$
B-Baum	alle Blätter gleich tief	$\mathcal{O}(\log_t n)$
Treap	Zufallsprioritäten	$\approx 3 \log_2 n$ erwartet
Splay	keines – Selbstanpassung	amortisiert $\mathcal{O}(\log n)$

Das gemeinsame Werkzeug: die Rotation.

*Eine **Rotation** verändert die Form eines Baumes lokal, **ohne die Suchbaumordnung zu verletzen.** **Sie ist der einzige Eingriff, den alle Balancierungsverfahren benutzen.***



Warum die Ordnung erhalten bleibt.
In beiden Bäumen gilt die Inorder-Reihenfolge

$$A \prec x \prec B \prec y \prec C.$$

Man liest sie links und rechts ab und vergleicht. **Sie ist identisch** – *und damit ist der rotierte Baum wieder ein Suchbaum.* ✓

Die Rotation kostet $\mathcal{O}(1)$: Es werden genau drei Zeiger umgehängt. **Das ist der Grund, warum Balancierung überhaupt bezahlbar ist.**

7.1 AVL-Bäume

Die erste balancierte Struktur – 1962

Definition 7.1. Ein **AVL-Baum** ist ein Suchbaum, in dem sich für jeden Knoten die Höhen der beiden Teilbäume um höchstens 1 unterscheiden.
Der **Balancefaktor** ist $\mathrm{bf}(v) = h(\text{rechts}) - h(\text{links}) \in \{-1, 0, +1\}$.

Benannt nach Georgi Adelson-Velski *und* Jewgeni Landis, *die die Struktur 1962 in Moskau veröffentlichten* – **die erste Datenstruktur mit garantiert logarithmischer Höhe.**

Satz 7.2 (Höhenschranke). *Ein AVL-Baum mit n Knoten hat die Höhe*

$$h \leq 1{,}4405 \log_2(n+2) - 0{,}328.$$

Der Beweis – und die Fibonaccizahlen tauchen auf.
*Sei $N(h)$ die **kleinste** Knotenzahl eines AVL-Baums der Höhe h – also der schlimmstmögliche Fall.*

Ein solcher Baum hat eine Wurzel, einen Teilbaum der Höhe $h-1$ und – um die Balance gerade noch einzuhalten – einen der Höhe $h-2$.

$$\boxed{N(h) = 1 + N(h-1) + N(h-2), \qquad N(0) = 1,\ N(1) = 2}$$

h	0	1	2	3	4
$N(h)$	1	2	4	7	12
F_{h+3}	2	3	5	8	13

Man erkennt: $N(h) = F_{h+3} - 1$ **mit den Fibonaccizahlen.** ✓

Mit der Binetschen Formel (Mathematik-Band 15, Kapitel 5) *gilt* $F_k \approx \frac{\varphi^k}{\sqrt{5}}$ *mit* $\varphi = 1{,}618$, *also*

$$n \geq N(h) \approx \frac{\varphi^{h+3}}{\sqrt{5}} \Longrightarrow h \leq \log_\varphi(n\sqrt{5}) - 3 = \frac{\log_2 n}{\log_2 \varphi} + \ldots$$

$$\frac{1}{\log_2 \varphi} = \frac{1}{0{,}6942} = 1{,}4405. \qquad \checkmark$$

Der schlimmste AVL-Baum ist also höchstens 44 Prozent höher als der bestmögliche. Und diese Bäume heißen tatsächlich **Fibonaccibäume** – sie sind die dünnsten, die die Bedingung noch erfüllen.

Die vier Rotationsfälle.

Fall	Situation	Behandlung
LL	links, dann links	eine Rechtsrotation
RR	rechts, dann rechts	eine Linksrotation
LR	links, dann rechts	**zwei** Rotationen
RL	rechts, dann links	**zwei** Rotationen

Bei den gemischten Fällen genügt eine Rotation nicht – der Baum kippt nur auf die andere Seite. **Man dreht zuerst das Kind in die passende Richtung, dann den Knoten selbst.**

Operation	Rotationen	Bemerkung
Einfügen	**höchstens 1** (bzw. 1 doppelte)	danach ist der Baum wieder gut
Löschen	bis zu $\mathcal{O}(\log n)$	Umbalancierung kann bis zur Wurzel laufen

Der Unterschied ist erklärungsbedürftig: **Beim Einfügen wächst die Höhe des reparierten Teilbaums nicht über den Ausgangswert – der Rest des Baumes merkt nichts.** *Beim Löschen kann sie um 1 sinken, und das pflanzt sich nach oben fort.*

7.2 Rot-Schwarz-Bäume

Die Struktur der Bibliotheken

Definition 7.3. Ein **Rot-Schwarz-Baum** ist ein Suchbaum, dessen Knoten rot oder schwarz gefärbt sind, mit den Regeln:

1. Jeder Knoten ist rot oder schwarz.
2. Die Wurzel ist schwarz.
3. Alle Blätter (gedachte leere Knoten) sind schwarz.
4. **Ein roter Knoten hat nur schwarze Kinder** – keine zwei roten hintereinander.
5. **Für jeden Knoten enthalten alle Wege zu den Blättern gleich viele schwarze Knoten.**

Satz 7.4. *Ein Rot-Schwarz-Baum mit n inneren Knoten hat die Höhe $h \leq 2\log_2(n+1)$.*

Der Beweis in zwei Zeilen.
*Sei s die **Schwarzhöhe** – die Zahl schwarzer Knoten auf einem Weg zur Wurzel. Nach Regel 5 ist sie für alle Wege gleich.*
Wegen Regel 4 ist höchstens jeder zweite Knoten auf einem Weg rot:

$$h \leq 2s.$$

Und ein Baum, in dem jeder Weg s schwarze Knoten hat, enthält mindestens $2^s - 1$ innere Knoten:

$$n \geq 2^s - 1 \Longrightarrow s \leq \log_2(n+1) \Longrightarrow h \leq 2\log_2(n+1). \qquad \checkmark$$

Regel 4 und 5 zusammen leisten alles: Regel 5 sorgt dafür, dass die schwarzen Wege gleich lang sind; Regel 4 begrenzt, wie viel Rot dazwischen passt. **Der längste Weg ist höchstens doppelt so lang wie der kürzeste.**

	AVL	**Rot-Schwarz**
Höhe	$\leq 1{,}44 \log n$	$\leq 2 \log n$
Suchen	**schneller**	etwas langsamer
Rotationen beim Einfügen	≤ 1	≤ 2
Rotationen beim Löschen	$\mathcal{O}(\log n)$	≤ 3
Zusatzspeicher	2 Bit (Balance)	1 Bit (Farbe)

> ***Die vierte Zeile hat die Praxis entschieden.*** Rot-Schwarz-Bäume brauchen beim Löschen **konstant** viele Rotationen – AVL-Bäume bis zu logarithmisch viele. *Bei änderungslastigen Anwendungen zählt das mehr als die etwas geringere Höhe.*

Deshalb sind Rot-Schwarz-Bäume die Struktur hinter `std::map` *(C++),* `TreeMap` *(Java), dem Prozessscheduler von Linux und vielen Dateisystemen.* **AVL-Bäume findet man dort, wo fast nur gesucht wird** *– etwa in Datenbankindizes im Hauptspeicher.*

Warum die Farben Rot und Schwarz

RUDOLF BAYER führte die Struktur 1972 unter dem Namen „symmetrische binäre B-Bäume" ein. LEONIDAS GUIBAS *und* ROBERT SEDGEWICK *gaben ihr 1978 die heutige Form und die Farben.*

> *Sedgewick wurde später oft gefragt, warum ausgerechnet Rot und Schwarz.* **Seine Antwort in einem Vortrag von 2007: Der Laserdrucker bei Xerox PARC konnte damals besonders schönes Rot drucken.**

Er fügte hinzu, es gebe auch eine literarische Deutung – STENDHALS *Roman „Le Rouge et le Noir" – und man habe damals über Spielkartenfarben nachgedacht.* **Aber der Drucker war der eigentliche Grund.**

Bayer selbst war da unbeteiligt. *Er hatte 1970 zusammen mit* EDWARD MCCREIGHT *bei Boeing den* ***B-Baum*** *erfunden, und auch dort ist der Name ein Rätsel geblieben.*

> *McCreight sagte 2013 in einem Interview, das „B" habe nie eine feste Bedeutung gehabt –* **es könne für „balanced", für „Bayer", für „Boeing" oder für „broad" stehen.** *„Je mehr man darüber nachdenkt, was das B bedeutet, desto besser versteht man B-Bäume."*

7.3 B-Bäume

Die Struktur für den Massenspeicher

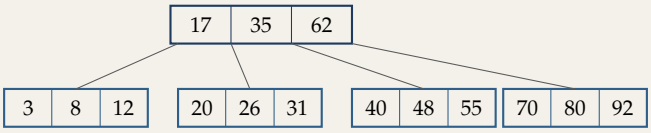
> *Alle bisherigen Strukturen setzen voraus, dass ein Speicherzugriff billig ist.* **Bei einer Festplatte oder SSD ist er das nicht** *– er kostet das Zehntausend- bis Millionenfache eines Hauptspeicherzugriffs* (Band 0, Kapitel 12).

Und man kann nicht einzelne Bytes lesen, sondern nur ganze ***Blöcke*** *von 4 bis 16 Kilobyte.*

> ***Daraus folgt der Entwurf unmittelbar:*** *Man mache jeden Knoten so groß wie einen Block* **– und packe so viele Schlüssel hinein, wie hineingehen.**

Definition 7.5. Ein **B-Baum** der Ordnung t erfüllt:

1. Jeder Knoten enthält zwischen $t-1$ und $2t-1$ Schlüssel (die Wurzel mindestens 1).
2. Ein innerer Knoten mit k Schlüsseln hat $k+1$ Kinder.
3. Die Schlüssel im Knoten sind sortiert und trennen die Kinderbereiche.
4. **Alle Blätter liegen auf derselben Ebene.**



Satz 7.6. *Ein B-Baum der Ordnung t mit n Schlüsseln hat die Höhe*

$$h \leq \log_t \frac{n+1}{2}.$$

Die Rechnung, die den Unterschied zeigt.
Ein Knoten von 4096 Byte fasst bei 16 Byte je Eintrag (Schlüssel plus Zeiger) etwa 255 Schlüssel, also $t = 128$.

Höhe	Schlüssel	Plattenzugriffe
1	255	1
2	65 535	2
3	$1{,}7 \cdot 10^7$	3
4	$4{,}3 \cdot 10^9$	**4**

Vier Milliarden Schlüssel in vier Plattenzugriffen. Ein Rot-Schwarz-Baum bräuchte dafür etwa $2\log_2(4 \cdot 10^9) = 64$ Zugriffe – **sechzehnmal so viele.**

Und die obersten ein bis zwei Ebenen stehen ohnehin im Cache – **in der Praxis genügen oft zwei echte Zugriffe.**

Das ist der Grund, warum jede Datenbank und jedes moderne Dateisystem B-Bäume benutzt – MySQL, PostgreSQL, Oracle, SQLite, NTFS, ext4, Btrfs, APFS. *Meist in der Variante **B^+-Baum**, bei der alle Daten in den Blättern liegen und diese zusätzlich verkettet sind.*

Die Verkettung der Blätter ist der Grund, warum Bereichsabfragen so schnell sind: *Man sucht den Anfang in $\mathcal{O}(\log_t n)$ und läuft dann sequentiell weiter* – **ohne den Baum wieder hinaufzusteigen.**

Bei `SELECT * FROM t WHERE datum BETWEEN a AND b` *ist genau das die Zugriffsart.*

Aufgabe 7.1.

(a) Führen Sie alle vier AVL-Rotationsfälle an konkreten Zahlen vor.
(b) Bauen Sie einen B-Baum der Ordnung $t = 2$ durch Einfügen auf.
(c) Untersuchen Sie den Treap – einen Baum, der sich selbst zufällig ausbalanciert.

Lösung Schritt für Schritt.

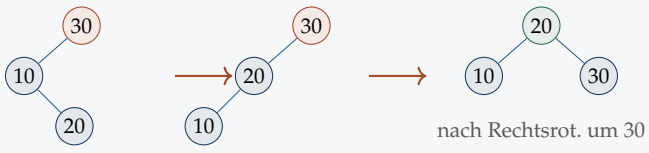
(a) Die vier Rotationsfälle.
Fall RR – Einfügen von $10, 20, 30$**.**

Schritt	Baum	Balancefaktor Wurzel
10	10	0
20	10 $\rightarrow$rechts$\rightarrow$ 20	$+1$
30	Kette $10, 20, 30$	$+2$ – Verstoß

Eine ***Linksrotation*** *um die 10:*



Höhe von 3 auf 2, Balancefaktor 0. ✓
Fall LL – Einfügen von $30, 20, 10$**.** *Spiegelbildlich; eine Rechtsrotation.* ✓
Fall LR – Einfügen von $30, 10, 20$**.**



Warum eine Rotation nicht genügt: Eine bloße Rechtsrotation um die 30 ergäbe die Kette $10 \rightarrow 20 \rightarrow 30$ nach rechts – **genauso schief wie vorher, nur andersherum.**

Die erste Rotation richtet das „Zickzack" zu einer geraden Kette; die zweite hebt die Mitte nach oben. ✓
Fall RL – *Einfügen von* $10, 30, 20$*; spiegelbildlich zu LR.* ✓
Die Merkregel.

gerade Kette (LL, RR)	**eine** Rotation, in die Gegenrichtung
Zickzack (LR, RL)	**erst geradebiegen**, dann rotieren

(b) Ein B-Baum der Ordnung $t = 2$**.**
Bei $t = 2$ *fasst jeder Knoten 1 bis 3 Schlüssel und hat 2 bis 4 Kinder* (**2-3-4-Baum**).
Wir fügen ein: $1, 2, 3, 4, 5, 6, 7$.

1	$[1]$
2	$[1, 2]$
3	$[1, 2, 3]$ – voll
4	**Knoten läuft über: spalte bei der 2**
	Wurzel $[2]$, Kinder $[1]$ und $[3, 4]$
5	Wurzel $[2]$, Kinder $[1]$ und $[3, 4, 5]$
6	**Spalte bei der 4:** Wurzel $[2, 4]$, Kinder $[1]$, $[3]$, $[5, 6]$
7	Wurzel $[2, 4]$, Kinder $[1]$, $[3]$, $[5, 6, 7]$

Die Beobachtung, die B-Bäume ausmacht.

> ***Der Baum wächst nicht an den Blättern, sondern an der Wurzel.*** Läuft ein Knoten über, wandert sein mittlerer Schlüssel nach oben. **Läuft die Wurzel über, entsteht eine neue Wurzel – und der ganze Baum wird um eine Ebene höher.**

Das ist der einzige Weg, auf dem die Höhe zunimmt – **und deshalb bleiben alle Blätter automatisch auf gleicher Ebene.** *Man muss die Bedingung 4 gar nicht durchsetzen; sie ergibt sich von selbst.* ✓

Zum Vergleich: Der binäre Suchbaum mit derselben Eingabe $1, \ldots, 7$ *wäre eine Kette der Höhe 7* (Kapitel 6). **Der B-Baum hat Höhe 2.** ✓

(c) Der Treap.

> *Der Name ist ein Kofferwort aus* ***Tree*** *und* ***Heap*** **– die Struktur ist beides zugleich.**

Definition 7.7. Ein **Treap** speichert zu jedem Element einen Schlüssel und eine **zufällige Priorität**. Er ist

nach den **Schlüsseln**	ein Suchbaum (links kleiner, rechts größer)
nach den **Prioritäten**	ein Heap (Eltern haben höhere Priorität)

Schritt 1 – Warum die Form eindeutig ist.

*Bei gegebenen Schlüssel-Prioritäts-Paaren gibt es **genau einen** Treap.* **Denn: Die Wurzel muss das Element höchster Priorität sein, und die Schlüsselordnung legt fest, was links und rechts davon liegt** – *rekursiv fort bis nach unten.*

Schritt 2 – Die Analyse, ohne jede Rechnung.

*Die Wurzel ist das Element mit der höchsten Priorität – und weil die Prioritäten zufällig sind, ist das ein **gleichverteilt zufälliges** Element.*

Das ist genau die Situation eines Suchbaums, in den man in zufälliger Reihenfolge eingefügt hat (Kapitel 6) – **oder von Quicksort mit zufälligem Pivot.**

$$\boxed{\mathbb{E}[\text{Höhe}] \approx 3\log_2 n, \qquad \mathbb{E}[\text{Suchtiefe}] \approx 1{,}39\log_2 n}$$

Die gesamte Analyse ist ein Verweis auf ein bekanntes Ergebnis. Man braucht keinen einzigen neuen Beweis – die Struktur wurde so entworfen, dass die Analyse geschenkt ist.

Schritt 3 – Einfügen.

1. Wie im gewöhnlichen Suchbaum als Blatt einfügen.
2. Würfle eine Priorität.
3. Solange die Priorität größer als die des Elternknotens ist: **rotiere nach oben**.

Die Rotation erhält die Schlüsselordnung (siehe Kapitelanfang) *und repariert die Heapordnung.* ✓

Schritt 4 – Der Vergleich.

	AVL	Rot-Schw.	Treap
Garantie	deterministisch	determ.	erwartet
Codelänge	lang	sehr lang	**kurz**
Löschen	aufwendig	aufwendig	einfach
Vereinigen zweier Bäume	schwierig	schwierig	$\mathcal{O}(\log n)$

Die zweite Zeile ist ein ernsthaftes Argument. Eine korrekte Rot-Schwarz-Löschoperation umfasst sechs Fälle und etwa hundert Zeilen; ein Treap-Löschen sind zehn. **Und kürzerer Code hat weniger Fehler.**

Die letzte Zeile ist der eigentliche Vorzug. *Treaps lassen sich* ***spalten*** *und* ***vereinigen*** *– Operationen, die bei Rot-Schwarz-Bäumen sehr aufwendig sind.* **Das macht sie zur bevorzugten Struktur für Aufgaben, bei denen ganze Bereiche verschoben oder eingefügt werden müssen** – *etwa in Texteditoren, wo ein Ausschneiden-und-Einfügen genau das ist.*

CECILIA ARAGON *und* RAIMUND SEIDEL *führten Treaps 1989 ein.* **Aragon war damals Doktorandin in Berkeley; sie wurde später Professorin für Datenwissenschaft und ist zugleich Kunstflugpilotin** – *1993 Mitglied der US-Nationalmannschaft im Kunstflug.*

Ottmann und Widmayer[22] behandeln sämtliche Balancierungsverfahren mit vollständigen Fallunterscheidungen; Tarjan[26] entwickelt die amortisierte Sicht; Mehlhorn und Sanders[19] zeigen die praktische Auswahl.

8 Heaps und Prioritätswarteschlangen

Die Aufgabe: immer das Wichtigste zuerst

Definition 8.1. Eine **Prioritätswarteschlange** verwaltet eine Menge von Elementen mit Schlüsseln und bietet:

`einfügen(x)`	Element hinzufügen
`minimum()`	kleinstes Element ansehen
`entnimm_min()`	kleinstes Element entfernen und liefern
`verringere(x,k)`	Schlüssel eines Elements herabsetzen

*Man beachte, was **nicht** dabei ist: Suchen nach beliebigen Schlüsseln.* **Eine Prioritätswarteschlange ist keine Suchstruktur** – *sie leistet weniger als ein Suchbaum und ist dafür einfacher und schneller.*

Umsetzung	**einfügen**	**entnimm**	**verringere**
unsortiertes Feld	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
sortiertes Feld	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
binärer Heap	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Fibonacci-Heap	$\mathcal{O}(1)$	$\mathcal{O}(\log n)^*$	$\mathcal{O}(1)^*$

* *amortisiert.*

Die Anwendungen sind zahlreich:

Dijkstra	nächstgelegene unbesuchte Ecke (Kap. 11)
Prim	leichteste Kante an den Baum (Kap. 12)
Huffman	zwei seltenste Symbole (Kap. 13)
Scheduler	Prozess höchster Priorität
Ereignissimulation	nächstes Ereignis auf der Zeitachse
Heapsort	Maximum entnehmen (Kap. 4)

8.1 Der binäre Heap

Ein Baum, der ein Feld ist

Definition 8.2. Ein **Min-Heap** ist ein **vollständiger** Binärbaum (alle Ebenen voll, die unterste von links gefüllt), in dem für jeden Knoten gilt

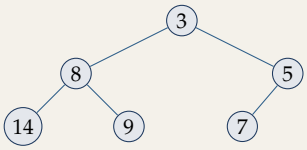
$$\text{Schlüssel}(\text{Elternknoten}) \leq \text{Schlüssel}(\text{Kind}).$$

Die Heapbedingung ist schwächer als die Suchbaumbedingung. Sie sagt nur etwas über Eltern und Kinder, nichts über Geschwister. **Deshalb ist die Wurzel das Minimum – aber sonst weiß man wenig.**

Die entscheidende Beobachtung: Man braucht keine Zeiger.

*Weil der Baum **vollständig** ist, kann man ihn ebenenweise in ein Feld schreiben* **– und die Verwandtschaftsbeziehungen ergeben sich durch Rechnung.**

Beziehung	**ab Index 1**	**ab Index 0**
Elternknoten von i	$\lfloor i/2 \rfloor$	$\lfloor (i-1)/2 \rfloor$
linkes Kind	$2i$	$2i+1$
rechtes Kind	$2i+1$	$2i+2$





Kein Zeiger, kein Speicherverschnitt, perfekte Cachelokalität. Ein Heap von einer Million Elementen ist ein Feld von einer Million Zahlen – nicht mehr. **Ein Rot-Schwarz-Baum bräuchte dafür etwa das Vierfache.**

Die beiden Grundoperationen.

Aufsteigen	Nach dem Einfügen am Ende: solange kleiner als der Elternknoten, mit ihm tauschen
Absinken	Nach dem Entnehmen der Wurzel: letztes Element nach oben, dann mit dem **kleineren** Kind tauschen, solange nötig

Beide laufen höchstens die Höhe entlang – $\mathcal{O}(\log n)$. ✓

Beim Absinken muss es das *kleinere* Kind sein

Tauscht man mit dem größeren, ist die Heapbedingung danach verletzt. *Beispiel: Wurzel 9, Kinder 3 und 5.*

Tausch (kleiner)	mit	3	3 oben, 9 und 5	gültig
Tausch (größer)	mit	5	5 oben, 3 und 9	$5 > 3$ – Verstoß

Ein Fehler, der beim ersten Selbstschreiben fast garantiert passiert. ✓

Der Heapaufbau – linear, nicht $n \log n$

Wie macht man aus einem beliebigen Feld einen Heap? **Der naive Weg: n-mal einfügen, also $\mathcal{O}(n \log n)$.** *Es geht besser.*

Floyds Verfahren (1964): von unten nach oben.

```
for i := n div 2 downto 1 do
absinken(i)
```

Die hintere Hälfte des Feldes besteht aus Blättern – sie sind trivial schon Heaps. **Man beginnt beim letzten inneren Knoten und arbeitet sich zur Wurzel vor.**

Satz 8.3. *Der Heapaufbau nach Floyd kostet $\Theta(n)$.*

Der Beweis – eine überraschende Summe.
Ein Knoten auf Höhe h (über den Blättern gezählt) kostet beim Absinken $\mathcal{O}(h)$. Und es gibt höchstens $\lceil n/2^{h+1} \rceil$ Knoten der Höhe h:

$$T(n) \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} \cdot c\,h \;\leq\; \frac{cn}{2} \sum_{h=0}^{\infty} \frac{h}{2^h}.$$

Die Reihe $\sum_{h\geq 0} h\,x^h = \frac{x}{(1-x)^2}$ liefert bei $x = \frac{1}{2}$:

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1/2)^2} = 2.$$

$$\boxed{T(n) \leq cn \Longrightarrow \Theta(n)}$$

Der Grund für das überraschende Ergebnis: *Die* ***meisten*** *Knoten sind Blätter oder blattnah und kosten fast nichts.* ***Die wenigen teuren Knoten sitzen oben und sind selten.***

Höhe	Knoten	Kosten je	Summe
0	$n/2$	0	0
1	$n/4$	1	$n/4$
2	$n/8$	2	$n/4$
3	$n/16$	3	$3n/16$
$\log n$	1	$\log n$	$\log n$

Die Spaltensumme fällt geometrisch – **daher die Konvergenz.**

Und trotzdem ist Heapsort nicht linear

Der *Aufbau* kostet $\Theta(n)$, aber der *Abbau* kostet $\Theta(n \log n)$.

Beim Abbau nimmt man jedes Mal die Wurzel und lässt ein Blatt von ganz oben absinken – **das ist der teure Fall, und er tritt n-mal ein.**

Die Asymmetrie ist lehrreich: Beim Aufbau sinken die ***meisten*** *Elemente nur wenig; beim Abbau sinkt* ***jedes*** *Element fast die volle Höhe.*

8.2 Fortgeschrittene Heaps

Der Fibonacci-Heap – Theorie gegen Praxis

MICHAEL FREDMAN *und* ROBERT TARJAN *entwarfen 1984 eine Struktur, bei der* `einfügen` *und* `verringere` *amortisiert* $\mathcal{O}(1)$ *kosten.*

Warum das wichtig ist. *Dijkstras Algorithmus* (Kapitel 11) *ruft* `verringere` *bis zu m-mal auf und* `entnimm_min` *n-mal:*

Heap	**Dijkstra**	**bei** $m = n^2$
binär	$\mathcal{O}((n+m)\log n)$	$\mathcal{O}(n^2 \log n)$
Fibonacci	$\mathcal{O}(m + n\log n)$	$\mathcal{O}(n^2)$

Ein Faktor $\log n$ *bei dichten Graphen* – **das ist ein echtes theoretisches Ergebnis.**

Die Idee: faul sein und später aufräumen.

Einfügen	Neuen Einzelbaum in eine Wurzelliste hängen – keine Ordnung herstellen
Verringern	Teilbaum abschneiden und in die Wurzelliste hängen
Entnehmen	**Jetzt** aufräumen: Bäume gleichen Grades verschmelzen

Die gesamte Arbeit wird auf die eine Operation geschoben, die ohnehin teuer ist. Das ist amortisierte Analyse in Reinform (Kapitel 1).

In der Praxis wird er kaum eingesetzt

Konstanten	Groß – jeder Knoten braucht vier Zeiger, Grad und Markierung
Cacheverhalten	schlecht – Knoten liegen verstreut
Codelänge	Ein korrekter Fibonacci-Heap sind mehrere hundert Zeilen
Wirklicher Bedarf	Bei dünnen Graphen ($m = \mathcal{O}(n)$) bringt er nichts

Messungen zeigen regelmäßig, dass ein schlichter binärer Heap – oder ein d-närer Heap mit $d = 4$ – in der Praxis schneller ist, **bis zu Graphengrößen, die weit jenseits des Üblichen liegen.**

Der Fibonacci-Heap ist trotzdem kein Fehlschlag. *Er zeigte, dass die Schranke erreichbar ist – und diese Erkenntnis hat eine ganze Reihe weiterer Ergebnisse angestoßen.*

Praktisch brauchbare Nachfolger gibt es inzwischen: ***Pairing-Heaps*** *(1986) sind fast so gut in der Theorie und deutlich einfacher –* **sie sind die übliche Wahl, wenn man `verringere` wirklich braucht.**

Aufgabe 8.1.

(a) Bauen Sie einen Heap auf und führen Sie Heapsort vollständig durch.
(b) Verschmelzen Sie k sortierte Listen mit insgesamt n Elementen effizient.
(c) Bestimmen Sie den laufenden Median eines Datenstroms.

Lösung Schritt für Schritt.

(a) Heapaufbau und Heapsort.
Feld: $[4, 10, 3, 5, 1]$, *gewünscht: Max-Heap für aufsteigendes Sortieren.*
Schritt 1 – Aufbau von unten.
$n = 5$, *also beginnt man bei* $i = \lfloor 5/2 \rfloor = 2$.

i	**Knoten**	**Kinder**	**Feld danach**
2	10	5, 1	$[4, 10, 3, 5, 1]$ *(ok)*
1	4	10, 3	tauscht mit 10
	4 (nun Pos. 2)	5, 1	tauscht mit 5
			$[10, 5, 3, 4, 1]$

Probe: $10 \geq 5$, $10 \geq 3$, $5 \geq 4$, $5 \geq 1$. **Gültiger Max-Heap.** ✓

Schritt 2 – Abbau.

Runde	**Tausch**	**Feld (Heap | sortiert)**
1	$10 \leftrightarrow 1$	$[5, 4, 3, 1 \mid 10]$
2	$5 \leftrightarrow 1$	$[4, 1, 3 \mid 5, 10]$
3	$4 \leftrightarrow 3$	$[3, 1 \mid 4, 5, 10]$
4	$3 \leftrightarrow 1$	$[1 \mid 3, 4, 5, 10]$
		$[1, 3, 4, 5, 10]$

Sortiert: $[1, 3, 4, 5, 10]$ ✓

Beachten Sie, dass kein Zusatzspeicher gebraucht wird. Das sortierte Teil wächst von rechts in dasselbe Feld hinein, während der Heap von links schrumpft. **Heapsort ist in situ – das ist sein Hauptvorzug.**

(b) k **sortierte Listen verschmelzen.**

Ansatz 1 – Paarweise verschmelzen, naiv.

Man verschmilzt Liste 1 mit 2, das Ergebnis mit 3, und so fort.

Bei k Listen zu je n/k Elementen wächst das Zwischenergebnis, und man liest es jedes Mal wieder durch:

$$\frac{2n}{k} + \frac{3n}{k} + \cdots + n = \frac{n}{k} \sum_{i=2}^{k} i = \Theta(nk).$$

Ansatz 2 – Turnierbaum, paarweise.

Man verschmilzt in $\log_2 k$ *Runden jeweils benachbarte Paare* – **wie bei Mergesort.**

$$T = \Theta(n \log k).$$

Ansatz 3 – Ein Heap der Größe k**.**

```
Lege den Kopf jeder Liste in einen
Min-Heap (Größe k);
while Heap nicht leer do {
  (wert, liste) := entnimm_min();
  gib wert aus;
  if liste hat noch Elemente then
einfügen(nächstes aus liste) }
```

Jedes der n Elemente wird genau einmal eingefügt und einmal entnommen, je $\mathcal{O}(\log k)$:

$$\boxed{T = \Theta(n \log k), \qquad \text{Speicher } \mathcal{O}(k)}$$

Verfahren	**Zeit**	**Speicher**	**extern?**
naiv	$\Theta(nk)$	$\mathcal{O}(n)$	nein
Turnierbaum	$\Theta(n \log k)$	$\mathcal{O}(n)$	nein
Heap	$\Theta(n \log k)$	$\mathcal{O}(k)$	**ja**

Die letzte Spalte entscheidet in der Praxis. Der Heap braucht nur k Elemente im Speicher – die Listen können auf der Platte liegen und Milliarden Einträge haben.

Das ist genau das externe Sortieren: *Man zerlegt eine riesige Datei in Blöcke, die in den Hauptspeicher passen, sortiert sie einzeln und verschmilzt sie dann mit einem Heap in einem einzigen Durchlauf.*

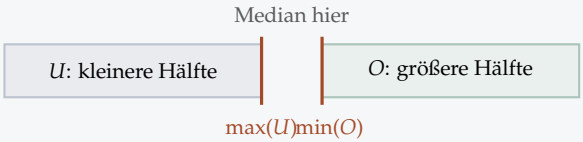
Dasselbe Muster steckt in der „Reduce"-Phase großer Datenverarbeitungssysteme und in jedem Datenbanksortierer.

(c) Der laufende Median.

Zahlen kommen nacheinander an. Nach jeder Zahl soll der Median aller bisherigen ausgegeben werden.

Der naive Weg: *jedes Mal neu sortieren* – $\mathcal{O}(n^2 \log n)$ insgesamt.
Die Idee: zwei Heaps, Rücken an Rücken.

Max-Heap U	enthält die **kleinere** Hälfte; seine Wurzel ist deren Maximum
Min-Heap O	enthält die **größere** Hälfte; seine Wurzel ist deren Minimum



Die Invarianten:

$$\max(U) \leq \min(O), \qquad \big||U| - |O|\big| \leq 1$$

Das Verfahren.

```
if U leer oder x <= wurzel(U) then
einfügen(U, x)
else einfügen(O, x);
if |U| > |O| + 1 then einfügen(O,
entnimm_max(U));
if |O| > |U| then einfügen(U,
entnimm_min(O));
```

Der Median ablesen:

$\lvert U\rvert > \lvert O\rvert$	Median = Wurzel(U)
$\lvert U\rvert = \lvert O\rvert$	Median = Mittel beider Wurzeln

Ein Durchlauf.
Strom: $5, 15, 1, 3, 8, 7, 9, 10, 6, 11, 4$.

x	U (Max-Heap)	O (Min-Heap)	**Median**
5	$\{5\}$	$\{\}$	5
15	$\{5\}$	$\{15\}$	10
1	$\{1,5\}$ wird $\{1\}$	$\{5,15\}$	5
3	$\{1,3\}$	$\{5,15\}$	4
8	$\{1,3,5\}$	$\{8,15\}$	5
7	$\{1,3,5\}$	$\{7,8,15\}$	6
9	$\{1,3,5,7\}$	$\{8,9,15\}$	7

Probe bei $x = 9$: Bisher $\{1,3,5,7,8,9,15\}$ – sieben Werte, der vierte ist 7. ✓

Der Aufwand.

$$\mathcal{O}(\log n) \text{ je Element}, \qquad \mathcal{O}(1) \text{ zum Ablesen}$$

Gesamt $\mathcal{O}(n \log n)$ *statt* $\mathcal{O}(n^2 \log n)$.

Die Konstruktion ist ein Musterbeispiel dafür, wie man eine Datenstruktur* aus *vorhandenen zusammensetzt. Beide Heaps sind Standardbausteine; die Leistung entsteht allein aus ihrer Anordnung.

Eine Einschränkung, die in der Praxis stört: *Man kann Elemente nicht ohne Weiteres wieder* ***entfernen*** **– ein Heap unterstützt kein Löschen an beliebiger Stelle.**

Für einen gleitenden Median über ein Zeitfenster („Median der letzten 1000 Messwerte“) braucht man deshalb entweder ***faules Löschen*** *– man merkt sich die zu entfernenden Elemente und wirft sie weg, wenn sie an die Wurzel kommen – oder zwei Suchbäume statt zweier Heaps.*

Der gleitende Median wird in der Signalverarbeitung als ***Medianfilter*** *eingesetzt* **– er entfernt Ausreißer, ohne Kanten zu verwischen, und ist deshalb in der Bildverarbeitung und bei EKG-Signalen die Standardvorverarbeitung.**

Cormen u. a.[6] behandeln Heaps und ihre Analyse vollständig; Tarjan[26] entwickelt die fortgeschrittenen Varianten; Skiena[25] bespricht ihre praktische Auswahl.

Part IV

Streuspeicherung

9 Hashing

Der Gedanke: rechnen statt suchen

Alle bisherigen Suchstrukturen ***vergleichen****. Und wer vergleicht, braucht* $\Omega(\log n)$ (Kapitel 3). **Hashing vergleicht nicht – es *rechnet* aus, wo das Element liegen muss.**

Definition 9.1. Eine **Hashfunktion** bildet den Schlüsselraum U auf $\{0, \ldots, m-1\}$ ab:

$$h : U \to \{0, \ldots, m-1\}.$$

Die **Hashtabelle** ist ein Feld dieser Größe; das Element mit Schlüssel k steht bei Index $h(k)$.

Operation	Aufwand	Bemerkung
Einfügen	$\mathcal{O}(1)$	*im Mittel*
Suchen	$\mathcal{O}(1)$	*im Mittel*
Löschen	$\mathcal{O}(1)$	*im Mittel*
schlechtester Fall	$\mathcal{O}(n)$	alle auf einem Platz
sortierte Ausgabe	unmöglich	**ohne Sortieren**
Nachbarn finden	unmöglich	keine Ordnung

Die letzten beiden Zeilen sind der Preis. Eine Hashtabelle weiß nicht, welches Element auf ein gegebenes folgt. **Wer Bereichsabfragen braucht, muss einen Baum nehmen** (Kapitel 7).

Das Grundproblem: Kollisionen.

Der Schlüsselraum ist riesig (2^{64} *mögliche Zahlen, alle Zeichenketten), die Tabelle klein.* **Nach dem Schubfachprinzip** (Band 0, Kapitel 1) **müssen verschiedene Schlüssel auf denselben Platz fallen.**

Die ganze Theorie des Hashings besteht aus zwei Fragen: **Wie wählt man h, damit Kollisionen selten sind? Und was tut man, wenn sie doch auftreten?**

Das Geburtstagsparadoxon – Kollisionen sind häufiger als man denkt

Wie viele Schlüssel darf man einfügen, bevor die erste Kollision wahrscheinlich wird?

*Bei m Plätzen und n Schlüsseln ist die Wahrscheinlichkeit, dass **alle** verschieden landen:*

$$\Pr[\text{keine Kollision}] = \prod_{i=0}^{n-1}\left(1 - \frac{i}{m}\right) \approx \mathrm{e}^{-n^2/(2m)}.$$

(Die Näherung benutzt $1 - x \approx \mathrm{e}^{-x}$ und $\sum_{i<n} i = \frac{n^2}{2}$.)
Das wird $\frac{1}{2}$ bei

$$\boxed{n \approx \sqrt{2 \ln 2 \cdot m} = 1{,}177\sqrt{m}}$$

Tabellengröße	erste Kollision	Beispiel
365	23	**Geburtstage**
10^6	1178	
2^{32}	77 000	32-Bit-Prüfsumme
2^{64}	$5{,}1 \cdot 10^9$	64-Bit-Hash
2^{128}	$2{,}2 \cdot 10^{19}$	MD5
2^{256}	$4 \cdot 10^{38}$	SHA-256

Die Wurzel ist das Erstaunliche. Ein 64-Bit-Hash klingt sicher – aber schon nach fünf Milliarden Werten ist eine Kollision wahrscheinlich. **Das ist der Grund, warum kryptographische Hashes 256 Bit haben** (Band 8).

Und für Hashtabellen heißt es: **Kollisionen sind der Normalfall, nicht die Ausnahme.** *Man muss sie einplanen, nicht vermeiden wollen.*

9.1 Hashfunktionen

Was eine gute Hashfunktion leistet

Gleichverteilung	Jeder Platz gleich wahrscheinlich
Unabhängigkeit	Ähnliche Schlüssel landen **nicht** nebeneinander
Schnelligkeit	Sie wird bei jedem Zugriff ausgewertet
Determinismus	Gleicher Schlüssel, gleicher Wert – immer

Die Divisionsmethode.

$$h(k) = k \bmod m.$$

Einfach und schnell. **Aber die Wahl von m ist heikel.**

> ***m* als Zweierpotenz ist eine schlechte Idee**
>
> **Ist $m = 2^p$, so ist $k \bmod m$ genau das Muster der untersten p Bit von k** (Band 0, Kapitel 2).
>
> *Alle höheren Bits werden* ***ignoriert*** **– und wenn die Schlüssel sich nur dort unterscheiden, kollidieren alle.**
>
> *Beispiel: Speicheradressen sind fast immer durch 8 oder 16 teilbar. Bei $m = 1024$ landen sie alle auf Plätzen, die durch 8 teilbar sind* **– sieben Achtel der Tabelle bleiben leer.**
>
> > ***Die Empfehlung:*** *m als Primzahl wählen, möglichst weit von einer Zweierpotenz entfernt.* **Dann tragen alle Bits des Schlüssels zum Ergebnis bei.**
>
> *Wer trotzdem Zweierpotenzen will – und das will man, weil dann* mod *zu einem UND wird –, muss die Bits vorher* ***durchmischen****.* **Genau das tut die Multiplikationsmethode.**

Die Multiplikationsmethode.

$$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor, \qquad 0 < A < 1.$$

Man multipliziert, nimmt den Nachkommateil und skaliert.

KNUTH *empfiehlt* $A = \frac{\sqrt{5}-1}{2} = 0{,}6180339887$ **– den goldenen Schnitt.**

Der Grund liegt im ***Dreiabstandssatz****: Die Folge $\{kA\}$ verteilt die Punkte für irrationales A besonders gleichmäßig, und der goldene Schnitt ist in einem präzisen Sinn die „irrationalste" aller Zahlen* (Mathematik-Band 1, Kettenbrüche).

Hier ist m als Zweierpotenz sogar günstig **– man rechnet mit ganzen Zahlen:**
$h(k) = (k \cdot A') \gg (w - p)$**, wobei** $A' = A \cdot 2^w$**.**

Für Zeichenketten: die Polynommethode.

$$h(s) = \Big(\sum_{i=0}^{L-1} s_i \cdot b^{L-1-i}\Big) \bmod m.$$

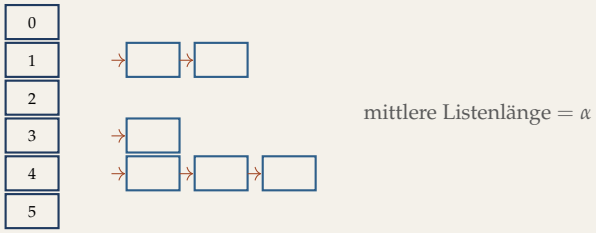
Man liest die Zeichenkette als Zahl zur Basis b. **Java benutzt $b = 31$, viele C-Bibliotheken $b = 33$ oder $b = 131$.**

Warum ungerade Primzahlen? Weil sonst Information verlorengeht: Bei geradem b wandern die unteren Bits nach oben hinaus. **Und 31 ist bequem, weil** $31k = (k \ll 5) - k$ *– eine Verschiebung und eine Subtraktion statt einer Multiplikation.*

9.2 Kollisionsbehandlung

Zwei Strategien

Strategie 1 – Verkettung.
Jeder Tabellenplatz enthält eine Liste aller Elemente, die dorthin gehashen.



Definition 9.2. Der **Belegungsfaktor** ist $\alpha = \frac{n}{m}$ – die mittlere Zahl der Elemente je Platz.

Satz 9.3. *Bei gleichmäßigem Hashing beträgt die erwartete Suchdauer*

$$\Theta(1 + \alpha).$$

Hält man α konstant (etwa $\alpha \leq 1$), so ist die Suche $\mathcal{O}(1)$. **Deshalb vergrößert man die Tabelle, sobald α eine Schwelle übersteigt** – *amortisiert kostet das nichts* (Kapitel 1).

Strategie 2 – Offene Adressierung.
Kein Zusatzspeicher: Ist der Platz belegt, sucht man nach einer festen Regel weiter.

Sondierung	Formel	Problem
linear	$h(k) + i$	**Klumpenbildung**
quadratisch	$h(k) + c_1 i + c_2 i^2$	sekundäre Klumpen
doppelt	$h_1(k) + i \cdot h_2(k)$	beste Streuung

Die primäre Klumpenbildung bei linearer Sondierung ist ein selbstverstärkender Effekt: Ein belegter Block wird mit höherer Wahrscheinlichkeit getroffen als ein einzelner Platz – **also wächst er schneller, und dadurch wird er noch wahrscheinlicher getroffen.**

Satz 9.4 (Knuth, 1963). *Bei linearer Sondierung beträgt die erwartete Zahl der Sondierungen bei erfolgloser Suche etwa*

$$\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right).$$

α	**linear**	**doppelt**
0,50	2,5	2,0
0,75	8,5	4,0
0,90	50,5	10,0
0,95	200,5	20,0

Ab $\alpha = 0{,}9$ wird lineare Sondierung katastrophal. **Die Faustregel: $\alpha \leq 0{,}7$ halten.**

Und trotzdem gewinnt lineare Sondierung in der Praxis

Die Tabelle oben zählt Sondierungen. Sie zählt nicht Cachefehler.

Lineare Sondierung läuft ***sequentiell*** *durch benachbarte Speicherzellen* **– alle Sondierungen eines Klumpens liegen in ein oder zwei Cachezeilen.** *Doppeltes Hashing springt bei jeder Sondierung an eine zufällige Adresse.*

Bei $\alpha \leq 0{,}7$ ist lineare Sondierung deshalb regelmäßig die schnellste Variante – **obwohl sie theoretisch die schlechteste ist.**

Sie wird in Googles `dense_hash_map`*, in Rusts* `HashMap` *und in vielen modernen Implementierungen benutzt* – **oft in der Verfeinerung Robin-Hood-Hashing, bei der Elemente mit langer Sondierstrecke die mit kurzer verdrängen und so die Streuung ausgleichen.**

Löschen bei offener Adressierung ist heikel

Man darf einen Platz nicht einfach leeren.
Denn die Suche nach einem später eingefügten Element bricht ab, sobald sie auf einen leeren Platz trifft – **und das Element wäre unauffindbar, obwohl es da ist.**

Man setzt stattdessen eine ***Grabsteinmarkierung****: „hier war etwas“.* **Die Suche läuft darüber hinweg, das Einfügen darf den Platz benutzen.**

Der Nachteil: Grabsteine sammeln sich an und verlängern die Suchwege, ohne dass α steigt. **Nach vielen Löschungen muss man die Tabelle neu aufbauen.**

Aufgabe 9.1.

(a) Führen Sie das Einfügen mit beiden Kollisionsstrategien vor und vergleichen Sie.
(b) Erklären Sie den Hash-Flooding-Angriff von 2011 und seine Abwehr.
(c) Entwickeln Sie den Bloom-Filter und bestimmen Sie die optimale Parameterwahl.

Lösung Schritt für Schritt.

(a) Zwei Strategien am selben Beispiel.
Tabellengröße $m = 7$, $h(k) = k \bmod 7$. *Einzufügen:* $50, 700, 76, 85, 92, 73, 101$.
Schritt 1 – Die Hashwerte.

k	50	700	76	85	92	73	101
$h(k)$	1	0	6	1	1	3	3

Drei Schlüssel auf Platz 1, zwei auf Platz 3. **Fünf Kollisionen bei sieben Elementen.**
Schritt 2 – Verkettung.

Platz	Liste	Länge
0	700	1
1	$50 \to 85 \to 92$	**3**
2	—	0
3	$73 \to 101$	2
4, 5	—	0
6	76	1

$$\alpha = \frac{7}{7} = 1{,}0, \qquad \text{erwartete Suchdauer} = 1 + \alpha = 2.$$

Tatsächliche mittlere erfolgreiche Suchdauer: $\frac{1+1+2+3+1+2+1}{7} = \frac{11}{7} = 1{,}57$. ✓

Schritt 3 – Lineare Sondierung.

k	$h(k)$	Sondierungen	Endplatz
50	1	1	1
700	0	1	0
76	6	1	6
85	1	1, 2	**2**
92	1	1, 2, 3	**3**
73	3	3, 4	**4**
101	3	3, 4, 5	**5**

0	1	2	3	4	5	6
700	50	85	92	73	101	76

Die Tabelle ist vollständig belegt, und man sieht die Klumpenbildung: Ab Platz 0 ist alles zusammenhängend besetzt. **Die Einfügung von 101 brauchte drei Sondierungen, obwohl sein Platz 3 nur *einmal* direkt getroffen wurde.**

Mittlere Sondierungszahl: $\frac{1+1+1+2+3+2+3}{7} = \frac{13}{7} = 1{,}86$.

Schritt 4 – Doppeltes Hashing.

Mit $h_2(k) = 5 - (k \bmod 5)$, *was Werte in* $\{1, \ldots, 5\}$ *liefert:*

k	h_1	h_2	**Sondierungsfolge**	**Platz**
85	1	$5 - 0 = 5$	1, 6	**6**
92	1	$5 - 2 = 3$	1, 4	**4**

*Die kollidierenden Schlüssel gehen in **verschiedene** Richtungen* – **keine Klumpen.** ✓

> **h_2 darf nie 0 werden und muss teilerfremd zu m sein**
>
> **Ist $h_2(k) = 0$, so springt die Sondierung nicht weiter** – *Endlosschleife.*
> *Und hat $h_2(k)$ mit m einen gemeinsamen Teiler $d > 1$, so werden nur m/d Plätze erreicht* – **die Tabelle ist nur teilweise nutzbar.**
>
> *Die übliche Lösung: m prim wählen, dann ist jedes $h_2 \in \{1, \ldots, m-1\}$ teilerfremd.*
> **Oder $m = 2^p$ und h_2 stets ungerade.**

(b) Der Hash-Flooding-Angriff.

> *Am 28. Dezember 2011 hielten* Alexander Klink *und* Julian Wälde *auf dem Chaos Communication Congress in Berlin einen Vortrag mit dem Titel „Effiziente Denial-of-Service-Angriffe auf Webanwendungsplattformen“.*

Schritt 1 – Die Beobachtung.
Jeder Webserver legt die Parameter einer Anfrage (`name=wert&...`) in einer Hashtabelle ab. **Die Hashfunktionen waren fest und öffentlich bekannt.**
Schritt 2 – Der Angriff.

> *Wer die Funktion kennt, kann Schlüssel konstruieren, die **alle denselben Hashwert** haben.* **Dann degeneriert die Tabelle zu einer einzigen Liste.**

Fall	**Aufwand**	**bei $n = 10^5$**
normal	$\mathcal{O}(n)$	Millisekunden
Angriff	$\mathcal{O}(n^2)$	**Minuten**

Für PHP genügte eine einzige POST-Anfrage von 500 Kilobyte, um einen Prozessorkern rund eine Minute zu beschäftigen.

> ***Mit einer gewöhnlichen DSL-Leitung konnte man Dutzende Server lahmlegen.*** Betroffen waren PHP, Python, Ruby, Java, ASP.NET, Node.js und V8 – **also praktisch das gesamte Web.**

Schritt 3 – Wie man solche Schlüssel findet.
Bei der Polynommethode mit $b = 33$ und Rechnung modulo 2^{32} lässt sich eine Kollision direkt konstruieren: Die Zeichenpaare `Ez` *und* `FY` *haben denselben Beitrag.*

$$69 \cdot 33 + 122 = 2399, \qquad 70 \cdot 33 + 89 = 2399. \qquad \checkmark$$

Damit erzeugt jede Kombination dieser Paare denselben Hashwert – **aus 16 Paaren erhält man $2^{16} = 65\,536$ kollidierende Schlüssel, ohne jede Suche.**

Schritt 4 – Die Abwehr.

Sofortmaßnahme	Zahl der Parameter je Anfrage begrenzen
Bäume statt Listen	Java 8: lange Ketten werden zu Rot-Schwarz-Bäumen – $\mathcal{O}(n \log n)$ statt $\mathcal{O}(n^2)$
Die richtige Lösung	**randomisiertes Hashing**

*Beim **universellen Hashing** wird die Hashfunktion beim Programmstart **zufällig** aus einer Familie gewählt.* **Der Angreifer kennt sie nicht und kann keine Kollisionen konstruieren.**

Definition 9.5. Eine Familie $\mathcal{H}$ heißt **universell**, wenn für alle $k \neq l$

$$\Pr_{h \in \mathcal{H}}[h(k) = h(l)] \leq \frac{1}{m}.$$

Das ist genau so gut, als würde man die Plätze auswürfeln – **und es gilt *unabhängig* davon, welche Schlüssel der Angreifer wählt.**

*Heute benutzen Python, Ruby, Rust und Perl standardmäßig **SipHash** mit einem beim Start gewürfelten Schlüssel* (Aumasson und Bernstein, 2012) – **eine Funktion, die eigens als Antwort auf diesen Angriff entworfen wurde.**

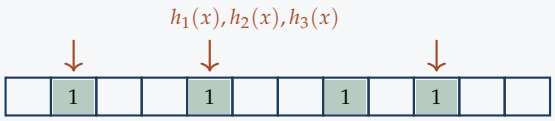
In Python kann man das an `hash("abc")` *beobachten: Der Wert ist bei jedem Programmstart ein anderer.*

(c) Der Bloom-Filter.

Burton Bloom *stellte 1970 eine Frage: Was, wenn man gar nicht die Elemente speichern muss, sondern nur wissen will, **ob** etwas dazugehört – und wenn man dabei gelegentliche Fehler hinnimmt?*

Schritt 1 – Die Struktur.

1. Ein Bitfeld der Länge m, anfangs alles null.
2. k unabhängige Hashfunktionen $h_1, \ldots, h_k$.
3. **Einfügen von x:** setze die Bits $h_1(x), \ldots, h_k(x)$ auf 1.
4. **Anfrage nach x:** sind **alle** k Bits gesetzt? Wenn nein, ist x sicher nicht enthalten.



Schritt 2 – Die Fehlerarten.

Antwort	Sicherheit	Bedeutung
„nicht enthalten“	**immer richtig**	kein Bit gesetzt $\Longrightarrow$ nie eingefügt
„enthalten“	kann falsch sein	andere Elemente haben die Bits gesetzt

Der Fehler geht nur in eine Richtung. Es gibt falsch-positive, aber keine falsch-negativen Antworten – **und genau das macht die Struktur brauchbar.**

Schritt 3 – Die Fehlerrate berechnen.
*Nach dem Einfügen von n Elementen ist die Wahrscheinlichkeit, dass ein bestimmtes Bit **nicht** gesetzt wurde:*

$$\left(1 - \frac{1}{m}\right)^{kn} \approx \mathrm{e}^{-kn/m}.$$

Also ist ein Bit mit Wahrscheinlichkeit $1 - \mathrm{e}^{-kn/m}$ gesetzt, und alle k Bits eines nicht enthaltenen Elements:

$$\boxed{p \approx \left(1 - \mathrm{e}^{-kn/m}\right)^k}$$

Schritt 4 – k optimieren.
Man leitet $\ln p$ nach k ab und setzt null. Das Ergebnis:

$$k_{\text{opt}} = \frac{m}{n} \ln 2 = 0{,}693\,\frac{m}{n}.$$

Eingesetzt:

$$p_{\min} = 2^{-k_{\text{opt}}} = (0{,}6185)^{m/n}.$$

Bei optimalem k ist genau die Hälfte aller Bits gesetzt. Das ist einleuchtend: Ein Bitfeld trägt die meiste Information, wenn Nullen und Einsen gleich häufig sind (Band 0, Kapitel 1).

Schritt 5 – Zahlen.

Bits je Elem.	k_{opt}	Fehlerrate	Bemerkung
4	3	14,7 %	grob
8	6	2,1 %	
10	7	0,82 %	**übliche Wahl**
16	11	0,046 %	
32	22	$2{,}5 \cdot 10^{-7}$	

Zehn Bit je Element für unter ein Prozent Fehler. Zum Vergleich: Eine Hashtabelle mit 64-Bit-Schlüsseln bräuchte über hundert Bit je Element. **Der Bloom-Filter ist etwa zehnmal sparsamer.**

Schritt 6 – Wo man ihn einsetzt.

Datenbanken	Bevor man auf die Platte geht: „Ist der Schlüssel überhaupt in diesem Block?“
Webbrowser	Prüfung gegen Listen schädlicher Adressen
Netzwerke	Weiterleitungsentscheidungen im Router
Rechtschreibung	Frühe Prüfprogramme speicherten so ihr Wörterbuch
Bioinformatik	Vorfilterung beim Sequenzabgleich

Das Einsatzmuster ist immer dasselbe: *Der Filter beantwortet die häufige Frage „sicher nicht da“ in Nanosekunden, und nur bei „vielleicht da“ greift man auf die teure Struktur zu.* **Ein Prozent Fehlalarm bedeutet ein Prozent überflüssige Plattenzugriffe** – *und neunundneunzig Prozent gesparte.*

Die Einschränkung: *Ein klassischer Bloom-Filter kann nicht* ***löschen*** **– man dürfte die Bits nicht zurücksetzen, weil sie zu anderen Elementen gehören könnten.**

*Abhilfe schaffen **Counting Bloom Filter** (Zähler statt Bits) oder **Cuckoo-Filter** (2014), die zusätzlich löschen können und bei niedrigen Fehlerraten sogar sparsamer sind.* Blooms Originalarbeit[4] ist vier Seiten lang; Mitzenmacher und andere haben die Varianten seither systematisch untersucht; Cormen u. a.[6] und Mehlhorn und Sanders[19] behandeln das universelle Hashing vollständig.

Part V

Graphen

10 Graphen darstellen und durchmustern

Warum Graphen die allgemeinste Datenstruktur sind

Ein Graph ist eine Menge von Objekten und eine Menge von Beziehungen zwischen ihnen. **Damit lässt sich fast alles modellieren** – *und Listen und Bäume sind bloß Sonderfälle.*

Anwendung	**Ecken**	**Kanten**
Straßennetz	Kreuzungen	Straßen
Internet	Rechner	Verbindungen
Soziales Netz	Personen	Bekanntschaften
Konnektom	**Hirnareale**	**Faserbahnen**
Übersetzerbau	Variablen	Konflikte (Färbung)
Ablaufplanung	Aufgaben	Abhängigkeiten
Zustandsraum	Konfigurationen	Züge

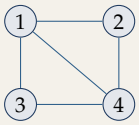
Die vierte Zeile verdient eine Bemerkung: ***Die vergleichende Konnektomik behandelt das Gehirn als Graphen*** – Areale als Ecken, nachgewiesene Faserverbindungen als Kanten. **Die Fragen, die man dann stellt, sind exakt die dieses und der beiden folgenden Kapitel:** *Wie lang sind die Wege? Welche Ecken sind zentral? Zerfällt der Graph in Module?*

Definition 10.1. Ein **Graph** $G = (V, E)$ besteht aus einer Eckenmenge V und einer Kantenmenge E. Man schreibt $n = |V|$ und $m = |E|$.

ungerichtet	Kanten sind Paare $\{u, v\}$
gerichtet	Kanten sind geordnete Paare (u, v)
gewichtet	jede Kante trägt eine Zahl
dünn	$m = \mathcal{O}(n)$
dicht	$m = \Theta(n^2)$

10.1 Zwei Darstellungen

Matrix oder Listen – und wann es umkippt



Adjazenzmatrix

0	1	1	1
1	0	0	1
1	0	0	1
1	1	1	0

Adjazenzlisten

1 : 2, 3, 4
2 : 1, 4
3 : 1, 4
4 : 1, 2, 3

Operation	**Matrix**	**Listen**
Speicher	$\Theta(n^2)$	$\Theta(n+m)$
Kante $\{u,v\}$ prüfen	$\mathcal{O}(1)$	$\mathcal{O}(\deg u)$
Alle Nachbarn von u	$\Theta(n)$	$\Theta(\deg u)$
Kante einfügen	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Durchmusterung	$\Theta(n^2)$	$\Theta(n+m)$

Die dritte und die fünfte Zeile entscheiden fast immer. Die meisten Graphenalgorithmen fragen nicht „gibt es diese Kante“, sondern „wer sind die Nachbarn“ – **und dafür ist die Matrix hoffnungslos.**

Die Zahlen für ein reales Beispiel.

Graph	n	m	**Matrix / Listen**
Konnektom (Makak)	95	2400	9025 / 4990
Straßennetz D	10^6	$2{,}5 \cdot 10^6$	10^{12} / $6 \cdot 10^6$
Soziales Netz	10^9	10^{11}	10^{18} / $2 \cdot 10^{11}$

Bei kleinen dichten Graphen – der ersten Zeile – ist die Matrix konkurrenzfähig und einfacher zu programmieren. Genau das ist der Fall bei Konnektivitätsmatrizen der Hirnforschung: rund hundert Areale, dicht verbunden. **Ab etwa zehntausend Ecken ist sie unbenutzbar.**

Die Matrix hat einen Vorzug, den man leicht übersieht

Man kann mit ihr *rechnen*.

Ist A die Adjazenzmatrix, so gibt der Eintrag $(A^k)_{ij}$ die Zahl der Wege der Länge k von i nach j an (Mathematik-Band 15, Kapitel 10).

*Und die **Eigenwerte** von A sagen etwas über die Struktur des Graphen aus* – **Zusammenhang, Bipartitheit, Expansionseigenschaften.** *Das ist das Gebiet der **Spektralen Graphentheorie**.*

In der Netzwerkanalyse – auch der von Konnektomen – benutzt man das ständig: *Die **Eigenvektorzentralität** einer Ecke ist ihr Eintrag im Eigenvektor zum größten Eigenwert*, **und der PageRank-Algorithmus ist eine Abwandlung davon** (Band 11).

10.2 Die beiden Durchmusterungen

Ein Algorithmus, zwei Datenstrukturen

```
durchmustere(s):
  lege s in den Behälter; markiere s;
  while Behälter nicht leer do {
    u := entnimm aus Behälter;
    bearbeite u;
    for jeden Nachbarn v von u do
      if v nicht markiert then {
markiere v; lege v hinein } }
```

Der Behälter entscheidet über alles.

Behälter	Verfahren	Reihenfolge
Warteschlange	**Breitensuche** (BFS)	ebenenweise
Stapel	**Tiefensuche** (DFS)	so weit wie möglich
Prioritätsschlange	**Dijkstra**	nach Entfernung (Kap. 11)

Dasselbe Muster wie bei der Baumdurchmusterung (Kapitel 6) – **und das ist kein Zufall, denn ein Baum ist ein Graph.**

Der Aufwand.

$$\boxed{\Theta(n+m) \text{ bei Adjazenzlisten}}$$

Jede Ecke wird einmal entnommen, jede Kante einmal (gerichtet) oder zweimal (ungerichtet) betrachtet. ✓

> ***Das ist optimal*** – man kann einen Graphen nicht durchmustern, ohne ihn anzusehen. **Und es ist *linear*, was bei Graphen bedeutet: linear in der Eingabegröße, nicht in der Eckenzahl.**

Was die beiden Verfahren jeweils leisten

Aufgabe	BFS	DFS
Kürzeste Wege (ungewichtet)	**ja**	nein
Zusammenhangskomponenten	ja	ja
Zyklus finden	ja	ja
Topologische Sortierung	mit Grad	**natürlich**
Starke Komponenten	nein	**ja**
Brücken, Artikulationspunkte	nein	**ja**
Speicherbedarf	$\mathcal{O}(\text{Breite})$	$\mathcal{O}(\text{Tiefe})$

Satz 10.2 (BFS findet kürzeste Wege)**.** *In einem ungewichteten Graphen liefert BFS von s aus für jede Ecke v die kleinste Kantenzahl eines Weges von s nach v.*

Der Beweis über die Invariante.

(I) Die Warteschlange enthält stets nur Ecken

vom Abstand d und $d+1$, in dieser Reihenfolge.

Anfangs enthält sie nur s mit Abstand 0. ✓
Wird eine Ecke vom Abstand d entnommen, so haben ihre unmarkierten Nachbarn den Abstand $d+1$ – kleiner können sie nicht sein, denn sonst wären sie schon markiert. ✓

Und sie werden hinten angefügt, also nach allen anderen mit Abstand $d+1$. **Die Reihenfolge bleibt erhalten.** ✓

Warum DFS keine kürzesten Wege liefert

Die Tiefensuche verfolgt einen Weg, so weit sie kann – und erreicht eine Ecke möglicherweise auf einem langen Umweg, bevor sie den kurzen Weg findet.
Da die Ecke dann schon markiert ist, wird der kurze Weg nie betrachtet.

> *Beispiel: Ein Dreieck $s \to a \to b$ und zusätzlich $s \to b$.* **Verfolgt DFS zuerst $s \to a$, so erreicht sie b mit Abstand 2, obwohl 1 genügt hätte.**

Der Fehler ist verbreitet und tückisch, *weil DFS bei vielen Testfällen zufällig das Richtige liefert.*

Die Tiefensuche als Analysewerkzeug

> ROBERT TARJAN *zeigte 1972, dass die Tiefensuche weit mehr ist als eine Besuchsreihenfolge:* **Sie legt eine Struktur über den Graphen, aus der sich zahlreiche Eigenschaften ablesen lassen.**

Man vergibt zwei Zeitstempel je Ecke.

ein$[v]$	wann v zum ersten Mal betreten wird
aus$[v]$	wann v endgültig verlassen wird

Damit klassifiziert man alle Kanten.

Kantenart	Bedingung	Bedeutung
Baumkante	v neu entdeckt	Teil des DFS-Baums
Rückwärtskante	v ist Vorfahre	**Zyklus!**
Vorwärtskante	v ist Nachfahre	Abkürzung
Querkante	sonst	andere Teilbäume

Satz 10.3. *Ein gerichteter Graph enthält genau dann einen Zyklus, wenn die Tiefensuche eine Rückwärtskante findet.*

Denn eine Rückwärtskante (u, v) führt zu einem Vorfahren v – und der Weg von v zu u im DFS-Baum schließt sich damit zum Zyklus. ✓

> ***Der Klammersatz:*** *Für zwei Ecken u, v sind die Intervalle $[ein[u], aus[u]]$ und $[ein[v], aus[v]]$ entweder disjunkt oder ineinander geschachtelt* – **nie teilweise überlappend.**
>
> *Das ist die Struktur eines wohlgeformten Klammerausdrucks* (Kapitel 2) – **und der Grund, warum die Stapelstruktur der Rekursion die Baumstruktur des Graphen abbildet.**
>
> *Aus diesem einen Satz folgen Tarjans Verfahren für starke Zusammenhangskomponenten, Brücken, Artikulationspunkte und zweifachen Zusammenhang* – **alle in $\Theta(n+m)$.**

Aufgabe 10.1.

(a) Führen Sie BFS und DFS an einem Graphen vollständig vor und lesen Sie die Abstände ab.

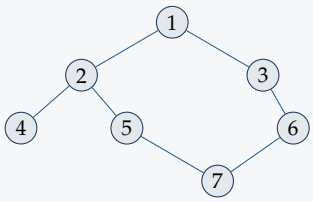
(b) Bestimmen Sie eine topologische Sortierung und erkennen Sie Zyklen.

(c) Berechnen Sie Kennzahlen eines kleinen Konnektomgraphen: Weglängen, Zentralität, Clusterkoeffizient.

Lösung Schritt für Schritt.

(a) Beide Durchmusterungen.
Graph mit $V = \{1, \ldots, 7\}$ und den Kanten

$$\{1,2\}, \{1,3\}, \{2,4\}, \{2,5\}, \{3,6\}, \{5,7\}, \{6,7\}.$$



Breitensuche von 1.

Schritt	entnommen	hinzugefügt	Schlange danach
1	1	2, 3	[2, 3]
2	2	4, 5	[3, 4, 5]
3	3	6	[4, 5, 6]
4	4	—	[5, 6]
5	5	7	[6, 7]
6	6	—	[7]
7	7	—	[ ]

Ecke	1	2	3	4	5	6	7
Abstand	0	1	1	2	2	2	3

Probe für Ecke 7: Weg $1 \to 2 \to 5 \to 7$*, drei Kanten.* **Der alternative Weg** $1 \to 3 \to 6 \to 7$ **ist ebenso lang.** ✓
Tiefensuche von 1 (Nachbarn aufsteigend):

Ecke	ein	aus	Weg dorthin
1	1	14	—
2	2	11	1→2
4	3	4	1→2→4
5	5	10	1→2→5
7	6	9	1→2→5→7
6	7	8	1→2→5→7→6
3	12	13	1→3

DFS erreicht Ecke 6 über einen Weg der Länge 4 – BFS hatte Abstand 2 gefunden. **Das ist der Beleg für die Warnung oben.**

Probe des Klammersatzes: Das Intervall von 6 ist $[7, 8]$*, das von 5 ist* $[5, 10]$ – **sauber geschachtelt.** ✓

(b) Topologische Sortierung.

Definition 10.4. Eine **topologische Sortierung** eines gerichteten Graphen ist eine Reihenfolge der Ecken, in der jede Kante von früher nach später zeigt.

*Sie existiert genau dann, wenn der Graph **zyklenfrei** ist.*

Beispiel – Abhängigkeiten beim Übersetzen:

`main.o`	braucht `main.c`, `util.h`
`util.o`	braucht `util.c`, `util.h`
`programm`	braucht `main.o`, `util.o`

Verfahren 1 – über die Eingangsgrade (Kahn, 1962).

```
Berechne für jede Ecke den
Eingangsgrad;
lege alle Ecken mit Grad 0 in eine
Schlange;
while Schlange nicht leer do {
  u := entnimm; gib u aus;
  for jede Kante (u,v) do {
    grad[v] := grad[v] - 1;
    if grad[v] = 0 then lege v hinein }
}
```

Die Zyklenerkennung fällt ab.

*Wurden am Ende **weniger als** n Ecken ausgegeben, so enthält der Rest einen Zyklus.* **Denn in einem Zyklus hat keine Ecke jemals Eingangsgrad 0.** ✓

Verfahren 2 – über DFS.

*Man führt DFS aus und gibt jede Ecke aus, wenn man sie **verlässt**. Die **umgekehrte** Reihenfolge ist eine topologische Sortierung.*

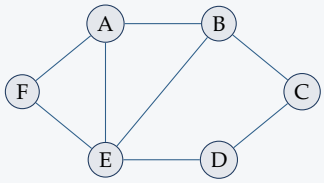
Warum: Eine Ecke wird erst verlassen, wenn alle von ihr erreichbaren Ecken fertig sind – **sie hat also die größte Ausgangszeit unter ihren Nachfolgern.** ✓
Ein Durchlauf am Beispiel.

Grad-0-Ecken	ausgegeben	Grade danach
`main.c`, `util.c`, `util.h`	`main.c`	`main.o`: 1
	`util.c`	`util.o`: 1
	`util.h`	`main.o`: 0, `util.o`: 0
`main.o`, `util.o`	`main.o`	`programm`: 1
	`util.o`	`programm`: 0
`programm`	`programm`	—

Das ist genau, was `make` tut – und was jedes Paketverwaltungssystem, jeder Übersetzer bei der Auflösung von Abhängigkeiten und jeder Tabellenkalkulationsneuberechnung tut.

Und die Zyklenmeldung kennt jeder: „circular dependency detected" – das ist der Fall, in dem weniger als n Ecken ausgegeben wurden.

(c) Kennzahlen eines Konnektomgraphen.
Wir nehmen einen kleinen ungerichteten Graphen mit sechs Arealen:



Kanten: $AB, AE, AF, BC, BE, CD, DE, EF$ *– also* $n = 6$, $m = 8$.
Schritt 1 – Die Grade.

Ecke	A	B	C	D	E	F
Grad	3	3	2	2	4	2

Probe mit dem Handschlaglemma (Mathematik-Band 15, Kapitel 1): $3+3+2+2+4+2 = 16 = 2 \cdot 8$. ✓
Schritt 2 – Alle kürzesten Wege durch BFS von jeder Ecke.

	A	B	C	D	E	F
A	0	1	2	2	1	1
B	1	0	1	2	1	2
C	2	1	0	1	2	3
D	2	2	1	0	1	2
E	1	1	2	1	0	1
F	1	2	3	2	1	0

Sechs BFS-Läufe zu je $\mathcal{O}(n+m)$*:* $\mathcal{O}(n(n+m)) = \mathcal{O}(6 \cdot 14) = 84$ Schritte.

Schritt 3 – Die charakteristische Weglänge.

$$L = \frac{1}{n(n-1)} \sum_{i \neq j} d(i,j)$$

$$= \frac{2 \cdot (1+2+2+1+1+1+2+1+2+1+3+2+1+2+3)}{30}.$$

Summe der oberen Dreiecksmatrix: $1{+}2{+}2{+}1{+}1 \; + \; 1{+}2{+}1{+}2 \; + \; 1{+}2{+}3 \; + \; 1{+}2 \; + \; 1 = 23$.

$$L = \frac{2 \cdot 23}{30} = 1{,}533.$$

Schritt 4 – Der Durchmesser.

$$D = \max_{i,j} d(i,j) = 3 \qquad (\text{zwischen } C \text{ und } F).$$

Schritt 5 – Die Nähezentralität.

$$C_N(v) = \frac{n-1}{\sum_u d(v,u)}.$$

Ecke	**Distanzsumme**	C_N	**Rang**
A	7	0,714	2
B	7	0,714	2
C	9	0,556	5
D	8	0,625	4
E	**6**	**0,833**	**1**
F	9	0,556	5

Ecke E ist die zentralste – sie hat den höchsten Grad *und* die kürzesten Wege zu allen anderen. **In der Konnektomik hieße sie ein Hub.**

Schritt 6 – Der Clusterkoeffizient.

Definition 10.5.

$$C(v) = \frac{\text{Zahl der Kanten unter den Nachbarn von } v}{\binom{\deg v}{2}}.$$

Er misst, wie stark die Nachbarn einer Ecke selbst untereinander verbunden sind – **„Sind die Freunde meiner Freunde auch befreundet?“**

Ecke	**Nachbarn**	**Kanten darunter**	$\binom{\deg}{2}$	$C(v)$
A	B, E, F	BE, EF	3	0,667
B	A, C, E	AE	3	0,333
C	B, D	—	1	0
D	C, E	—	1	0
E	A, B, D, F	AB, AF	6	0,333
F	A, E	AE	1	1,000

$$\bar{C} = \frac{0{,}667 + 0{,}333 + 0 + 0 + 0{,}333 + 1{,}000}{6} = 0{,}389.$$

Schritt 7 – Die Deutung.

Maß	**Wert**	**Vergleich**
Weglänge L	1,53	Zufallsgraph: $\approx 1{,}6$
Clusterkoeff. $\bar{C}$	0,39	Zufallsgraph: $\approx 0{,}27$

Kurze Wege bei gleichzeitig hoher Clusterung – das ist die Signatur eines Kleine-Welt-Netzwerks (WATTS und STROGATZ, 1998). **Reale Konnektome zeigen diese Signatur sehr ausgeprägt**, *und man hält sie für einen Kompromiss zwischen Verdrahtungsaufwand und Integrationsfähigkeit.*

Der Aufwand aller Kennzahlen:

Kennzahl	**Aufwand**	**bei $n = 100$**
Grade	$\mathcal{O}(n+m)$	sofort
Alle kürzesten Wege	$\mathcal{O}(n(n+m))$	$\approx 250\,000$
Clusterkoeffizient	$\mathcal{O}(\sum \deg^2)$	$\approx 10^5$

> ***Alle diese Kennzahlen sind für hundert Areale eine Sache von Millisekunden*** – vorausgesetzt, man benutzt BFS von jeder Ecke. **Wer stattdessen für jedes Eckenpaar einzeln einen Weg sucht und dabei jedes Mal von vorn beginnt, landet bei** $\mathcal{O}(n^2(n+m))$ – *und das ist der Unterschied zwischen Sekunden und Stunden.*

Diestel und West liefern die Graphentheorie (Mathematik-Band 15, Kapitel 10–12); Cormen u. a.[6] und Sedgewick und Wayne[24] entwickeln die Algorithmen; Skiena[25] bespricht die Wahl der Darstellung praxisnah.

11 Kürzeste Wege

Vier Fragestellungen, vier Verfahren

*Ein Graph, dessen Kanten **Gewichte** tragen – Entfernungen, Fahrzeiten, Kosten, Leitungsverzögerungen. Gesucht ist der Weg mit der kleinsten Gewichtssumme.*

Frage	Verfahren	Aufwand
alle Gewichte gleich	**Breitensuche**	$\mathcal{O}(n+m)$
eine Quelle, $w \geq 0$	**Dijkstra**	$\mathcal{O}(m + n \log n)$
eine Quelle, w beliebig	**Bellman-Ford**	$\mathcal{O}(n\,m)$
alle Paare	**Floyd-Warshall**	$\mathcal{O}(n^3)$
eine Quelle, ein Ziel, mit Schätzer	**A***	wie Dijkstra, meist viel weniger

Die zweite Zeile ist der Normalfall, und die dritte Zeile ist der Preis dafür, negative Gewichte zuzulassen – **ein Faktor** n**.**

Die Grundlage aller Verfahren: die Optimalitätseigenschaft

Satz 11.1. *Ist $v_0, v_1, \ldots, v_k$ ein kürzester Weg von v_0 nach v_k, so ist jeder Teilweg $v_i, \ldots, v_j$ ein kürzester Weg von v_i nach v_j.*

Beweis durch Austausch. Gäbe es einen kürzeren Weg von v_i nach v_j, so könnte man ihn einsetzen und erhielte einen kürzeren Gesamtweg. **Widerspruch.** ✓

> ***Das ist die Voraussetzung dafür, dass man überhaupt schrittweise vorgehen kann.*** Man darf Teillösungen speichern und wiederverwenden, weil sie in der Gesamtlösung unverändert vorkommen.

Man nennt diese Eigenschaft das ***Optimalitätsprinzip*** *nach* RICHARD BELLMAN – **sie ist zugleich die Grundlage der dynamischen Programmierung** (Kapitel 14).

11.1 Der Algorithmus von Dijkstra

Zwanzig Minuten auf einer Café-Terrasse

EDSGER WYBE DIJKSTRA war 1956 Programmierer am Mathematischen Zentrum in Amsterdam. *Man baute dort den Rechner ARMAC und suchte für die feierliche Vorführung ein Problem, das auch Laien verstehen würden.*

> *Dijkstra wählte: „Was ist die kürzeste Verbindung zwischen zwei niederländischen Städten?“* **Er hatte ein Netz aus 64 Städten in die Maschine eingegeben – 64, weil sich damit die Städtenummer in sechs Bit unterbrachte.**

Über die Entstehung des Verfahrens berichtete er 2001 in einem Interview:

> *„Eines Morgens war ich mit meiner jungen Verlobten in Amsterdam einkaufen, und müde vom Bummeln setzten wir uns auf die Café-Terrasse, um eine Tasse Kaffee zu trinken, und ich dachte nur nach, ob ich das könnte, und dann entwarf ich den Algorithmus für den kürzesten Weg. Wie gesagt, es war eine zwanzigminütige Erfindung.“*

Und weiter:

> *„Einer der Gründe dafür, dass er so schön ist, war, dass ich ihn ohne Bleistift und Papier entworfen habe. Ich erfuhr später, dass einer der Vorteile des Entwerfens ohne Bleistift und Papier darin besteht, dass man fast gezwungen ist, alle vermeidbaren Komplexitäten zu vermeiden.“*

Veröffentlicht wurde er erst 1959, auf zweieinhalb Seiten, in der ersten Ausgabe der Numerische Mathematik. **Dijkstra selbst hielt die Arbeit für nebensächlich.**

Sie ist heute eine der meistzitierten Arbeiten der Informatik überhaupt – *und läuft in jedem Navigationsgerät, in jedem Netzwerkrouter und in jedem Routenplaner.*

Das Verfahren

Man führt für jede Ecke eine ***vorläufige Entfernung*** $d[v]$ *mit – die Länge des kürzesten bisher gefundenen Weges. Anfangs ist* $d[s] = 0$ *und alles andere* ∞.

```
für alle v:  d[v] := unendlich;  d[s]
:= 0;
Q := alle Ecken;
while Q nicht leer do {
  u := Ecke aus Q mit kleinstem d[u];
Extrakt-Minimum
  entferne u aus Q;
  für jede Kante (u,v) mit Gewicht w do
    if d[u] + w < d[v] then d[v] := d[u]
+ w }
```

Die letzte Zeile heißt ***Relaxation*** **– man prüft, ob der Umweg über** u **kürzer ist als das bisher Bekannte.**

Die Invariante – und warum das Verfahren stimmt.

Für jede Ecke außerhalb von Q ist $d[v]$ die **endgültige** kürzeste Entfernung.

Der Beweis, dass die Invariante erhalten bleibt.
Sei u die Ecke mit kleinstem $d[u]$ *in* Q. *Angenommen, es gäbe einen kürzeren Weg* P *von* s *nach* u.
Dieser Weg beginnt außerhalb von Q *(bei* s*) und endet in* Q *(bei* u*), muss also irgendwo die Grenze überqueren. Sei* y *die erste Ecke von* P*, die in* Q *liegt.*

Der Vorgänger x *von* y *liegt außerhalb von* Q*, hat also nach Invariante das endgültige* $d[x]$ *– und die Kante* (x, y) *wurde bereits relaxiert.* **Also ist** $d[y]$ **bereits die Länge des Wegstücks bis** y**.**

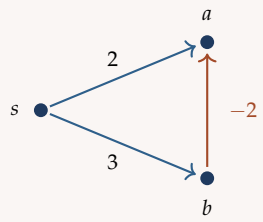
Da alle Gewichte ≥ 0 *sind, ist der Rest des Weges von* y *nach* u *nicht negativ:*

$$d[y] \leq \text{Länge}(P) < d[u].$$

Damit wäre $d[y] < d[u]$ – im Widerspruch dazu, dass u das Minimum in Q war.

Genau hier gehen negative Gewichte kaputt

Der Schritt „der Rest des Weges ist nicht negativ" ist die einzige Stelle, an der die Voraussetzung $w \geq 0$ eingeht – *und ohne sie bricht der Beweis zusammen. Ein Gegenbeispiel mit drei Ecken:*



Dijkstra nimmt zuerst a (mit $d = 2$) und erklärt es für endgültig. **Der wahre kürzeste Weg $s \to b \to a$ hat aber die Länge $3 - 2 = 1$.** *Das Ergebnis ist falsch, und der Fehler wird nie bemerkt.* ✓

> *Das ist eine besonders unangenehme Sorte Fehler:* ***Das Verfahren stürzt nicht ab und meldet nichts*** – es liefert stillschweigend ein falsches Ergebnis.

Der Aufwand hängt an der Datenstruktur

> *Das Verfahren braucht n-mal „Minimum entnehmen" und bis zu m-mal „Schlüssel verkleinern".* **Beides leistet eine Prioritätswarteschlange** (Kapitel 8).

Struktur	Min entn.	Verkleinern	gesamt
Feld	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n^2)$
Binärheap	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}((n+m)\log n)$
Fibonacci-Heap	$\mathcal{O}(\log n)$	$\mathcal{O}(1)^*$	$\mathcal{O}(\mathbf{m + n \log n})$

* *amortisiert.*

> ***Für dichte Graphen ($m \approx n^2$) ist das Feld die beste Wahl*** – $\mathcal{O}(n^2)$ schlägt $\mathcal{O}(n^2 \log n)$. **Für dünne Graphen ($m \approx n$) gewinnt der Heap deutlich.**

Ein Straßennetz ist extrem dünn: Jede Kreuzung hat drei bis fünf Anschlüsse, also $m \approx 2n$. **Deshalb benutzen Routenplaner Heaps.**

Der Fibonacci-Heap ist theoretisch schön und praktisch selten

Michael Fredman und Robert Tarjan entwarfen ihn 1984 genau für diesen Zweck – *und benannten ihn nach den Fibonaccizahlen, weil in der Analyse ein Baum mit k Kindern mindestens F_{k+2} Knoten haben muss.*

In der Praxis wird er selten eingesetzt. **Die versteckten Konstanten sind groß, die Struktur braucht vier Zeiger pro Knoten, und das Speicherverhalten ist schlecht.**

*Messungen zeigen regelmäßig, dass ein einfacher Binärheap oder ein **Pairing-Heap** in realen Graphen schneller ist* – **obwohl er die schlechtere Schranke hat.** *Ein Musterfall dafür, dass die asymptotische Analyse nicht das letzte Wort hat.*

11.2 Bellman-Ford und Floyd-Warshall

Wenn Gewichte negativ sein dürfen

Negative Gewichte klingen künstlich, sind es aber nicht: **Gewinne und Verluste bei Devisengeschäften, Energiegewinn beim Bergabfahren, Belohnungen in Entscheidungsprozessen.**

Der Algorithmus von Bellman und Ford (1955/56) ist verblüffend einfach:

```
d[s] := 0; alle anderen unendlich;
wiederhole (n-1)-mal:
  für jede Kante (u,v,w):  if d[u]+w <
d[v] then d[v] := d[u]+w
für jede Kante (u,v,w):  if d[u]+w <
d[v] then melde "negativer Zyklus"
```

*Man relaxiert einfach **alle** Kanten, $n-1$ mal.* **Ohne jede Cleverness.**

Warum $n-1$ Durchläufe genügen.

Ein kürzester Weg hat höchstens $n-1$ Kanten – er kann keine Ecke zweimal enthalten (sonst gäbe es einen Zyklus, und der wäre nichtnegativ und könnte weggelassen werden).

Nach Durchlauf k sind alle kürzesten Wege mit höchstens k Kanten korrekt berechnet – **das zeigt man durch Induktion über** *k*. ✓
Und die letzte Zeile ist ein Geschenk.

Wenn sich nach $n-1$ Durchläufen noch etwas verbessern lässt, gibt es einen Zyklus mit negativer Gesamtlänge – **und dann existiert kein kürzester Weg, denn man kann den Zyklus beliebig oft durchlaufen.**

***Das Verfahren* erkennt *also den Fall, in dem die Frage sinnlos wird.* Dijkstra kann das nicht.**

	Dijkstra	**Bellman-Ford**
Gewichte	≥ 0	**beliebig**
Aufwand	$\mathcal{O}(m + n \log n)$	$\mathcal{O}(n\,m)$
neg. Zyklen	unbemerkt falsch	**erkannt**
verteilt lauffähig	nein	ja

Die letzte Zeile erklärt, warum Bellman-Ford in der Netzwerktechnik überlebt hat: **Jeder Router kennt nur seine Nachbarn und tauscht mit ihnen Entfernungstabellen aus** – *genau eine verteilte Fassung der Relaxation.* **Das ist das Distanzvektorprotokoll (RIP).**

Floyd-Warshall – alle Paare in drei Zeilen

```
für k := 1 bis n do
  für i := 1 bis n do
    für j := 1 bis n do
      if d[i][k] + d[k][j] < d[i][j]
then d[i][j] := d[i][k]+d[k][j]
```

Vier Zeilen für ein Problem, das man kaum einfacher formulieren kann. Und die Reihenfolge der Schleifen ist entscheidend: k muss außen stehen.

Die Invariante – und warum k außen gehört.

Nach Durchlauf k ist $d[i][j]$ die kürzeste

Entfernung über Zwischenecken aus $\{1,\ldots,k\}$.

Der Induktionsschritt: Ein kürzester Weg über $\{1,\ldots,k\}$ benutzt die Ecke k entweder nicht – dann steht der Wert schon da – oder er benutzt sie genau einmal, und dann zerfällt er in zwei Teilwege über $\{1,\ldots,k-1\}$. ✓

Stünde k innen, so wäre die Invariante nicht formulierbar – **man würde Teilwege benutzen, die noch gar nicht berechnet sind.**

Vergleich für alle Paare	Aufwand		günstig bei
Floyd-Warshall	$\mathcal{O}(n^3)$		**dichten Graphen**
$n\times$ Dijkstra	$\mathcal{O}(nm$ $n^2 \log n)$	+	dünnen Graphen
$n\times$ Bellman-Ford	$\mathcal{O}(n^2 m)$		—
Johnson	$\mathcal{O}(nm$ $n^2 \log n)$	+	dünn, mit negativen Kanten

Bei $n = 100$ Ecken und einer dichten Matrix ist Floyd-Warshall unschlagbar: 10^6 *Operationen, ein einfaches Dreifach-Schleifennest, perfekt cachefreundlich* – **das rechnet auch ein langsamer Interpreter in Sekunden.**

Genau das ist der Fall bei Konnektivitätsmatrizen in der Hirnforschung: wenige hundert Areale, aber dicht verbunden. **Wer dort n-mal Dijkstra laufen lässt, verliert.**

Aufgabe 11.1.

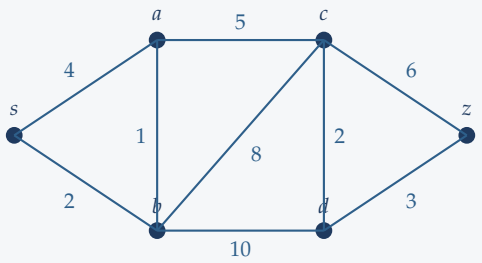
(a) Führen Sie Dijkstras Verfahren an einem konkreten Graphen vollständig durch.

(b) Untersuchen Sie A* und beweisen Sie, wann es korrekt bleibt.

(c) Wenden Sie Floyd-Warshall auf ein Netzwerk an und deuten Sie die Ergebnisse.

Lösung Schritt für Schritt.

(a) Dijkstra, Schritt für Schritt.
Der Graph:



Der Ablauf.

Schritt	gewählt	$d[s]$	$d[a]$	$d[b]$	$d[c]$	$d[d]$
Start	—	0	∞	∞	∞	∞
1	s (0)	0	**4**	**2**	∞	∞
2	b (2)	0	**3**	2	**10**	**12**
3	a (3)	0	3	2	**8**	12
4	c (8)	0	3	2	8	**10**
5	d (10)	0	3	2	8	10

Die einzelnen Schritte begründet.
Schritt 2: *Über b erreicht man a mit $2+1=3<4$* – **eine Verbesserung.** ✓ *Und c mit $2+8=10$, d mit $2+10=12$.*
Schritt 3: *Über a erreicht man c mit $3+5=8<10$* – **wieder eine Verbesserung.** ✓
Schritt 4: *Über c erreicht man d mit $8+2=10<12$.* ✓
Und z:

über	Rechnung	Ergebnis
c	$8+6$	14
d	$10+3$	**13**

$$\boxed{d[z] = 13 \text{ über den Weg } s \to b \to a \to c \to d \to z}$$

Probe: $2+1+5+2+3=13$. ✓

> ***Bemerkenswert:*** *Der kürzeste Weg zu z führt über **fünf** Ecken, obwohl es einen direkten Weg über c mit nur vier gibt.* **Die Kantenzahl sagt nichts über die Weglänge.**

(b) A* – Dijkstra mit Ortskenntnis.

> *Dijkstra sucht in alle Richtungen gleichmäßig – wie eine Wasserwelle.* ***Wenn man das Ziel kennt, ist das verschwenderisch.***

Die Idee (Hart, Nilsson, Raphael, 1968).
Man wählt nicht die Ecke mit kleinstem $d[u]$, sondern die mit kleinstem

$$f(u) = d[u] + h(u),$$

*wobei $h(u)$ eine **Schätzung** der Restentfernung von u zum Ziel ist.*

$d[u]$	was der Weg bis hierher **gekostet** hat – bekannt
$h(u)$	was er von hier noch kosten **wird** – geschätzt

Definition 11.2. Die Schätzfunktion h heißt

zulässig	wenn $h(u) \leq$ wahre Restentfernung – sie unterschätzt nie
konsistent	wenn $h(u) \leq w(u,v) + h(v)$ für jede Kante

Satz 11.3. *Ist h zulässig, so findet A^* den kürzesten Weg. Ist h zusätzlich konsistent, so muss keine Ecke zweimal betrachtet werden.*

Beweisskizze für die Zulässigkeit.
Angenommen, A^* entnimmt das Ziel z mit $f(z) = d[z] > \delta$, wobei δ die wahre kürzeste Entfernung ist.
Auf dem wahren kürzesten Weg liegt dann noch eine unbearbeitete Ecke y mit

$$f(y) = d[y] + h(y) \leq \delta_{\text{bis } y} + \delta_{\text{ab } y} = \delta < d[z] = f(z).$$

Also hätte A^* y vor z nehmen müssen. **Widerspruch.**

Der Kern ist die Ungleichung $h(y) \leq \delta_{ab\,y}$ – also die Zulässigkeit. **Eine Schätzung, die *überschätzt*, kann A* auf einen falschen Weg locken.**

Typische Schätzfunktionen.

Anwendung	$h(u)$	**zulässig, weil**
Straßennetz	Luftlinie zum Ziel	kein Weg ist kürzer
Gitter, 4 Richtungen	Manhattan-Abstand	Mindestzahl der Schritte
Schiebepuzzle	Zahl falscher Steine	jeder braucht ≥ 1 Zug
Dijkstra	$h \equiv 0$	**trivial zulässig**

Die letzte Zeile ist die wichtigste Einsicht: *Dijkstra ist der Sonderfall $h \equiv 0$.*
A* ist keine andere Methode, sondern dieselbe mit Zusatzwissen.

Wie viel es bringt.

Verfahren	besuchte Ecken	Form der Suche
Dijkstra	$\approx \pi r^2$	Kreisscheibe
A* mit Luftlinie	$\approx$ schmale Ellipse	zum Ziel gerichtet

In Straßennetzen spart A typischerweise 80 bis 95 Prozent der Ecken.* **Moderne Routenplaner gehen weit darüber hinaus** – *mit* ***Contraction Hierarchies*** *(2008), die nach einer Vorverarbeitung eine kontinentweite Route in Mikrosekunden liefern.*

(c) Floyd-Warshall an einem Netzwerk.
Vier Knoten, gerichtete Kanten mit Gewichten:

$1 \to 2 : 3, \quad 1 \to 4 : 7, \quad 2 \to 1 : 8, \quad 2 \to 3 : 2, \quad 3 \to 1 : 5, \quad 3 \to 4 : 1, \quad 4 \to 1 : 2.$

Die Ausgangsmatrix.

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

Nach $k = 1$ *(Wege über Knoten 1):*

	1	2	3	4
1	0	3	∞	7
2	8	0	2	**15**
3	5	**8**	0	1
4	2	**5**	∞	0

Etwa $d[3][2] = d[3][1] + d[1][2] = 5 + 3 = 8$. ✓
Nach $k = 2$ *(zusätzlich über Knoten 2):*

	1	2	3	4
1	0	3	**5**	7
2	8	0	2	15
3	5	8	0	1
4	2	5	**7**	0

$d[1][3] = d[1][2] + d[2][3] = 3 + 2 = 5$. ✓
Nach $k = 3$ *(zusätzlich über Knoten 3):*

	1	2	3	4
1	0	3	5	**6**
2	**7**	0	2	**3**
3	5	8	0	1
4	2	5	7	0

$d[1][4] = d[1][3] + d[3][4] = 5 + 1 = 6 < 7$ **– der Umweg** $1 \to 2 \to 3 \to 4$ **ist kürzer als die direkte Kante!** ✓
Nach $k = 4$**:** *keine weiteren Verbesserungen.*

Endmatrix wie nach $k = 3$

Was man aus der Matrix ablesen kann.

Größe	**Rechnung**	**Ergebnis**
Durchmesser	$\max_{i,j} d[i][j]$	8 (von 3 nach 2)
mittlere Weglänge	Summe/12	4,25
Exzentrizität von 1	$\max_j d[1][j]$	6
Exzentrizität von 3		8
Zentrum	kleinste Exzentrizität	Knoten 1

Genau diese Größen sind es, die man in der Netzwerkanalyse braucht – Zentralität, charakteristische Weglänge, Effizienz. **Sie alle fallen aus *einem* Lauf von Floyd-Warshall ab.**

In der Konnektomik heißt die mittlere Weglänge ***characteristic path length*** *und ist zusammen mit dem Häufungskoeffizienten das Maß, mit dem* WATTS *und* STROGATZ *1998 die „Kleine-Welt"-Eigenschaft definierten.*

> *Ihre Arbeit prüfte drei Netze: das Stromnetz des westlichen Nordamerika, die Schauspielerkooperationen der Filmdatenbank – und das* ***vollständige Nervensystem des Fadenwurms*** **C. elegans** *mit 302 Neuronen.*

Alle drei zeigten dasselbe Muster: *kurze mittlere Wege wie in einem Zufallsgraphen, hohe lokale Verklumpung wie in einem Gitter.* **Band 11 und Band 14 dieser Reihe führen das aus.** Dijkstras Originalarbeit[8] ist zweieinhalb Seiten lang; Cormen u. a.[6] beweisen alle vier Verfahren vollständig; Tarjan[26] verbindet Datenstrukturen und Netzwerkalgorithmen; Papadimitriou und Steiglitz[23] ordnen sie in die Optimierung ein.

12 Spannbäume und Flüsse

Das billigste Netz

Definition 12.1. Ein **Spannbaum** eines zusammenhängenden Graphen ist ein Teilgraph, der alle Ecken enthält, zusammenhängend und kreisfrei ist. Bei gewichteten Kanten heißt ein Spannbaum mit minimaler Gewichtssumme **minimaler Spannbaum** (MST).

> *Ein Spannbaum auf n Ecken hat immer genau $n - 1$ Kanten* (Mathematik-Band 15, Kapitel 10) – **das ist die kleinste Zahl, mit der man alles verbinden kann.**

Stromnetz	Alle Orte verbinden, minimale Leitungslänge
Rohrleitungen	Ursprüngliche Anwendung (Borůvka 1926)
Clusteranalyse	MST bilden, teuerste Kanten entfernen
Bildsegmentierung	Regionen als Komponenten des MST
Näherung für TSP	MST liefert eine 2-Näherung

Ein Elektrizitätsnetz für Mähren

Otakar Borůvka war 1926 Mathematiker in Brünn. *Ein Bekannter von der Westmährischen Elektrizitätsgesellschaft fragte ihn, wie man die Orte Südmährens mit möglichst wenig Leitungslänge ans Netz bringe.*

> *Borůvka löste das Problem und veröffentlichte es – auf Tschechisch, in einer mährischen Zeitschrift.* **Es war der erste MST-Algorithmus der Geschichte, und er blieb dreißig Jahre lang praktisch unbekannt.**

Vojtěch Jarník *verbesserte 1930 das Verfahren, ebenfalls auf Tschechisch.* **1956 fand Joseph Kruskal sein Verfahren, 1957 Robert Prim unabhängig dasselbe wie Jarník.**

Der Algorithmus heißt deshalb korrekt „Jarník-Prim", wird aber meist nur nach Prim benannt – **ein Muster, das sich in der Wissenschaftsgeschichte oft findet: Wer auf Englisch publiziert, wird zitiert.**

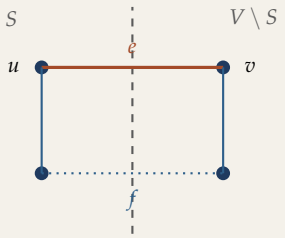
Borůvkas Verfahren erlebte übrigens eine späte Rückkehr: *Weil es alle Komponenten gleichzeitig bearbeitet, lässt es sich hervorragend parallelisieren* – **und ist heute das Verfahren der Wahl auf Grafikprozessoren.**

12.1 Die Schnitteigenschaft

Der eine Satz, aus dem beide Verfahren folgen

Satz 12.2 (Schnitteigenschaft). *Sei $(S, V \setminus S)$ eine beliebige Zerlegung der Eckenmenge und e eine Kante mit **minimalem** Gewicht unter allen, die S mit $V \setminus S$ verbinden. Dann gehört e zu einem minimalen Spannbaum.*

Beweis durch Austausch.
Sei T ein minimaler Spannbaum, der $e = (u, v)$ nicht enthält.
Fügt man e zu T hinzu, entsteht genau ein Kreis (Mathematik-Band 15, Kapitel 10). *Dieser Kreis führt von $u \in S$ nach $v \notin S$ und wieder zurück, muss den Schnitt also noch ein zweites Mal überqueren – über eine Kante f.*



Da e minimal unter den Schnittkanten ist, gilt $w(e) \leq w(f)$. Also ist

$$T' := T - f + e$$

wieder ein Spannbaum (der Kreis ist aufgebrochen, der Zusammenhang bleibt) und hat höchstens dasselbe Gewicht. ✓

Damit ist T' ebenfalls minimal und enthält e. Also gehört e zu *einem* minimalen Spannbaum – was die Behauptung war.

*Sind alle Gewichte **verschieden**, so ist $w(e) < w(f)$ strikt, und der minimale Spannbaum ist sogar **eindeutig**.*

Beide Standardverfahren sind nur zwei Arten, diesen Satz anzuwenden – sie unterscheiden sich allein darin, welchen Schnitt sie betrachten.

	Prim	**Kruskal**
Schnitt	wachsender Baum gegen Rest	jeweils die Komponenten
Wächst	**ein** Baum	**ein Wald**, der zusammenwächst
Struktur	Prioritätswarteschlange	**Union-Find**
Aufwand	$\mathcal{O}(m + n \log n)$	$\mathcal{O}(m \log m)$
gut bei	dichten Graphen	**dünnen Graphen**

Kruskal und die Union-Find-Struktur

```
sortiere alle Kanten nach Gewicht
aufsteigend;
für jede Ecke v:  MakeSet(v);
T := leer;
für jede Kante (u,v) in dieser
Reihenfolge do
  if Find(u) <> Find(v) then { T := T +
(u,v); Union(u,v) }
```

Der ganze Algorithmus besteht darin, die Kanten billigste zuerst durchzugehen und jede zu nehmen, die keinen Kreis schließt. Und „schließt einen Kreis“ heißt genau: Beide Enden liegen schon in derselben Komponente.

Die Struktur dahinter – Union-Find.

`MakeSet(x)`	Legt eine einelementige Menge an
`Find(x)`	Liefert einen Vertreter der Menge von x
`Union(x,y)`	Vereinigt die beiden Mengen

Die Umsetzung: ein Wald von Bäumen, in dem jeder Knoten auf seinen Vater zeigt.

`Find` läuft bis zur Wurzel; `Union` hängt eine Wurzel unter die andere.

Zwei Verbesserungen, die zusammen fast alles wegoptimieren:

Vereinigung nach Rang	Der flachere Baum wird unter den tieferen gehängt – die Tiefe wächst nur langsam
Pfadverkürzung	Bei `Find` werden **alle** durchlaufenen Knoten direkt an die Wurzel gehängt

Satz 12.3 (Tarjan, 1975)**.** *Mit beiden Verbesserungen kosten m Operationen auf n Elementen*

$$\mathcal{O}\big(m \cdot \alpha(n)\big),$$

wobei α die inverse Ackermannfunktion ist.

$\alpha(n)$ ***ist für jedes praktisch vorkommende*** n ***höchstens 4.*** Genauer: $\alpha(n) \leq 4$ für alle n unterhalb von $2^{2^{2^{65536}}}$ **– einer Zahl, die sich nicht als Dezimalzahl im Universum unterbringen ließe.**

Praktisch ist Union-Find also ***konstant*** **– theoretisch aber nicht**, *und* Fredman *und* Saks *bewiesen 1989, dass es besser auch nicht geht.*

Eine der wenigen scharfen unteren Schranken der Algorithmik **– und eine der merkwürdigsten.**

12.2 Flüsse in Netzwerken

Max-Flow-Min-Cut

*Ein gerichteter Graph mit **Kapazitäten** auf den Kanten, einer Quelle s und einer Senke t.* **Wie viel lässt sich höchstens von s nach t transportieren?**

Definition 12.4. Ein **Fluss** ordnet jeder Kante einen Wert $0 \leq f(e) \leq c(e)$ zu, sodass in jeder Ecke außer s und t gilt: Zufluss $=$ Abfluss.
Ein **Schnitt** zerlegt V in $S \ni s$ und $T \ni t$; seine **Kapazität** ist die Summe der Kapazitäten aller Kanten von S nach T.

Satz 12.5 (Ford und Fulkerson, 1956)**.**

$$\max_f |f| = \min_{(S,T)} c(S,T)$$

Der maximale Durchsatz ist genau die Kapazität des Engpasses. **Der vollständige Beweis steht in Mathematik-Band 15, Kapitel 12** – *hier interessiert der Algorithmus.*

Das Verfahren.

1. Beginne mit dem Nullfluss.
2. Suche im **Restnetzwerk** einen Weg von s nach t.
3. Erhöhe den Fluss längs dieses Weges um die kleinste Restkapazität.
4. Wiederhole, bis kein Weg mehr existiert.

*Das **Restnetzwerk** enthält für jede Kante e mit Fluss $f(e)$ zwei Einträge:*

vorwärts	Restkapazität $c(e) - f(e)$
rückwärts	$f(e)$ – man kann Fluss **zurücknehmen**

Die Rückwärtskanten sind der entscheidende Kunstgriff. Ohne sie bliebe das Verfahren in schlechten Zwischenlösungen stecken. **Mit ihnen kann es frühere Entscheidungen revidieren** – ohne dass man jemals „zurückgehen“ müsste.

Die Wegwahl entscheidet über die Laufzeit

Wegwahl	Aufwand	Bemerkung
beliebig	$\mathcal{O}(m \cdot \lvert f^* \rvert)$	hängt am Zahlenwert
kürzester (Edmonds-Karp)	$\mathcal{O}(n\,m^2)$	unabhängig davon
Dinic	$\mathcal{O}(n^2 m)$	mit Schichtgraphen
Push-Relabel	$\mathcal{O}(n^3)$	praktisch schnell
Orlin	$\mathcal{O}(n\,m)$	2013, theoretisch

Die erste Zeile ist gefährlich. *Ein Netz mit vier Ecken und Kapazität* 10^6 *kann Ford-Fulkerson zu einer Million Durchläufen zwingen* – **wenn man immer über die eine Kante mit Kapazität 1 hin und her pendelt.**

Bei ***irrationalen*** *Kapazitäten kann das Verfahren sogar unendlich lange laufen und gegen einen* ***falschen*** *Wert konvergieren.* **Ford und Fulkerson gaben 1962 selbst ein solches Beispiel an.**

Jack Edmonds und Richard Karp zeigten 1972: Nimmt man stets einen *kürzesten* Weg (Breitensuche), so ist die Zahl der Durchläufe durch $\mathcal{O}(nm)$ beschränkt – *unabhängig von den Kapazitäten.*

> *Ihre Arbeit ist zugleich eine der Geburtsurkunden der Komplexitätstheorie:* **Sie unterschied erstmals ausdrücklich zwischen Laufzeiten, die *polynomiell* in der Eingabegröße sind, und solchen, die nur polynomiell in den vorkommenden *Zahlenwerten* sind** (**pseudopolynomiell**).

Aufgabe 12.1.

(a) Führen Sie Kruskal und Prim an demselben Graphen durch und vergleichen Sie.

(b) Analysieren Sie Union-Find mit und ohne Verbesserungen.

(c) Berechnen Sie einen maximalen Fluss und lesen Sie den minimalen Schnitt ab.

Lösung Schritt für Schritt.

(a) Zwei Verfahren, ein Graph.

Sechs Orte, neun mögliche Leitungen:

Kante	w	Kante	w	Kante	w
AB	4	BC	8	CD	7
AF	2	BF	5	CE	6
EF	3	BE	11	DE	9

Kruskal – Kanten aufsteigend.

Kante	w	**Komponenten vorher**	**Entscheidung**
AF	2	alle einzeln	nehmen
EF	3	$\{A, F\}$	nehmen
AB	4	$\{A, E, F\}$	nehmen
BF	5	$\{A, B, E, F\}$	**Kreis** – verwerfen
CE	6	$\{A, B, E, F\}$, $\{C\}$	nehmen
CD	7	$\{A, B, C, E, F\}$	nehmen
BC	8	alles verbunden	verwerfen

$$T = \{AF, EF, AB, CE, CD\}, \qquad w(T) = 2 + 3 + 4 + 6 + 7 = 22$$

Fünf Kanten für sechs Ecken – **genau** $n - 1$. ✓

Prim, gestartet bei A.

Schritt	**Baum**	**billigste Schnittkante**	**genommen**
1	$\{A\}$	AF (2)	F
2	$\{A, F\}$	EF (3)	E
3	$\{A, E, F\}$	AB (4)	B
4	$\{A, B, E, F\}$	CE (6)	C
5	$\{A, B, C, E, F\}$	CD (7)	D

$$w(T) = 2 + 3 + 4 + 6 + 7 = 22. \qquad \checkmark$$

Beide Verfahren liefern denselben Baum – hier sogar dieselben Kanten in derselben Reihenfolge, was Zufall ist. **Da alle Gewichte verschieden sind, ist der minimale Spannbaum eindeutig** – *sie* mussten *übereinstimmen.*

Der Unterschied liegt im Vorgehen.

Kruskal	schaut **global**: die billigste Kante überhaupt; braucht Sortierung
Prim	schaut **lokal**: die billigste Kante am Rand des Baums; braucht keine Sortierung

(b) Union-Find analysieren.
Fall 1 – Ohne jede Verbesserung.
Man hängt bei `Union` *immer den zweiten Baum unter den ersten.*

`Union(1,2)`	$2 \to 1$
`Union(2,3)`	$3 \to 1$? Nein: $3 \to$ Wurzel von $2 = 1$

Ungünstige Reihenfolge: `Union(2,1), Union(3,2), Union(4,3),...`

Es entsteht eine ***Kette*** *der Länge n.* **Ein `Find` am Ende kostet $\mathcal{O}(n)$.**

$$\implies m \text{ Operationen kosten } \mathcal{O}(m\,n).$$

Fall 2 – Nur Vereinigung nach Rang.

Satz 12.6. *Ein Baum vom Rang r enthält mindestens 2^r Knoten.*

Beweis durch Induktion: Der Rang steigt nur, wenn zwei Bäume gleichen Rangs r vereinigt werden – dann hat der neue Baum Rang $r+1$ und mindestens $2^r + 2^r = 2^{r+1}$ Knoten. ✓

$$\implies 2^r \leq n \implies r \leq \log_2 n \implies \texttt{Find} = \mathcal{O}(\log n).$$

Aus $\mathcal{O}(n)$ ist $\mathcal{O}(\log n)$ geworden – **allein durch die Regel, den kleineren Baum unterzuhängen.**

Fall 3 – Beides zusammen.

$$\mathcal{O}(m\,\alpha(n)) \approx \mathcal{O}(m).$$

n	$\alpha(n)$	**Bemerkung**
≤ 2	0	
≤ 3	1	
≤ 7	2	
≤ 2047	3	
$\leq 2^{2^{65536}}$	**4**	**alles Praktische**

Ein Zahlenbeispiel für die Wirkung.

Variante	$m = n = 10^6$	**Verhältnis**
ohne alles	10^{12}	250 000
nur Rang	$2 \cdot 10^7$	5
beides	$4 \cdot 10^6$	**1**

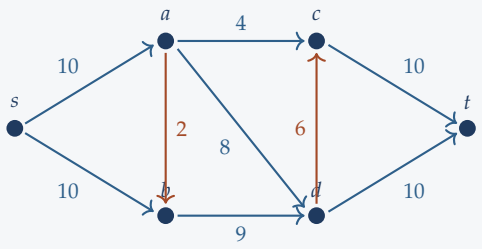
Die Pfadverkürzung ist drei Zeilen Code – man merkt sich beim Hochlaufen die besuchten Knoten und hängt sie danach direkt an die Wurzel. **Der Gewinn ist ein Faktor 250 000.**

Die Rekursion in drei Zeilen:

```
Find(x):
  if vater[x] <> x then vater[x] :=
Find(vater[x]);
  return vater[x]
```

Die Zuweisung in der zweiten Zeile ***ist*** *die Pfadverkürzung.* **Ein einziger Zusatz an der richtigen Stelle.**

(c) Maximaler Fluss.
Netzwerk mit Quelle s, Senke t und vier Zwischenknoten:



Durchlauf 1. *Weg* $s \to a \to c \to t$*; Engpass* $\min(10, 4, 10) = 4$.

$$|f| = 4.$$

Durchlauf 2. *Weg* $s \to a \to d \to t$*; Engpass* $\min(6, 8, 10) = 6$.

(Die Kante $s \to a$ *hat noch Restkapazität* $10 - 4 = 6$*.)*

$$|f| = 10.$$

Durchlauf 3. *Weg* $s \to b \to d \to t$*; Engpass* $\min(10, 9, 4) = 4$.

(Die Kante $d \to t$ *hat noch* $10 - 6 = 4$*.)*

$$|f| = 14.$$

Durchlauf 4. *Weg* $s \to b \to d \to c \to t$*; Engpass* $\min(6, 5, 6, 6) = 5$.

(Hier wird die Kante $d \to c$ *mit Kapazität 6 genutzt, und* $c \to t$ *hat noch* $10 - 4 = 6$*.)*

$$|f| = 19.$$

Durchlauf 5. *Kein Weg mehr:* $s \to a$ *hat Rest 0,* $s \to b$ *hat Rest 1, aber* $b \to d$ *ist erschöpft (9 voll) und b hat keine andere ausgehende Kante.*

$$\boxed{|f^*| = 19}$$

Den minimalen Schnitt ablesen.

S ist die Menge der Ecken, die im ***Restnetzwerk*** *noch von s aus erreichbar sind.*

s	Startpunkt
b	über $s \to b$ (Rest 1)
a	nicht erreichbar – $s \to a$ ist voll
c, d, t	nicht erreichbar

$$S = \{s, b\}, \qquad T = \{a, c, d, t\}.$$

Die Kapazität des Schnitts.

Kante $S \to T$	**Kapazität**	**Fluss**
$s \to a$	10	10 (voll)
$b \to d$	9	9 (voll)
Summe	**19**	**19**

$$\boxed{\max f = 19 = \min\text{-Schnitt} \qquad \checkmark}$$

Alle Schnittkanten sind voll ausgelastet, und keine Kante läuft zurück über den Schnitt. Das ist genau die Bedingung, die den Satz beweist – und zugleich das **Zertifikat**: Wer Fluss und Schnitt gleicher Größe vorlegt, hat die Optimalität *bewiesen*.

Warum das praktisch zählt.

Man muss dem Ergebnis nicht glauben und den Algorithmus nicht prüfen. **Die beiden Zahlen prüft man in $\mathcal{O}(m)$ nach, und wenn sie übereinstimmen, ist beides optimal.**

*Man nennt das ein **zertifizierendes Verfahren*** **– ein Ideal, das in der Algorithmik selten erreicht wird und das MEHLHORN zu einem Programm für ganze Bibliotheken gemacht hat**[19].

*Der Engpass ist übrigens aufschlussreich: Er liegt **nicht** an den Kanten mit den kleinsten Kapazitäten, sondern an der Kombination $s \to a$ und $b \to d$* **– eine Stelle, die man dem Bild nicht ansieht.**

Kleinberg und Tardos[15] entwickeln Flüsse mit besonders vielen Anwendungen; Cormen u. a.[6] beweisen alles vollständig; Erickson[9] stellt sein Lehrbuch frei zur Verfügung und ist bei den Beweisen ungewöhnlich sorgfältig.

Part VI

Entwurfsmuster

13 Teile und herrsche, und die gierigen Verfahren

Drei Muster, aus denen fast alles besteht

Muster	Vorgehen	Voraussetzung
Teile und herrsche	Zerlegen, lösen, zusammensetzen	Teilprobleme sind **unabhängig**
Gierig	In jedem Schritt das lokal Beste	**Matroid-Eigenschaft** oder Austauschargument
Dynamisch	Teilergebnisse speichern	Teilprobleme **überlappen** sich

Die dritte Spalte enthält die eigentliche Aussage. Die drei Muster unterscheiden sich nicht darin, was sie tun, sondern darin, wann sie erlaubt sind.

Kapitel 14 behandelt das dritte Muster; hier geht es um die ersten beiden.

13.1 Teile und herrsche

Das Schema

1. **Teile** das Problem in a Teilprobleme der Größe n/b.
2. **Herrsche**: Löse jedes Teilproblem rekursiv.
3. **Verbinde** die Teillösungen zur Gesamtlösung.

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n).$$

Das Master-Theorem (Kapitel 1) *löst diese Gleichung.*

> ***Der Gewinn entsteht dadurch, dass man a klein hält.*** Wer aus vier Teilproblemen drei macht, senkt den Exponenten von $\log_b 4$ auf $\log_b 3$ – **und das ist der ganze Trick von Karatsuba und Strassen.**

Wie ein Student eine Vermutung Kolmogorows widerlegte

Andrei Kolmogorow hielt 1960 an der Moskauer Universität ein Seminar über Komplexität. *Er vermutete dort, dass die Multiplikation zweier n-stelliger Zahlen mindestens $\Omega(n^2)$ Elementaroperationen erfordert* **– die Schulmethode also im Wesentlichen optimal sei.**

> *Im Publikum saß* Anatoli Karatsuba, *23 Jahre alt.* **Er widerlegte die Vermutung binnen einer Woche.**

Seine Idee. *Man zerlegt zwei n-stellige Zahlen in Hälften:*

$$x = x_1 \cdot B + x_0, \qquad y = y_1 \cdot B + y_0, \qquad B = 10^{n/2}.$$

Dann ist

$$x \cdot y = x_1 y_1 \, B^2 + (x_1 y_0 + x_0 y_1)\, B + x_0 y_0.$$

*Das sind **vier** Multiplikationen halber Größe* **– und $T(n) = 4T(n/2) + \mathcal{O}(n)$ ergibt wieder $\Theta(n^2)$.** *Kein Gewinn.*

Karatsubas Beobachtung war, dass drei genügen:

$$p := x_1 y_1, \qquad q := x_0 y_0, \qquad r := (x_1 + x_0)(y_1 + y_0).$$

Denn

$$x_1 y_0 + x_0 y_1 = r - p - q.$$

Nachrechnen: $r = x_1 y_1 + x_1 y_0 + x_0 y_1 + x_0 y_0 = p + (x_1 y_0 + x_0 y_1) + q$. ✓

> ***Drei Multiplikationen statt vier, dafür ein paar Additionen mehr.***

$$T(n) = 3T\left(\frac{n}{2}\right) + \mathcal{O}(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1{,}585}).$$

Kolmogorow war so beeindruckt, dass er die Arbeit selbst schrieb und einreichte *– allerdings, so berichtete Karatsuba später, ohne ihn zu fragen und mit einigen Ungenauigkeiten.* **Karatsuba erfuhr von der Veröffentlichung erst, als er die gedruckten Belege erhielt.**

> *Das Seminar wurde nach diesem Ergebnis geschlossen.*

Verfahren	Aufwand	Jahr
Schulmethode	$\Theta(n^2)$	antik
Karatsuba	$\Theta(n^{1{,}585})$	1960
Toom-Cook	$\Theta(n^{1{,}465})$	1963
Schönhage-Strassen	$\mathcal{O}(n \log n \log\log n)$	1971
Harvey–van der Hoeven	$\mathcal{O}(n \log n)$	**2019**

Die letzte Zeile schloss 2019 ein Problem ab, das seit 1971 offen war – **Schönhage und Strassen hatten damals vermutet,** $\mathcal{O}(n \log n)$ **sei die Wahrheit, und genau das wurde bewiesen.** *Das Verfahren ist allerdings erst für Zahlen mit mehr als* $2^{1729^{12}}$ *Bit schneller –* **ein galaktischer Algorithmus.**

13.2 Gierige Verfahren

Wann darf man den Augenblick entscheiden lassen?

Ein ***gieriges Verfahren*** *trifft in jedem Schritt die lokal beste Entscheidung und* ***nimmt sie nie zurück.*** **Das ist schnell und meistens falsch** *– aber in bemerkenswert vielen Fällen richtig.*

Problem	Gierig?	Bemerkung
Minimaler Spannbaum	**optimal**	Kruskal, Prim
Kürzeste Wege ($w \geq 0$)	**optimal**	Dijkstra
Huffman-Kodierung	**optimal**	
Aktivitätenauswahl	**optimal**	
Wechselgeld (Euro)	optimal	**nur wegen der Münzwerte**
Wechselgeld (allgemein)	**falsch**	
Rucksackproblem (ganzz.)	**falsch**	
Rucksack (teilbar)	optimal	

Das Wechselgeldproblem – ein Musterbeispiel

Aufgabe: Ein Betrag ist mit möglichst wenigen Münzen zu geben.
Gieriges Verfahren: Nimm stets die größte Münze, die noch passt.
Mit den Euro-Werten $\{1, 2, 5, 10, 20, 50, 100, 200\}$ *funktioniert das:*

$$0{,}68 \text{ Euro} \Longrightarrow 50 + 10 + 5 + 2 + 1 = 5 \text{ Münzen.} \qquad \checkmark$$

Mit den Werten $\{1, 3, 4\}$ **und dem Betrag 6 nicht:**

Verfahren	Zerlegung	Münzen
gierig	$4 + 1 + 1$	3
optimal	$3 + 3$	2

> ***Die 4 ist eine Falle:*** *Sie sieht lokal besser aus als die 3, verhindert aber die bessere Gesamtlösung.* **Genau das ist das Wesen des Scheiterns gieriger Verfahren.**

Ob ein Münzsystem „gierigtauglich" ist, lässt sich prüfen – **Pearson gab 2005 ein Verfahren in** $\mathcal{O}(k^3)$ **an, das nur die** k **Münzwerte betrachtet.** *Das allgemeine Wechselgeldproblem dagegen ist NP-schwer und braucht dynamische Programmierung* (Kapitel 14).

Historische Randnotiz: Das britische Vor-1971-System mit 1, 3, 6, 12, 24, 30 Pence war ***nicht*** *gierigtauglich –* **man konnte durch gieriges Herausgeben zu viele Münzen abzählen.**

Wie man beweist, dass Gier optimal ist

Es gibt zwei Standardargumente, und beide lohnen sich zu kennen.

Argument 1 – Die gierige Wahl bleibt vorn (*greedy stays ahead*).

> *Man zeigt durch Induktion: Nach k Schritten ist die gierige Teillösung mindestens so gut wie jede andere Teillösung mit k Schritten.*

Argument 2 – Austausch (*exchange argument*).

> *Man nimmt eine optimale Lösung und formt sie schrittweise in die gierige um, ohne sie zu verschlechtern.* **Also ist die gierige ebenfalls optimal.**

Der Beweis der Schnitteigenschaft in Kapitel 12 war ein Austauschargument. ✓

> *Die tieferliegende Erklärung liefert die* ***Matroidtheorie*** *(*WHITNEY *1935,* EDMONDS *1971):* ***Gierige Verfahren sind genau dann für alle Gewichtungen optimal, wenn die zulässigen Mengen ein Matroid bilden.***
>
> *Die kreisfreien Kantenmengen eines Graphen bilden ein Matroid* – **deshalb funktioniert Kruskal.** *Die Münzzerlegungen bilden keines.*

Aufgabe 13.1.

(a) Lösen Sie die Aktivitätenauswahl gierig und beweisen Sie die Optimalität.
(b) Entwickeln Sie die Huffman-Kodierung und beweisen Sie sie.
(c) Vergleichen Sie das teilbare und das ganzzahlige Rucksackproblem.

Lösung Schritt für Schritt.

(a) Aktivitätenauswahl.
Gegeben sind n Veranstaltungen mit Anfangs- und Endzeit. Gesucht ist die größte Zahl von Veranstaltungen, die sich nicht überschneiden – etwa für einen einzigen Hörsaal.

Nr.	Start	Ende	Nr.	Start	Ende
1	1	4	6	5	9
2	3	5	7	6	10
3	0	6	8	8	11
4	5	7	9	2	14
5	3	9	10	12	16

Schritt 1 – Die richtige Gierregel finden.

Regel	**optimal?**	**Gegenbeispiel**
kürzeste zuerst	nein	eine kurze in der Mitte blockiert zwei lange
frühester Anfang	nein	eine sehr lange am Anfang
wenigste Konflikte	nein	konstruierbar
frühestes Ende	**ja**	—

Die Regel „frühestes Ende zuerst“ lässt möglichst viel Restzeit übrig – und genau das ist die richtige Intuition. **Die anderen drei sind alle plausibel und alle falsch.**

Schritt 2 – Durchführen.
Sortiert nach Endzeit: 1(4), 2(5), 3(6), 4(7), 5(9), 6(9), 7(10), 8(11), 9(14), 10(16).

Nr.	Zeitraum	letztes Ende	Entscheidung
1	$[1,4]$	—	nehmen
2	$[3,5]$	4	$3 < 4$: nein
3	$[0,6]$	4	nein
4	$[5,7]$	4	nehmen
5	$[3,9]$	7	nein
6	$[5,9]$	7	nein
7	$[6,10]$	7	nein
8	$[8,11]$	7	nehmen
9	$[2,14]$	11	nein
10	$[12,16]$	11	nehmen

$\{1,4,8,10\}$ $--$ vier Veranstaltungen

Schritt 3 – Der Beweis (Austauschargument).
Sei $A = \{a_1, \ldots, a_k\}$ die gierige Lösung, nach Endzeit geordnet, und $O = \{o_1, \ldots, o_m\}$ eine optimale, ebenfalls geordnet.
Behauptung: *Für jedes i endet a_i nicht später als o_i.*
Induktionsanfang. a_1 ist die Veranstaltung mit dem frühesten Ende überhaupt, also endet sie nicht später als o_1. ✓
Schritt. Gelte $\text{ende}(a_{i-1}) \leq \text{ende}(o_{i-1})$. Da o_i nach o_{i-1} beginnt, beginnt es auch nach a_{i-1} – **es war also für das gierige Verfahren wählbar.** *Und da dieses das früheste Ende nimmt:*

$$\text{ende}(a_i) \leq \text{ende}(o_i). \qquad \checkmark$$

Schluss. Wäre $m > k$, so gäbe es ein o_{k+1}. Es beginnt nach $\text{ende}(o_k) \geq \text{ende}(a_k)$ – **also wäre es auch für das gierige Verfahren noch wählbar gewesen**, *im Widerspruch dazu, dass dieses bei a_k aufgehört hat.*

$$\Longrightarrow m = k.$$

Schritt 4 – Aufwand.

$$\mathcal{O}(n \log n) \text{ für das Sortieren} + \mathcal{O}(n) \text{ für den Durchlauf} = \mathcal{O}(n \log n).$$

Eine Variante ist NP-schwer: *Fragt man nach der kleinsten Zahl von* ***Hörsälen****, in denen sich* alle *Veranstaltungen unterbringen lassen, so ist auch das gierig lösbar (Intervallgraphenfärbung).* **Fragt man aber nach der maximalen Gesamt*dauer* statt der Anzahl, wird es dynamische Programmierung** (Kapitel 14).

(b) Die Huffman-Kodierung.

Aufgabe: Ein Text soll mit möglichst wenigen Bits kodiert werden. Häufige Zeichen sollen kurze Codes bekommen, seltene lange – **und der Code muss ohne Trennzeichen lesbar bleiben.**

Definition 13.1. Ein Code heißt **präfixfrei**, wenn kein Codewort Anfangsstück eines anderen ist.

Präfixfreie Codes entsprechen genau den Binärbäumen, deren ***Blätter*** *die Zeichen tragen.* **Der Weg von der Wurzel zum Blatt ist das Codewort** (links $= 0$, rechts $= 1$).

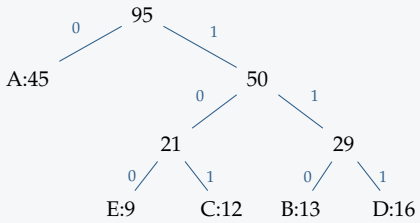
Das Verfahren (Huffman, 1952).

1. Lege für jedes Zeichen einen Knoten mit seiner Häufigkeit an.
2. Nimm die **zwei kleinsten** Knoten, verbinde sie unter einem neuen Knoten mit der Summe ihrer Häufigkeiten.
3. Wiederhole, bis nur ein Knoten übrig ist.

Ein Beispiel.

Zeichen	A	B	C	D	E
Häufigkeit	45	13	12	16	9

Schritt	**verbunden**	**Vorrat danach**
1	E(9) + C(12) = 21	A45, B13, D16, **21**
2	B(13) + D(16) = 29	A45, 21, **29**
3	21 + 29 = 50	A45, **50**
4	A(45) + 50 = 95	**95**



Zeichen	Code	Länge	Häufigk.	Bits
A	`0`	1	45	45
E	`100`	3	9	27
C	`101`	3	12	36
B	`110`	3	13	39
D	`111`	3	16	48
			95	**195**

Der Vergleich.

Kodierung	**Bits**	**Bits/Zeichen**
feste Länge (3 Bit)	285	3,00
Huffman	**195**	**2,05**
Entropie (untere Schranke)	≈ 193	2,03

> ***Huffman erreicht 195 Bit, die informationstheoretische Untergrenze liegt bei 193*** (Band 0, Kapitel 1). **Die Lücke rührt daher, dass Codewortlängen ganzzahlig sein müssen.**

Der Beweis der Optimalität – Austauschargument.
Hilfssatz 1. *In einem optimalen Baum sind die beiden* ***seltensten*** *Zeichen Geschwister auf der tiefsten Ebene.*
Beweis: Wären sie es nicht, so vertausche man sie mit den beiden tiefsten Blättern. Da sie die kleinsten Häufigkeiten haben, wird die Gesamtlänge dabei nicht größer. ✓
Hilfssatz 2. *Ersetzt man die beiden seltensten Zeichen durch ein zusammengefasstes mit der Summenhäufigkeit, so ist ein optimaler Baum des kleineren Problems zu einem optimalen des größeren erweiterbar.*
Beweis: Die Kosten unterscheiden sich um genau $f_1 + f_2$ – unabhängig davon, wie der Rest aussieht. ✓

Induktion über die Zeichenzahl liefert die Behauptung.

DAVID HUFFMAN *war 1951 Doktorand am MIT.* Sein Betreuer ROBERT FANO hatte der Klasse freigestellt, eine Abschlussarbeit zu schreiben oder eine Prüfung abzulegen. **Die Aufgabe: den optimalen präfixfreien Code finden.**

Huffman arbeitete monatelang erfolglos und wollte gerade aufgeben und für die Prüfung lernen – **als ihm beim Wegwerfen seiner Notizen die Idee kam, den Baum *von unten* statt von oben zu bauen.**

Was er nicht wusste: Fano und SHANNON *hatten an genau diesem Problem gearbeitet und es nicht gelöst.* **Der Shannon-Fano-Code arbeitet von oben nach unten und ist nicht optimal.**

(c) Rucksack, teilbar und ganzzahlig.
Ein Rucksack trägt 50 kg. Zur Auswahl stehen:

Gegenst.	Gewicht	Wert	Wert/Gewicht
1	10	60	**6,0**
2	20	100	**5,0**
3	30	120	**4,0**

Fall 1 – Teilbar (*fractional knapsack*).
Man darf Bruchteile mitnehmen. Gierig nach Wertdichte:

Gegenstand	genommen	Gewicht	Wert
1	ganz	10	60
2	ganz	20	100
3	$\frac{20}{30} = \frac{2}{3}$	20	80
		50	**240**

Optimal: 240

Warum die Gier hier stimmt.
Austauschargument: Enthielte eine optimale Lösung weniger von einem Gegenstand höherer Dichte und mehr von einem niedrigerer, so könnte man ein Kilogramm tauschen und den Wert erhöhen. **Also ist sie nicht optimal.** ✓
Fall 2 – Ganzzahlig (*0/1 knapsack*).
Jeder Gegenstand ganz oder gar nicht. Gierig nach Dichte:

Verfahren	Auswahl	Gewicht	Wert
gierig	$\{1,2\}$	30	**160**
optimal	$\{2,3\}$	50	**220**

Die Gier nimmt Gegenstand 1 mit der höchsten Dichte – und blockiert damit die bessere Kombination. Danach passt Gegenstand 3 nicht mehr hinein, und 20 kg bleiben ungenutzt.

Der Unterschied in einem Satz.

teilbar	Man kann den Rucksack **immer randvoll** machen – es gibt keinen verschwendeten Platz
ganzzahlig	Es bleibt Platz übrig, und wie viel, hängt von der ganzen Auswahl ab

Genau diese Kopplung zerstört die Voraussetzung des gierigen Vorgehens: *Die Entscheidung über Gegenstand 1 lässt sich nicht treffen, ohne die Folgen für alle anderen zu kennen.*

Der Status der beiden Probleme.

	teilbar	**ganzzahlig**
Verfahren	gierig	**dynamisch**
Aufwand	$\mathcal{O}(n \log n)$	$\mathcal{O}(n\,W)$
Komplexität	in P	**NP-schwer**

$\mathcal{O}(nW)$ ist nicht polynomiell

Die Schranke $\mathcal{O}(nW)$ sieht harmlos aus und täuscht.
*Die **Eingabegröße** ist nicht W, sondern die Zahl der Bits, die man zum Aufschreiben von W braucht – also $\log_2 W$.*

Bei $W = 2^{64}$ ist die Eingabe wenige Byte lang, und $\mathcal{O}(nW)$ bedeutet 10^{19} Schritte.

*Man nennt solche Verfahren **pseudopolynomiell**: polynomiell im **Zahlenwert**, exponentiell in der **Eingabelänge**.* **Der Unterschied wurde 1972 von EDMONDS und KARP eingeführt** (Kapitel 12) **und ist eine der Grundunterscheidungen der Komplexitätstheorie** (Band 3).

Kleinberg und Tardos[15] behandeln gierige Verfahren mit ungewöhnlich sorgfältigen Beweisen; Dasgupta, Papadimitriou und Vazirani[7] sind knapp und elegant; Bird[3] leitet viele dieser Algorithmen formal aus ihrer Spezifikation her.

14 Dynamische Programmierung

Wenn Teilprobleme sich überlappen

*Bei „Teile und herrsche“ sind die Teilprobleme **verschieden** – die linke und die rechte Hälfte haben nichts miteinander zu tun.* **Bei der dynamischen Programmierung überlappen sie sich, und dieselben Teilprobleme tauchen immer wieder auf.**

Das Musterbeispiel steht schon in Kapitel 1: die Fibonaccizahlen.

n	**naiv rekursiv**	**mit Speicherung**
30	2,7 Mio. Aufrufe	59
50	$4{,}0 \cdot 10^{10}$	99
100	10^{21}	199

Von exponentiell auf linear – **allein dadurch, dass man aufschreibt, was man schon ausgerechnet hat.**

Definition 14.1. Ein Problem eignet sich für dynamische Programmierung, wenn es

optimale Teilstruktur	Die optimale Lösung enthält optimale Lösungen der Teilprobleme
überlappende Teilprobleme	Dieselben Teilprobleme kommen mehrfach vor

besitzt.

Die erste Bedingung teilt es mit den gierigen Verfahren (Kapitel 13). **Die zweite ist der Unterschied zu „Teile und herrsche“.**

Zwei Bauformen:

	Memoisierung	**Tabellenaufbau**
Richtung	**oben nach unten**	**unten nach oben**
Form	Rekursion + Speicher	Schleifen
Berechnet	nur Benötigtes	**alles**
Vorteil	näher am Gedanken	kein Stapel, cachefreundlich

Warum es „dynamische Programmierung“ heißt

Der Name ist irreführend – und das mit Absicht.
RICHARD BELLMAN *entwickelte das Verfahren ab 1949 bei der RAND Corporation. In seiner Autobiographie* Eye of the Hurricane *(1984) erklärte er die Namenswahl:*

> *„Die 1950er Jahre waren keine guten Jahre für mathematische Forschung. Wir hatten in Washington einen sehr interessanten Herrn namens Wilson. Er war Verteidigungsminister, und er hatte tatsächlich eine krankhafte Angst und einen Hass auf das Wort* Forschung. [...] *Sie können sich vorstellen, wie er sich fühlte, wenn er den Begriff* mathematisch *hörte.“*
>
> *„Ich musste also einen Schutzschirm haben. Welchen Titel, welchen Namen konnte ich wählen?* [...] Programmierung *klang gut, und* dynamisch *hat eine absolut präzise Bedeutung im Sinne der Physik. Außerdem ist es ein Adjektiv, und es ist unmöglich, das Wort* dynamisch *in einem abwertenden Sinn zu gebrauchen.“*
>
> *„Nehmen Sie es also als einen Namen, den man nicht anfechten konnte.“*

Historiker haben Bellmans Darstellung teilweise angezweifelt – *Charles Erwin Wilson wurde erst 1953 Verteidigungsminister, während Bellman den Begriff schon 1952 benutzte.* **Der Kern stimmt aber: „Programmierung“ meinte damals *Planung*, nicht Programmieren am Rechner**, *genau wie in „lineare Programmierung“.*

> *Bellman formulierte auch das* ***Optimalitätsprinzip****, das dem ganzen Gebiet zugrundeliegt:*
>
> *„Eine optimale Strategie hat die Eigenschaft, dass – unabhängig vom Anfangszustand und der ersten Entscheidung – die verbleibenden Entscheidungen eine optimale Strategie bezüglich des Zustands bilden, der aus der ersten Entscheidung hervorgeht.“*

Dieselbe Aussage, die wir in Kapitel 11 für kürzeste Wege bewiesen haben – *nur in der Sprache der Regelungstheorie.*

14.1 Das Schema

Fünf Schritte, immer dieselben

1. **Teilprobleme benennen**: Was genau soll $T[i]$ bzw. $T[i][j]$ bedeuten?
2. **Rekursion aufstellen**: Wie setzt sich $T[i]$ aus kleineren zusammen?
3. **Randfälle** festlegen.
4. **Reihenfolge** bestimmen, in der die Tabelle gefüllt wird.
5. **Lösung rekonstruieren** – meist braucht man nicht nur den Wert, sondern die Wahl.

Schritt 1 ist der schwierigste und wird meist unterschätzt. Wer das Teilproblem falsch ansetzt, findet keine Rekursion – und merkt oft nicht, dass das der Grund ist.

Schritt 5 wird häufig vergessen. **Die Tabelle enthält den *Wert* der optimalen Lösung; um die Lösung selbst zu bekommen, muss man rückwärts durch die Tabelle laufen** – *oder sich beim Füllen merken, welche Wahl jeweils gewonnen hat.*

Aufgabe 14.1.

(a) Berechnen Sie den Editierabstand zweier Wörter vollständig und rekonstruieren Sie die Umformung.
(b) Lösen Sie das ganzzahlige Rucksackproblem und vergleichen Sie mit dem gierigen Ergebnis.
(c) Bestimmen Sie die längste gemeinsame Teilfolge und diskutieren Sie den Speicherbedarf.

Lösung Schritt für Schritt.

(a) Der Editierabstand.

Definition 14.2. Der **Levenshtein-Abstand** zweier Zeichenketten ist die kleinste Zahl von Einfügungen, Löschungen und Ersetzungen, die eine in die andere überführt.

WLADIMIR LEWENSTEIN *führte ihn 1965 ein – in einer Arbeit über fehlerkorrigierende Codes, nicht über Text.*

Schritt 1 – Teilproblem benennen.

$$\boxed{D[i][j] := \text{Editierabstand zwischen } a[1\ldots i] \text{ und } b[1\ldots j]}$$

Diese Definition ist der ganze Trick. Man betrachtet nicht die vollen Wörter, sondern alle **Präfixpaare** – und davon gibt es nur $(m+1)(n+1)$.

Schritt 2 – Rekursion.
Wie kann der letzte Schritt einer optimalen Umformung aussehen? Es gibt genau drei Möglichkeiten:

Letzter Schritt	**Rest**	**Kosten**
$a[i]$ löschen	$D[i-1][j]$	$+1$
$b[j]$ einfügen	$D[i][j-1]$	$+1$
$a[i]$ zu $b[j]$ machen	$D[i-1][j-1]$	$+1$, oder $+0$ falls gleich

$$D[i][j] = \min \begin{cases} D[i-1][j]+1 \\ D[i][j-1]+1 \\ D[i-1][j-1] + [a_i \neq b_j] \end{cases}$$

Schritt 3 – Randfälle.

$$D[i][0] = i \quad \text{(alles löschen)}, \qquad D[0][j] = j \quad \text{(alles einfügen)}.$$

Schritt 4 – Die Tabelle für `KATZE` → `KASSE`.

	ε	K	A	S	S	E
ε	0	1	2	3	4	5
K	1	**0**	1	2	3	4
A	2	1	**0**	1	2	3
T	3	2	1	**1**	2	3
Z	4	3	2	2	**2**	3
E	5	4	3	3	3	**2**

Zwei Zellen ausgerechnet.
$D[3][3]$ *(also* `KAT` *gegen* `KAS`*):* Die Buchstaben `T` und `S` sind verschieden.

$$\min\big(D[2][3]+1,\ D[3][2]+1,\ D[2][2]+1\big) = \min(1+1,\ 1+1,\ 0+1) = 1.$$

Die Diagonale gewinnt: eine Ersetzung. ✓

$D[5][5]$ *(also* `KATZE` *gegen* `KASSE`*):* Beide enden auf `E`, also kostet die Diagonale nichts.

$$\min\big(3+1,\ 3+1,\ 2+0\big) = 2. \qquad ✓$$

$$\boxed{D(\texttt{KATZE},\texttt{KASSE}) = 2}$$

Schritt 5 – Die Umformung rekonstruieren.
Man läuft von $D[5][5]$ rückwärts und merkt sich, welcher Fall gewonnen hat:

Zelle		Herkunft	Operation
$D[5][5]$ 2	=	diagonal, $+0$	`E` → `E`: nichts
$D[4][4]$ 2	=	diagonal, $+1$	**`Z`** → **`S`**
$D[3][3]$ 1	=	diagonal, $+1$	**`T`** → **`S`**
$D[2][2]$ 0	=	diagonal, $+0$	`A` → `A`
$D[1][1]$ 0	=	diagonal, $+0$	`K` → `K`

`KATZE` → `KASZE` → `KASSE`. ✓

Aufwand.

$\mathcal{O}(m\,n)$ Zeit und Speicher.

> *Braucht man nur den* ***Wert*** *und nicht die Umformung, so genügen* ***zwei Zeilen*** *–* $\mathcal{O}(\min(m,n))$ **Speicher.** *Denn jede Zelle hängt nur von der Zeile darüber und der Zelle links ab.*

Wo das eingesetzt wird.

Rechtschreibprüfung	Vorschläge mit kleinem Abstand
`diff`	Zeilenweiser Vergleich zweier Dateien
Bioinformatik	Sequenzvergleich (Needleman-Wunsch, 1970)
Spracherkennung	Vergleich von Lautfolgen
Plagiatserkennung	Ähnlichkeit von Textabschnitten

Schneller als $\mathcal{O}(mn)$ geht es vermutlich nicht

Backurs und Indyk zeigten 2015 ein bemerkenswertes Ergebnis:

> *Gäbe es ein Verfahren mit Laufzeit $\mathcal{O}(n^{2-\varepsilon})$ für irgendein $\varepsilon > 0$, so wäre die **starke Exponentialzeithypothese** falsch* – **und damit ließe sich das Erfüllbarkeitsproblem der Aussagenlogik deutlich schneller lösen als bisher bekannt.**

Das ist kein Beweis der Unmöglichkeit, aber ein starkes Indiz. **Man nennt solche Aussagen feinkörnige Komplexität** – *ein Gebiet, das seit etwa 2010 untersucht, warum Probleme in* P *trotzdem nicht schneller werden* (Band 3).

(b) Das Rucksackproblem, ganzzahlig.
Wir nehmen das Beispiel aus Kapitel 13: Kapazität 50; Gegenstände $(10, 60)$, $(20, 100)$, $(30, 120)$.
Schritt 1 – Teilproblem.

$$\boxed{K[i][w] := \text{größter Wert mit den ersten } i \text{ Gegenständen und Kapazität } w}$$

Schritt 2 – Rekursion.

$$K[i][w] = \max \begin{cases} K[i-1][w] & \text{Gegenstand } i \text{ weglassen} \\ K[i-1][w-g_i] + v_i & \text{mitnehmen (falls } g_i \le w) \end{cases}$$

Schritt 3 – Randfall. $K[0][w] = 0$ für alle w.
Schritt 4 – Die Tabelle (nur die Kapazitäten $0, 10, 20, 30, 40, 50$):

$i \setminus w$	0	10	20	30	40	50
0	0	0	0	0	0	0
1 (10,60)	0	60	60	60	60	60
2 (20,100)	0	60	100	**160**	160	160
3 (30,120)	0	60	100	160	180	**220**

Zwei Zellen begründet.
$K[2][30]$*:* weglassen ergibt $K[1][30] = 60$; mitnehmen ergibt $K[1][10] + 100 = 60 + 100 = 160$.

$$\max(60, 160) = 160. \qquad \checkmark$$

$K[3][50]$*:* weglassen ergibt $K[2][50] = 160$; mitnehmen ergibt $K[2][20] + 120 = 100 + 120 = 220$.

$$\max(160, 220) = \mathbf{220}. \qquad \checkmark$$

Schritt 5 – Rekonstruktion.

Zelle		**Vergleich**	**Schluss**
$K[3][50]$ 220	$=$	$\neq K[2][50] = 160$	**Gegenstand 3 dabei**
$K[2][20]$ 100	$=$	$\neq K[1][20] = 60$	**Gegenstand 2 dabei**
$K[1][0] = 0$		$= K[0][0] = 0$	Gegenstand 1 nicht

$\{2, 3\}$, Gewicht 50, Wert 220

Der Vergleich mit der Gier.

Verfahren	**Auswahl**	**Wert**	**Aufwand**
gierig	$\{1, 2\}$	160	$\mathcal{O}(n \log n)$
dynamisch	$\{2, 3\}$	**220**	$\mathcal{O}(nW)$

38 Prozent mehr Wert – für einen Aufwand, der in diesem Fall $3 \cdot 51 = 153$ ***Zellen beträgt.*** Bei größeren Kapazitäten kehrt sich das Verhältnis allerdings um: $W = 10^9$ macht die Tabelle unbenutzbar.

(c) Längste gemeinsame Teilfolge.

Definition 14.3. Eine **Teilfolge** entsteht durch Streichen von Zeichen, ohne die Reihenfolge zu ändern. Sie muss **nicht** zusammenhängend sein.

*`ACE` ist Teilfolge von `ABCDE`, `ACE` ist **keine** Teilkette* (substring).

Schritt 1 – Teilproblem und Rekursion.

$L[i][j] :=$ Länge der längsten gemeinsamen Teilfolge von $a[1 \dots i]$ und $b[1 \dots j]$.

$$L[i][j] = \begin{cases} L[i-1][j-1] + 1 & \text{falls } a_i = b_j, \\ \max\big(L[i-1][j],\ L[i][j-1]\big) & \text{sonst.} \end{cases}$$

Die erste Zeile ist die eigentliche Einsicht: *Stimmen die letzten Zeichen überein, so gehören sie **immer** zu einer längsten gemeinsamen Teilfolge.*

Beweis: Gäbe es eine längste, die a_i nicht benutzt, so könnte man a_i und b_j anhängen und erhielte eine längere. ✓

Schritt 2 – Die Tabelle für `AGCAT` und `GAC`.

	ε	G	A	C
ε	0	0	0	0
A	0	0	**1**	1
G	0	**1**	1	1
C	0	1	1	**2**
A	0	1	**2**	2
T	0	1	2	**2**

Länge 2

*Es gibt **mehrere** längste gemeinsame Teilfolgen: `GA`, `GC` und `AC` –* **alle der Länge 2.** *Welche man beim Rückwärtslaufen findet, hängt davon ab, welche Richtung man bei Gleichstand bevorzugt.*

Schritt 3 – Der Speicherbedarf.

Was gebraucht wird	**Speicher**	**Zeit**
nur die Länge	$\mathcal{O}(\min(m,n))$	$\mathcal{O}(mn)$
die Teilfolge, naiv	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$
die Teilfolge, Hirschberg	$\mathcal{O}(\min(\mathbf{m},\mathbf{n}))$	$\mathcal{O}(mn)$

Der Kunstgriff von Hirschberg (1975).

> *Man teilt die erste Folge in der Mitte. Für jede mögliche Teilungsstelle der zweiten Folge berechnet man mit dem sparsamen Zweizeilenverfahren die Länge links und – **rückwärts gerechnet** – die Länge rechts.*

Die Stelle, an der die Summe maximal wird, ist die richtige Teilung. **Dann löst man beide Hälften rekursiv.**
Die Rekursionsgleichung:

$$T(m,n) = \mathcal{O}(mn) + T\Big(\frac{m}{2}, k\Big) + T\Big(\frac{m}{2}, n-k\Big).$$

Die Arbeit halbiert sich auf jeder Ebene, also ist die Summe $\mathcal{O}(mn)\cdot(1+\frac{1}{2}+\frac{1}{4}+\dots) = \mathcal{O}(2mn)$. **Doppelte Zeit, aber linearer Speicher.** ✓

> ***Warum das zählt:*** *Bei zwei Genomsequenzen von je einer Million Basen bräuchte die volle Tabelle* 10^{12} *Zellen* – **mehrere Terabyte.** *Hirschbergs Verfahren braucht wenige Megabyte.*

Es ist das Verfahren, das im Unix-Werkzeug `diff` steckt – *und, in einer Verfeinerung von* MYERS *(1986), in praktisch jedem Versionsverwaltungssystem.*

Myers' Verfahren läuft in $\mathcal{O}((m+n)d)$*, wobei* d *die Zahl der Unterschiede ist* – **bei zwei fast gleichen Dateiversionen also nahezu linear.** *Genau der Fall, der in der Praxis vorkommt.*

Bellmans Optimalitätsprinzip steht in vielen Lehrbüchern; Cormen u. a.[6] entwickeln die dynamische Programmierung besonders systematisch; Skiena[25] zeigt, wie man die Teilprobleme findet; Levitin[18] und Goodrich und Tamassia[12] bieten weitere Beispiele.

Part VII

Aktueller Stand

15 Wo die Algorithmik heute steht

Was fest steht und was sich bewegt

Abgeschlossen	
Vergleichssortieren	$\Theta(n \log n)$, obere und untere Schranke stimmen überein
Kürzeste Wege, $w \geq 0$	Dijkstra mit Fibonacci-Heap ist optimal für den allgemeinen Fall
Union-Find	$\Theta(m\,\alpha(n))$, scharf (Fredman-Saks 1989)
Minimaler Spannbaum	seit 2002 in $\mathcal{O}(m\,\alpha(n))$ – allerdings nichtkonstruktiv
Offen	
P gegen NP	Millenniumsproblem (Band 3)
Matrixmultiplikation	Exponent zwischen 2 und 2,371
Shellsort	optimale Schrittweitenfolge unbekannt
Editierabstand	$\mathcal{O}(n^{2-\varepsilon})$ vermutlich unmöglich

Die vierte Zeile oben ist merkwürdig. BERNARD CHAZELLE gab 2000 ein MST-Verfahren mit $\mathcal{O}(m\,\alpha(n))$ an, und PETTIE und RAMACHANDRAN zeigten 2002 ein **beweisbar optimales** – *dessen genaue Laufzeit aber niemand kennt.*

Man weiß, dass es optimal ist, ohne zu wissen, wie schnell es ist. **Ein Ergebnis, das die eigentümliche Lage des Gebiets gut beschreibt.**

15.1 Die Matrixmultiplikation

Ein Exponent, der seit 1969 sinkt

Die Schulmethode für zwei $n \times n$-Matrizen braucht n^3 Multiplikationen. **Bis 1969 hielt man das für optimal** – *genau wie bei der Zahlenmultiplikation vor Karatsuba.*

VOLKER STRASSEN zeigte 1969, dass sieben Multiplikationen für den 2×2-Fall genügen – *statt acht.*

$$T(n) = 7\,T\left(\frac{n}{2}\right) + \mathcal{O}(n^2) = \Theta(n^{\log_2 7}) = \Theta(n^{2{,}807}).$$

Verfahren	**Exponent**	**Jahr**
Schulmethode	3,000	antik
Strassen	2,807	1969
Pan	2,796	1978
Coppersmith-Winograd	2,376	1990
Williams	2,3729	2012
Alman-Williams	2,3719	2021
Stand 2024	**2,3713**	Duan u. a.
untere Schranke	**2**	trivial (Ausgabegröße)

Seit 1990 hat sich der Exponent um weniger als 0,005 verbessert. Viele vermuten, dass der wahre Wert 2 ist – **bewiesen ist davon nichts.**

Nur Strassen wird tatsächlich benutzt

Verfahren	lohnt ab	Praxis
Strassen	$n \approx 500$	in Bibliotheken
Coppersmith-Winograd	astronomisch	nie
neuere	noch schlimmer	nie

*Man nennt sie **galaktische Algorithmen*** – **sie schlagen die Konkurrenz erst bei Eingabegrößen, die im beobachtbaren Universum nicht vorkommen.**

Und selbst Strassen hat einen Preis: *Er ist numerisch weniger stabil als die Schulmethode, weil er Differenzen bildet, die sich auslöschen können* (Band 0, Kapitel 3). **Deshalb schalten Bibliotheken ihn nur bei großen, gutartigen Matrizen zu.**

> *2022 fand* DeepMinds *System „AlphaTensor“ durch bestärkendes Lernen neue Zerlegungen* – **unter anderem eine für 4×5-Matrizen mit 76 statt 80 Multiplikationen**, *die erste Verbesserung in diesem Format seit fünfzig Jahren.* **Menschliche Forscher verbesserten das Ergebnis binnen Wochen weiter.**

15.2 Was sich seit 2000 verschoben hat

Vier Entwicklungen

Erstens: Das Kostenmodell stimmt nicht mehr.

> *Die klassische Analyse zählt Operationen und nimmt an, jeder Speicherzugriff koste gleich viel.* **Seit den 1990er Jahren ist der Unterschied zwischen Cache und Hauptspeicher größer als der zwischen vielen Algorithmen.**

Daraus sind eigene Modelle entstanden:

Externspeicher	Zählt Blocktransfers statt Operationen (Aggarwal-Vitter 1988)
Cache-oblivious	Verfahren, die **ohne Kenntnis** der Blockgröße optimal sind (Frigo u. a. 1999)

Das zweite ist das elegantere Ergebnis: ***Ein rekursiv aufgebautes Verfahren nutzt* jede *Cacheebene automatisch gut*** – weil die Teilprobleme irgendwann in jede Ebene passen, ganz gleich wie groß sie ist.

Aho, Hopcroft und Ullman[1] legten 1974 den Grundstein dieser Modellfragen; Gonnet und Baeza-Yates[11] sammeln die exakten Analysen, Motwani und Raghavan[16] die randomisierten Verfahren, und Vöcking und Mitautoren[27] stellen fünfundvierzig Algorithmen allgemeinverständlich dar.

Zweitens: Randomisierung ist Standard geworden.

Quicksort	zufälliges Pivot gegen den schlechtesten Fall
Hashing	zufällige Hashfunktion gegen Angriffe (Kapitel 9)
Skip-Listen	zufällige Höhen statt Balancierung
Min-Cut (Karger)	zufälliges Kantenzusammenziehen

Der Grund ist meist nicht Geschwindigkeit, sondern ***Einfachheit*** – **eine randomisierte Skip-Liste ist in zwanzig Zeilen geschrieben, ein Rot-Schwarz-Baum in zweihundert.**

Drittens: Näherungen und Datenströme.

Datenströme	Ein Durchlauf, sublinearer Speicher – HyperLogLog schätzt die Zahl verschiedener Elemente mit wenigen Kilobyte
Sublineare Zeit	Antworten, ohne die Eingabe ganz zu lesen
Skizzen	Bloom-Filter, Count-Min-Sketch (Kapitel 9)

Das ist die Antwort auf Datenmengen, die nicht mehr in den Speicher passen – und die Verschiebung von „exakt und langsam“ zu „fast richtig und sofort“.

Viertens: Gelernte Datenstrukturen.

KRASKA *und Mitarbeiter schlugen 2018 vor, den Index einer Datenbank durch ein* ***gelerntes Modell*** *zu ersetzen: Statt einen B-Baum zu durchlaufen, sagt ein kleines neuronales Netz die ungefähre Position voraus, und man sucht lokal nach.*

Bei gleichmäßig verteilten Schlüsseln ist das eine Interpolationssuche mit gelernter Verteilungsfunktion (Kapitel 5).

Stand August 2026 ist das Gebiet aktiv, aber nicht entschieden. **Die Verfahren gewinnen bei statischen, gutartig verteilten Daten und verlieren bei häufigen Änderungen und schiefen Verteilungen** – *und sie geben die Worst-Case-Garantie auf, die ein B-Baum bietet.*

Stand August 2026 – eine Momentaufnahme

Frage	Status	Bemerkung
Vergleichssortieren	abgeschlossen	$\Theta(n \log n)$
Matrixmult.-Exponent	offen	$[2;\ 2{,}371]$
Ganzzahlmultiplikation	2019 gelöst	$\mathcal{O}(n \log n)$
Shellsort	offen	seit 1959
Editierabstand	bedingt erledigt	unter SETH
P gegen NP	**offen**	Millenniumsproblem
MST optimal	2002	Laufzeit unbekannt
Timsort-Korrektheit	2015 verifiziert	nach gefundenem Fehler

Drei Beobachtungen.
Erstens: Die Grundlagen sind alt und tragen. *Mergesort 1945, Quicksort 1960, Dijkstra 1959, B-Bäume 1970, Union-Find 1975.* **Alles, was in diesem Band steht, war 1980 im Wesentlichen bekannt.**

Zweitens: Die Verbesserungen kommen aus der Praxisbeobachtung, nicht aus der Theorie. *Timsort, Introsort, Cache-oblivious-Verfahren, Contraction Hierarchies* – **alle entstanden, weil jemand gemessen hat, wie sich reale Daten verhalten.**
Drittens: Der Beweis kehrt zurück. *Der Timsort-Fehler von 2015, die verifizierten Bibliotheken, die Formalisierung in Coq und Lean* – **Techniken, die lange als akademisch galten, finden Fehler, die dreizehn Jahre lang niemand fand.**

Schlussgedanke: drei Einsichten

Erstens: Die Struktur entscheidet, nicht die Rechnung.

Aufgabe	naiv	mit Struktur
Suchen	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (Hashing)
Minimum entnehmen	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$ (Heap)
Komponenten vereinigen	$\mathcal{O}(n)$	$\mathcal{O}(\alpha(n))$
Fibonacci	$\mathcal{O}(\varphi^n)$	$\mathcal{O}(n)$

In keinem dieser Fälle wird anders gerechnet. Es wird anders **abgelegt** – und daraus folgt alles. WIRTHS *Gleichung „Algorithmen + Datenstrukturen = Programme“ ist keine Formel, sondern eine Rangordnung.*

Zweitens: Die Analyse ist ein Werkzeug, kein Urteil.

Einfügesortieren $\mathcal{O}(n^2)$	und trotzdem in jeder Bibliothek
Fibonacci-Heap optimal	und trotzdem selten benutzt
Coppersmith-Winograd	und niemals eingesetzt
verkettete Liste $\mathcal{O}(1)$	und langsamer als das Feld

*Die $\mathcal{O}$-Notation sagt, was bei **wachsendem** n geschieht.* **Sie sagt nichts über das n, das man tatsächlich hat, über die Konstanten und über den Speicher.** *Wer sie für ein Urteil hält statt für eine Auskunft, trifft regelmäßig die falsche Wahl.*

Drittens: Ein Verfahren zu verstehen heißt, seine Invariante zu kennen.

Einfügesortieren	$a[1 \ldots i-1]$ ist sortiert
Binäre Suche	Falls vorhanden, liegt es in $[lo, hi]$
Dijkstra	Außerhalb von Q ist $d[v]$ endgültig
Floyd-Warshall	$d[i][j]$ nutzt Zwischenecken bis k
Quicksort	links vom Pivot alles kleiner

In jeder Zeile steht das Verfahren in einem Satz. Wer die Invariante hinschreiben kann, kann den Algorithmus rekonstruieren, seine Korrektheit beweisen und seinen Fehler finden. **Wer nur den Code kennt, kann nichts davon.**

Das ist zugleich die praktische Lehre aus dem Binärsuchfehler (Kapitel 5) *und aus dem Timsort-Fehler* (Kapitel 4): **Beide steckten nicht in der Idee, sondern in der Lücke zwischen Idee und Umsetzung.**

Schlechte Programmierer sorgen sich um den Code.
Gute Programmierer sorgen sich um Datenstrukturen
und ihre Beziehungen.

— LINUS TORVALDS, 2006,
in einer Nachricht an die Git-Entwicklerliste.

Part VIII

Anhang

A Übersicht und Formelsammlung

A.1 Die große Tabelle

Datenstrukturen im Vergleich

Struktur	Suchen	Einfügen	Löschen	Minimum
Feld, unsortiert	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Feld, sortiert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
verkettete Liste	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)^*$	$\mathcal{O}(n)$
Stapel / Schlange	—	$\mathcal{O}(1)$	$\mathcal{O}(1)$	—
Suchbaum, entartet	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
AVL / Rot-Schwarz	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
B-Baum	$\mathcal{O}(\log_B n)$	$\mathcal{O}(\log_B n)$	$\mathcal{O}(\log_B n)$	$\mathcal{O}(\log_B n)$
Skip-Liste	$\mathcal{O}(\log n)^{**}$	$\mathcal{O}(\log n)^{**}$	$\mathcal{O}(\log n)^{**}$	$\mathcal{O}(1)$
Binärheap	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
Fibonacci-Heap	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(\log n)^*$	$\mathcal{O}(1)$
Hashtabelle	$\mathcal{O}(1)^{**}$	$\mathcal{O}(1)^{**}$	$\mathcal{O}(1)^{**}$	$\mathcal{O}(n)$
Union-Find	$\mathcal{O}(\alpha(n))^*$	$\mathcal{O}(1)$	—	—

* *amortisiert* ** *im Mittel bzw. mit hoher Wahrscheinlichkeit*

Die letzte Spalte ist die oft übersehene: *Eine Hashtabelle ist bei jeder Einzeloperation unschlagbar und kennt* ***keine Ordnung*** – kein Minimum, kein Bereichsdurchlauf, keine sortierte Ausgabe. **Das ist der Grund, warum Datenbanken B-Bäume und nicht Hashtabellen als Standardindex benutzen.**

Sortierverfahren im Vergleich

Verfahren	mittel	schlecht.	Speicher	stabil	adaptiv
Auswahl	n^2	n^2	1	nein	nein
Einfügen	n^2	n^2	1	**ja**	**ja**
Blase	n^2	n^2	1	ja	ja
Shell	$n^{4/3}$?	offen	1	nein	teilweise
Merge	$n \log n$	$n \log n$	n	**ja**	nein
Quick	$n \log n$	n^2	$\log n$	nein	nein
Heap	$n \log n$	$n \log n$	1	nein	nein
Introsort	$n \log n$	$n \log n$	$\log n$	nein	nein
Timsort	$n \log n$	$n \log n$	n	**ja**	**ja**
Zählen	$n + k$	$n + k$	$n + k$	ja	—
Radix	$d(n + b)$	$d(n + b)$	$n + b$	ja	—

Die untere Schranke für vergleichsbasiertes Sortieren: $\lceil \log_2(n!) \rceil \approx n \log_2 n - 1{,}44\, n$.

Graphenalgorithmen

Aufgabe	Verfahren	Aufwand
Erreichbarkeit	BFS / DFS	$\mathcal{O}(n+m)$
Kürzeste Wege, ungewichtet	BFS	$\mathcal{O}(n+m)$
Topologische Sortierung	DFS / Kahn	$\mathcal{O}(n+m)$
Zusammenhangskomponenten	DFS	$\mathcal{O}(n+m)$
Starke Komponenten	Tarjan / Kosaraju	$\mathcal{O}(n+m)$
Kürzeste Wege, $w \geq 0$	Dijkstra	$\mathcal{O}(m + n \log n)$
Kürzeste Wege, w beliebig	Bellman-Ford	$\mathcal{O}(n\,m)$
Alle Paare	Floyd-Warshall	$\mathcal{O}(n^3)$
Alle Paare, dünn	Johnson	$\mathcal{O}(nm + n^2 \log n)$
Ein Ziel, mit Schätzer	A*	$\leq$ Dijkstra
Minimaler Spannbaum	Kruskal	$\mathcal{O}(m \log m)$
Minimaler Spannbaum	Prim	$\mathcal{O}(m + n \log n)$
Maximaler Fluss	Edmonds-Karp	$\mathcal{O}(n\,m^2)$
Maximaler Fluss	Push-Relabel	$\mathcal{O}(n^3)$
Bipartites Matching	Hopcroft-Karp	$\mathcal{O}(m\sqrt{n})$

A.2 Formeln und Rekursionen

Was man im Kopf haben sollte

Das Master-Theorem. *Für $T(n) = a\,T(n/b) + f(n)$ und $\kappa = \log_b a$:*

$f(n) = \mathcal{O}(n^{\kappa-\varepsilon})$	$T(n) = \Theta(n^{\kappa})$
$f(n) = \Theta(n^{\kappa})$	$T(n) = \Theta(n^{\kappa} \log n)$
$f(n) = \Omega(n^{\kappa+\varepsilon})$	$T(n) = \Theta(f(n))$

Die Standardrekursionen.

Gleichung	Lösung	Beispiel
$T(n) = T(n/2) + c$	$\Theta(\log n)$	binäre Suche
$T(n) = T(n-1) + c$	$\Theta(n)$	lineare Suche
$T(n) = 2T(n/2) + c$	$\Theta(n)$	Baumdurchlauf
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	Mergesort
$T(n) = T(n-1) + n$	$\Theta(n^2)$	Einfügesortieren
$T(n) = 3T(n/2) + n$	$\Theta(n^{1{,}585})$	Karatsuba
$T(n) = 7T(n/2) + n^2$	$\Theta(n^{2{,}807})$	Strassen
$T(n) = 2T(n-1) + c$	$\Theta(2^n)$	Türme von Hanoi

Nützliche Summen.

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}, \qquad \sum_{k=0}^{n} 2^k = 2^{n+1} - 1, \qquad \sum_{k=0}^{\infty} q^k = \frac{1}{1-q} \; (|q| < 1).$$

$$H_n = \sum_{k=1}^{n} \frac{1}{k} \approx \ln n + 0{,}5772, \qquad \log_2(n!) \approx n \log_2 n - 1{,}4427\, n.$$

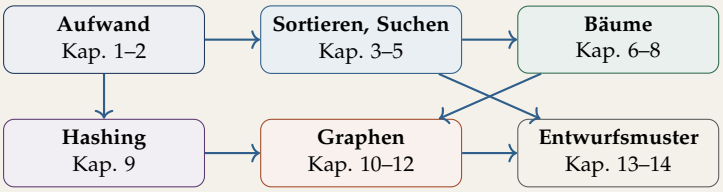
Größenordnungen zum Abschätzen.

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
10	3,3	33	10^2	1024
10^3	10	10^4	10^6	$\approx 10^{301}$
10^6	20	$2 \cdot 10^7$	10^{12}	—
10^9	30	$3 \cdot 10^{10}$	10^{18}	—

Faustregel für den Entwurf: Bei 10^9 Grundoperationen pro Sekunde sind $n \log n$ bis $n = 10^8$ machbar, n^2 bis $n \approx 3 \cdot 10^4$, 2^n bis $n \approx 30$.

A.3 Die Landkarte des Bandes

Aufbau und Querverbindungen



Querverbindungen.

Informatik-Band 0	Zahldarstellung, Überlauf, Speicherhierarchie, Halteproblem
Informatik-Band 3	Komplexität, P gegen NP, NP-schwere Probleme
Informatik-Band 4	Cache, Pipelining – warum die Analyse lügt
Informatik-Band 10	B-Bäume, Indexstrukturen, externes Sortieren
Informatik-Band 11	Graphen und Netzwerkanalyse
Mathematik-Band 0	Logarithmus, Induktion, Summenformeln
Mathematik-Band 3	Grenzwerte, harmonische Reihe
Mathematik-Band 4	Erwartungswert, Linearität, Zufallsvariablen
Mathematik-Band 15	Bäume, Cayley-Formel, Flüsse, Matchings

A.4 Zeittafel

Von Euklid bis zur Gegenwart

≈300 v. Chr.	Euklid	ggT-Algorithmus
≈1650 v. Chr.	Papyrus Rhind	Multiplikation durch Verdoppeln
≈825	al-Chwarizmi	Namensgeber des Fachs
1844	Lamé	erste Laufzeitanalyse
1945	**von Neumann**	**Mergesort**
1946	Mauchly	binäre Suche
1953	Luhn	Hashing (internes IBM-Memo)
1954	Kruskal	minimaler Spannbaum
1956	Ford, Fulkerson	Max-Flow-Min-Cut
1957	Prim	MST (nach Jarník 1930, Borůvka 1926)
1959	**Dijkstra**	**kürzeste Wege**
1959	Shell	Shellsort
1960	**Karatsuba**	**schnelle Multiplikation**
1960/62	**Hoare**	**Quicksort**
1962	Adelson-Velskij, Landis	AVL-Baum
1962	Floyd, Warshall	alle kürzesten Wege
1964	Williams, Floyd	Heap, Heapsort
1965	Levenshtein	Editierabstand
1968	Hart, Nilsson, Raphael	A*
1969	**Strassen**	**Matrixmultiplikation**
1970	Bloom	Bloom-Filter
1970	**Bayer, McCreight**	**B-Baum**
1970	Needleman, Wunsch	Sequenzvergleich
1971	Schönhage, Strassen	$\mathcal{O}(n \log n \log\log n)$
1972	Tarjan	Tiefensuche, starke Komponenten
1972	Edmonds, Karp	polynomielle Flussverfahren
1973	Blum u. a.	Median in linearer Zeit
1975	Tarjan	Union-Find mit Pfadverkürzung
1975	Hirschberg	LCS in linearem Speicher
1975	Wirth	*Algorithmen + Datenstrukturen*

A.5 Glossar

Die Begriffe dieses Bandes

adaptiv	schneller bei vorsortierter Eingabe
amortisiert	über eine Folge von Operationen gemittelt, garantiert
Balancierung	Höhe eines Baums bei $\mathcal{O}(\log n)$ halten
Bloom-Filter	Mengentest mit einseitigem Fehler, sehr sparsam
Entscheidungsbaum	Modell für vergleichsbasierte Verfahren
gierig	lokal beste Wahl, nie zurückgenommen
Hashfunktion	Abbildung von Schlüsseln auf Positionen
Heap	Baum mit Vater-Kind-Ordnung; Prioritätswarteschlange
in situ	nur $\mathcal{O}(1)$ Zusatzspeicher
Inversion	Paar in falscher Reihenfolge
Invariante	Aussage, die jeder Schleifendurchlauf erhält
Kollision	zwei Schlüssel mit gleichem Hashwert
Matroid	Struktur, auf der Gier stets optimal ist
Memoisierung	Zwischenergebnisse speichern statt neu rechnen
$\mathcal{O}$-Notation	Wachstumsordnung, Konstanten ignoriert
Optimalitätsprinzip	Teillösungen optimaler Lösungen sind optimal
Pivot	Trennelement bei Quicksort
Präfixfrei	kein Codewort ist Anfang eines anderen
pseudopolynomiell	polynomiell im Zahlenwert, nicht in der Eingabelänge
Relaxation	Verbesserung einer vorläufigen Entfernung
stabil	gleiche Schlüssel behalten ihre Reihenfolge
Spannbaum	kreisfreier Teilgraph, der alle Ecken verbindet
Union-Find	Struktur für disjunkte Mengen
Zertifikat	Beleg, mit dem sich Optimalität nachprüfen lässt

B Kommentierte Literatur

Achtundzwanzig Titel

Die Auswahl deckt alle Teile des Bandes ab – *Analyse, Sortieren, Datenstrukturen, Graphen und die Entwurfsmuster* – **und enthält fünf Originalarbeiten.** *Die Reihenfolge ist alphabetisch.*

[1] Aho, A. V.; Hopcroft, J. E.; Ullman, J. D.: *The Design and Analysis of Computer Algorithms.* Addison-Wesley 1974.

Das Buch, das das Fach als eigenständige Disziplin begründete. Es führte das RAM-Modell und die systematische Laufzeitanalyse ein. Historisch bedeutsam und immer noch lesenswert.

[2] Bentley, J.: *Programming Pearls.* 2. Aufl., Addison-Wesley 1999.

Kurze Kolumnen über konkrete Programmierprobleme – wie man eine Million Zahlen mit einem Megabyte sortiert, wie man einen Text rotiert, wie man messen statt raten sollte. Das lehrreichste schmale Buch des Fachs.

[3] Bird, R.; Gibbons, J.: *Algorithm Design with Haskell.* Cambridge 2020.

Leitet Algorithmen durch Gleichungsumformung aus ihrer Spezifikation her, statt sie zu erraten und nachträglich zu beweisen. Ein völlig anderer Zugang, der die Muster hinter den Verfahren sichtbar macht.

[4] Bloom, B. H.: *Space/Time Trade-offs in Hash Coding with Allowable Errors.* CACM 13(7), 1970.

Vier Seiten, die eine ganze Familie von Datenstrukturen begründeten. Blooms Anlass war die Silbentrennung: Ein Wörterbuch der Ausnahmen passte nicht in den Speicher.

[5] Blum, M.; Floyd, R. W.; Pratt, V.; Rivest, R. L.; Tarjan, R. E.: *Time Bounds for Selection.* JCSS 7(4), 1973.

Der Median in garantiert linearer Zeit – ein Ergebnis, das damals als unmöglich galt. Fünf Autoren, von denen vier später den Turing-Award erhielten.

[6] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C.: *Introduction to Algorithms.* 4. Aufl., MIT Press 2022.

Das Standardwerk, meist nach den Initialen „CLRS“ zitiert. Über tausend Seiten, alles vollständig bewiesen, mit sorgfältig gestuften Aufgaben. Wenn Sie *ein* Buch zu diesem Band kaufen: dieses.

[7] Dasgupta, S.; Papadimitriou, C.; Vazirani, U.: *Algorithms.* McGraw-Hill 2006.

Der Gegenentwurf zu [6]: dreihundert Seiten statt tausend, knapp und elegant, mit dem Blick auf die Ideen statt auf die Vollständigkeit. Der Entwurf ist frei verfügbar.

[8] Dijkstra, E. W.: *A Note on Two Problems in Connexion with Graphs.* Numerische Mathematik 1, 1959.

Zweieinhalb Seiten, die kürzeste Wege und minimale Spannbäume gleichzeitig behandeln. Dijkstra hielt die Arbeit für nebensächlich; sie ist heute eine der meistzitierten der Informatik.

[9] Erickson, J.: *Algorithms.* Selbstverlag 2019.

Aus jahrzehntelangen Vorlesungsnotizen in Illinois entstanden, frei verfügbar. Ungewöhnlich sorgfältig bei den Korrektheitsbeweisen und mit trockenem Humor geschrieben.

[10] Sedgewick, R.; Flajolet, P.: *An Introduction to the Analysis of Algorithms.* 2. Aufl., Addison-Wesley 2013.

Die *exakte* Analyse statt der $\mathcal{O}$-Notation – mit erzeugenden Funktionen und Asymptotik. Hier stehen die Konstanten, die [6] weglässt.

[11] Gonnet, G. H.; Baeza-Yates, R.: *Handbook of Algorithms and Data Structures.* 2. Aufl., Addison-Wesley 1991.

Eine Sammlung exakter Analysen und Formeln für nahezu jede Struktur dieses Bandes. Zum Nachschlagen, wenn man wissen will, was eine Konstante wirklich ist.

[12] Goodrich, M. T.; Tamassia, R.: *Algorithm Design and Applications.* Wiley 2015.

Anwendungsnah aufgebaut, mit vielen Beispielen aus Netzwerken und Geometrie. Besonders gut bei Hashing und bei den randomisierten Strukturen.

[13] Graham, R. L.; Knuth, D. E.; Patashnik, O.: *Concrete Mathematics.* 2. Aufl., Addison-Wesley 1994.

Das mathematische Handwerkszeug: Summen, Rekursionen, erzeugende Funktionen, Asymptotik. Entstanden aus Knuths Bedürfnis, die Mathematik für TAOCP an einem Ort zu haben.

[14] Hoare, C. A. R.: *Quicksort.* The Computer Journal 5(1), 1962.

Die vollständige Darstellung mit Analyse, nachdem 1961 nur der Algorithmus als halbseitiger Beitrag erschienen war. Bemerkenswert klar für eine Erstveröffentlichung.

[15] Kleinberg, J.; Tardos, É.: *Algorithm Design.* Addison-Wesley 2005.

Das beste Buch über die Entwurfs*muster*. Kleinberg und Tardos zeigen nicht nur die Verfahren, sondern wie man auf sie kommt und wie man Optimalität beweist. Die Kapitel über gierige Verfahren und Flüsse sind vorbildlich.

[16] Motwani, R.; Raghavan, P.: *Randomized Algorithms.* Cambridge 1995.

Das Standardwerk zur Randomisierung – von Quicksort über Hashing bis zu algebraischen Verfahren. Setzt Wahrscheinlichkeitsrechnung voraus (Mathematik-Band 4).

[17] Knuth, D. E.: *The Art of Computer Programming, Vol. 3: Sorting and Searching.* 2. Aufl., Addison-Wesley 1998.

Vierhundert Seiten allein über Sortieren und weitere über Suchen – mit historischer Gründlichkeit, exakten Analysen und Aufgaben in sieben Schwierigkeitsgraden. Unerreicht und unersetzlich.

[18] Levitin, A.: *Introduction to the Design and Analysis of Algorithms.* 3. Aufl., Pearson 2011.

Ordnet die Verfahren konsequent nach Entwurfsmustern statt nach Problemen. Ein didaktisch überzeugender Zugang für den zweiten Durchgang.

[19] Mehlhorn, K.; Sanders, P.: *Algorithms and Data Structures: The Basic Toolbox.* Springer 2008.

Knapp, streng und mit ungewöhnlichem Blick auf die Praxis – Mehlhorn hat mit LEDA eine der ersten ernsthaften Algorithmenbibliotheken gebaut und daraus das Programm der *zertifizierenden* Algorithmen entwickelt.

[20] Motwani, R.; Raghavan, P.; siehe [16].

—

[21] Okasaki, C.: *Purely Functional Data Structures.* Cambridge 1998.

Wie man Datenstrukturen ohne veränderbaren Speicher baut – mit amortisierter Analyse unter Verzögerungsauswertung. Ein Buch, das man auch dann liest, wenn man nie funktional programmiert.

[22] Ottmann, T.; Widmayer, P.: *Algorithmen und Datenstrukturen.* 6. Aufl., Springer 2017.

Die gründlichste deutschsprachige Darstellung, seit Jahrzehnten Standard an deutschen Universitäten. Besonders sorgfältig bei den Baumstrukturen.

[23] Papadimitriou, C. H.; Steiglitz, K.: *Combinatorial Optimization.* Dover 1998.

Ordnet Flüsse, Matchings und Spannbäume in die lineare Optimierung ein und macht die Dualität sichtbar, die hinter allen Min-Max-Sätzen steht.

[24] Sedgewick, R.; Wayne, K.: *Algorithms.* 4. Aufl., Addison-Wesley 2011.

Das anschaulichste Buch des Fachs – mit ausgezeichneten Bildern, lauffähigem Code und systematischen Messungen. Wer sehen will, wie sich die Verfahren *verhalten*, nimmt dieses.

[25] Skiena, S. S.: *The Algorithm Design Manual.* 3. Aufl., Springer 2020.

Zweigeteilt: erst eine knappe Einführung, dann ein „Katalog“ von 75 Problemen mit Hinweisen, welches Verfahren wann taugt. Der zweite Teil ist im Alltag der nützlichste Text, den ich kenne.

[26] Tarjan, R. E.: *Data Structures and Network Algorithms.* SIAM 1983.

Schmal und dicht, von einem der einflussreichsten Autoren des Gebiets. Enthält die amortisierte Analyse von Union-Find und die Grundlagen der Splay-Bäume.

[27] Vöcking, B.; Alt, H.; Dietzfelbinger, M. u. a. (Hrsg.): *Taschenbuch der Algorithmen.* Springer 2008.

Fünfundvierzig Algorithmen, jeder von einem Fachvertreter allgemeinverständlich dargestellt. Aus einem Schülerwettbewerb hervorgegangen und ungewöhnlich gut lesbar.

[28] Wirth, N.: *Algorithmen und Datenstrukturen.* 5. Aufl., Teubner 1999.

Der Titel der Erstausgabe von 1975 lautete „Algorithms + Data Structures = Programs“ und ist zugleich die These dieses Bandes. Knapp, streng, ohne jede Redundanz.

Wenn Sie nur drei Bücher kaufen

[6]	**Cormen u. a.**	zum Nachschlagen – alles, vollständig bewiesen
[15]	**Kleinberg, Tardos**	zum Verstehen – wie man auf die Verfahren kommt
[25]	**Skiena**	zum Arbeiten – welches Verfahren wann taugt

Auf Deutsch nehmen Sie **[22] Ottmann/Widmayer**; **zum Anschauen [24] Sedgewick/Wayne**; **zum Vergnügen [2] Bentley**.

Und wer den Konstanten nachgehen will, die die $\mathcal{O}$-Notation verschweigt, findet sie bei ***[17] Knuth*** **– dort und sonst nirgends.**